UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II



SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA
INGEGNERIA DEL SOFTWARE AA 2023/2024

Specifica, progettazione, implementazione e validazione del sistema "DietiDeals24"

Gruppo INGSW2324_63

Docenti

Sergio Di Martino
Sergio Di Meglio

Autori

Oreste Leone N86/1980
Giuseppe Falso N86/2941

1. Indice

1. Indice....3
2. Revisioni....4
3. Descrizione Progetto....5
4. Documento dei Requisiti Software....7
a. Analisi dei Requisiti....7
i. Requisiti Funzionali....7
ii. Requisiti Non Funzionali....9
iii. Requisiti Dominio....10
iv. Modellazione dei Casi D'Uso....12
v. Tabelle di Cockburn....13
vi. Mock-up Interfaccia Utente....19
vii. Glossario....22
b. Specifica dei Requisiti....27
i. UML Class Diagram di Analisi....27
ii. UML Sequence Diagram di Analisi....35
iii. UML Statechart Diagram di Analisi....38
5. Documento di Design del Sistema....40
a. Analisi dell'Architettura e scelte Tecnologiche....40
i. Architettura Generale e Tecnologie utilizzate....40
ii. Criteri di Design....44
b. UML Class Diagram di Design....50
c. UML Sequence Diagram di Design....57
d. UML Statechart Diagram di Design....60
6. Documento di Testing del Sistema....62
a. Codice xUnit....62
i. Validator.validateEmailAndPassword....62
ii. AuctionService.CreateAuctionAsync....69
iii. NotificationWorker.AddNotificationAsync....80
iv. NotificationWorker.SendNotificationAsync....89

2. Revisioni

Data	Revisione	Descrizione
15/03/25	1.0	Prima Stesura completa
28/03/25	1.1	Ultima revisione

3. Descrizione Progetto

DietiDeals24 è una piattaforma per la gestione di aste online. Il sistema consiste in un'applicazione mobile, desktop o web-based, performante e affidabile, attraverso cui gli utenti possono fruire delle funzionalità del sistema in modo intuitivo, rapido e piacevole.
Le principali funzionalità offerte da DietiDeals24 sono indicate di seguito (funzionalità 1, 2, 3, 5, 6) :

- Un utente può registrare un nuovo account ed utilizzarlo per accedere al sistema. Un account può essere di due tipi: venditore o acquirente. La stessa e-mail può essere utilizzata per al più un account venditore e un account compratore. È apprezzata la possibilità di effettuare la registrazione utilizzando credenziali di terze parti (e.g.: Google, Facebook, GitHub, etc…). Gli utenti possono personalizzare il proprio profilo con una short bio, link al proprio sito web / ai propri social, area geografica, etc.

- Il sistema permette ai venditori/compratori di creare aste di diverso tipo per la vendita/acquisto di beni/servizi e presentare offerte per le aste correntemente attive. Ciascuna asta è caratterizzata da una descrizione del bene/servizio in vendita e, opzionalmente, da una fotografia dello stesso. Ciascuna asta inoltre è caratterizzata da una categoria (e.g.: elettronica, informatica, giocattoli, alimentari, servizi, etc…), introdotta per facilitare la navigazione tra le tante aste presenti nel sistema.

- Venditori e compratori possono effettuare ricerche tra le aste correntemente attive, filtrando per categoria e/o per parole chiave, e visualizzare i dettagli di ciascuna asta. Inoltre, è possibile visualizzare anche il profilo utente del venditore che ha creato l'asta.

- Un venditore può creare una nuova Asta all'inglese (o incrementale). Un'asta all'inglese è caratterizzata da una base d'asta iniziale pubblica, specificata dal venditore al momento della creazione dell'asta, da un intervallo di tempo fisso per presentare nuove offerte (di default 1 ora), e da una soglia di rialzo (di default 10€). Gli acquirenti possono presentare un'offerta per il prezzo corrente. Quando viene presentata un'offerta, il timer per la presentazione di nuove offerte viene resettato. Quando il

timer raggiunge lo zero senza che siano presentate nuove offerte, l'ultima offerta si aggiudica il bene/servizio in vendita, e il venditore e gli acquirenti che hanno partecipato all'asta visualizzano una notifica

- Un venditore può creare una nuova Asta al ribasso. Un'asta al ribasso è caratterizzata da un prezzo iniziale elevato specificato dal venditore, da un timer per il decremento del prezzo (di default 1 ora), da un importo, in €, per ciascun decremento, e da un prezzo minimo (segreto) a cui vendere il prodotto/servizio. Il prodotto/servizio sarà in vendita al prezzo iniziale stabilito dal venditore. Al raggiungimento del timer, il prezzo verrà decrementato dell'importo previsto. Il primo compratore a presentare un'offerta si aggiudica il prodotto/servizio. Se il prezzo viene decrementato fino a raggiungere il prezzo minimo segreto, l'asta viene considerata fallita e il venditore visualizza una notifica.

Tutte le funzionalità assegnate devono essere realizzate implementando un sistema distribuito in cui è possibile individuare due macro-componenti:

- una parte front-end, comprensiva delle interfacce utente;
- una parte back-end, che espone delle interfacce di programmazione (e.g.: REST API) che vengono utilizzate dal front-end.

È altresì auspicabile che il back-end sia distribuito come container Docker e che, al momento della discussione del progetto con demo del prodotto, sia messo in opera utilizzando tecnologie allo stato dell'arte quali ad esempio servizi di public Cloud Computing come Azure o AWS, e sia quindi accessibile attraverso la rete Internet.

4. Documento dei Requisiti Software

a. Analisi dei Requisiti

i. Requisiti Funzionali

ID	RF1
Nome	Registrazione Utente
Descrizione	Un utente può registrare un nuovo account sulla piattaforma. La registrazione può essere effettuata inserendo email, username e password.

ID	RF2
Nome	Gestione Account
Descrizione	Un account utente può essere di due tipi: venditore o acquirente. La stessa e-mail può essere utilizzata per al più un account venditore e un account compratore. Gli utenti possono personalizzare il proprio profilo con una short bio, link al proprio sito web/ai propri social, area geografica, ecc.

ID	RF3
Nome	Creazione di Aste
Descrizione	I venditori possono creare aste di diverso tipo per la vendita di beni o servizi.

ID	RF4
Nome	Partecipazione ad Aste
Descrizione	Gli acquirenti possono presentare offerte per le aste correntemente attive.

ID	RF5
Nome	Descrizione e Classificazione delle Aste
Descrizione	Ogni asta deve includere una descrizione del bene o servizio in vendita e, opzionalmente, una fotografia. Ogni asta deve essere classificata in una categoria per facilitare la navigazione.

ID	RF6
Nome	Ricerca e Filtraggio Aste
Descrizione	Gli utenti possono effettuare ricerche tra le aste attive, filtrando per categoria e/o parole chiave, e visualizzare i dettagli di ciascuna asta.

ID	RF7
Nome	Visualizzazione del Profilo Utente Venditore
Descrizione	Gli utenti possono visualizzare il profilo del venditore che ha creato un’asta.

ID	RF8
Nome	Asta all'Inglese
Descrizione	I venditori possono creare aste all'inglese con base d'asta iniziale, intervallo di tempo fisso per le offerte e soglia di rialzo. Gli acquirenti possono fare offerte e, quando il timer raggiunge zero senza nuove offerte, l'ultima offerta vince. Notifica al venditore e agli acquirenti partecipanti.

ID	RF9
Nome	Asta al Ribasso
Descrizione	I venditori possono creare aste al ribasso con prezzo iniziale elevato, timer per il decremento del prezzo, importo per ciascun decremento e prezzo minimo segreto. Se il prezzo scende fino al prezzo minimo segreto, l'asta fallisce e il venditore riceve una notifica.

ID	RF10
Nome	Notifiche
Descrizione	Il sistema invia notifiche ai venditori e agli acquirenti partecipanti su eventi rilevanti, come l'aggiudicazione di un’asta o l’annullamento di un’asta al ribasso.

ii. Requisiti Non Funzionali

ID	RNF1
Nome	Performante e Affidabile
Descrizione	L'applicazione deve essere performante e affidabile, garantendo un'esperienza utente rapida e senza interruzioni.

ID	RNF2
Nome	Accessibilità Piattaforma
Descrizione	L'applicazione deve essere accessibile da dispositivi mobili, desktop o tramite web.

ID	RNF3
Nome	Interfacce Utente Intuitive
Descrizione	Le interfacce utente (UI) devono essere intuitive, consentendo agli utenti di navigare e utilizzare tutte le funzionalità in modo facile e piacevole.

ID	RNF4
Nome	Architettura Distribuita
Descrizione	Il sistema deve essere implementato come un sistema distribuito, con una parte front-end per le interfacce utente e una parte back-end che espone le API REST utilizzate dal front-end.

ID	RNF5
Nome	Containerizzazione
Descrizione	Il back-end deve essere distribuito come container Docker per facilitare il deployment e la scalabilità.

ID	RNF6
Nome	Deploy su Public Cloud
Descrizione	Durante la demo del progetto, il back-end deve essere distribuito utilizzando servizi di public Cloud Computing, come Azure o AWS, e deve essere accessibile attraverso la rete Internet.

iii. Requisiti Dominio

ID	RD1
Nome	Account unico per tipo
Descrizione	Ogni e-mail può essere associata ad al più un account venditore e un account acquirente.

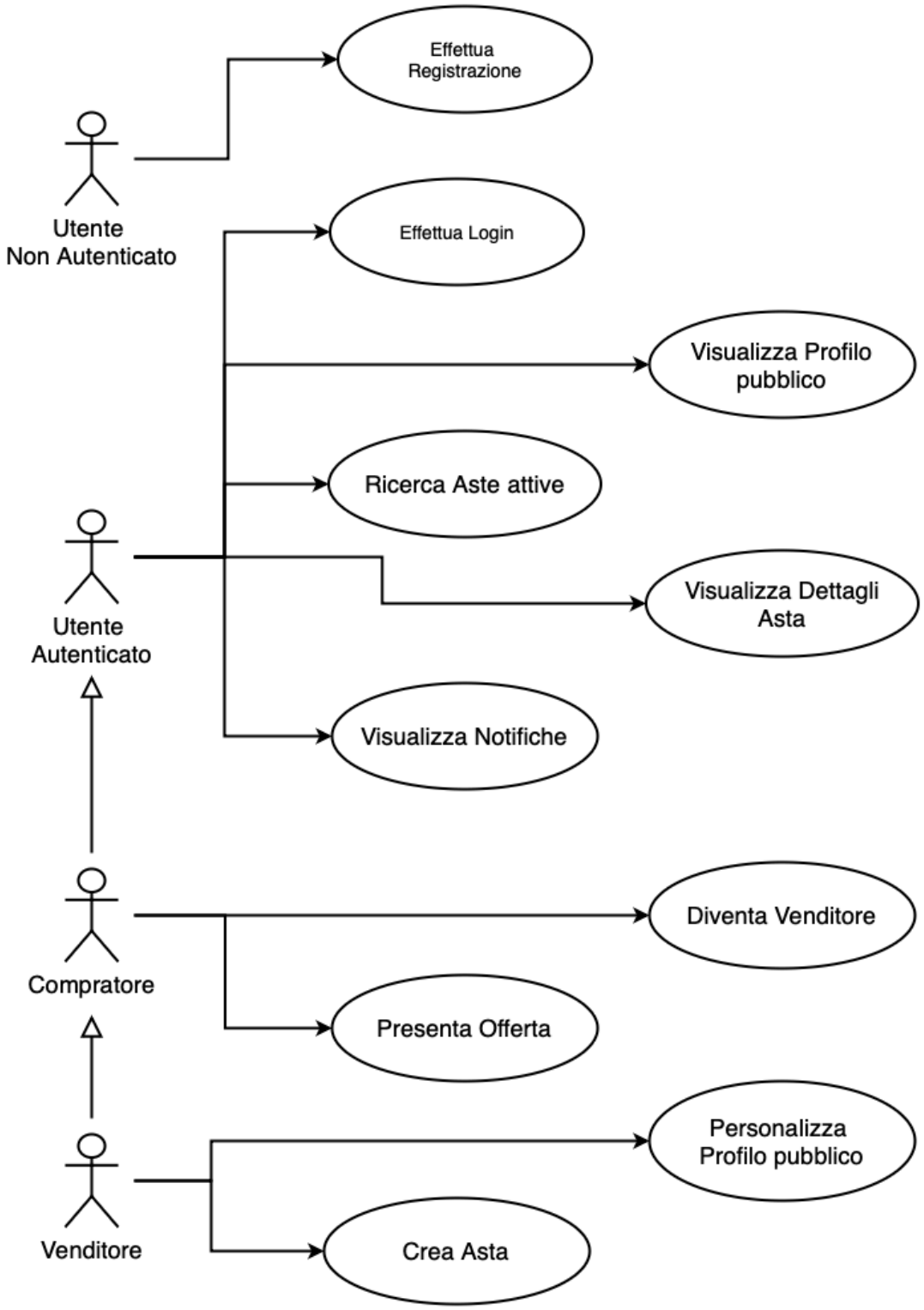
ID	RD2
Nome	Categorie Aste
Descrizione	Le aste devono essere classificate in categorie specifiche (es. elettronica, informatica, giocattoli, alimentari, servizi) per facilitare la navigazione e la ricerca da parte degli utenti

ID	RD3
Nome	Asta all'Inglese
Descrizione	Un’asta incrementale che prevede offerte in rialzo partendo da una base d’asta iniziale, con timer che si resetta a ogni nuova offerta e una soglia minima di rialzo.

ID	RD4
Nome	Asta al Ribasso
Descrizione	Un tipo di asta in cui il prezzo di partenza è elevato e diminuisce a intervalli regolari di un importo stabilito, fino a quando un acquirente fa un'offerta o il prezzo raggiunge il minimo segreto fissato dal venditore.

ID	RD5
Nome	Timer e Decremento dell'Asta
Descrizione	Il timer associato a ogni tipo di asta (inglese o al ribasso) definisce la durata delle offerte o il momento del decremento del prezzo, influenzando così la strategia degli utenti durante le aste.

iv. Modellazione dei Casi D'Uso



Use Case Diagram DietiDeals24

v. Tabelle di Cockburn

USE CASE	**#1 - Creazione Asta**		
Goal in Context	Creazione e Pubblicazione dell'Asta		
Precondition	L'utente ha il ruolo di venditore		
Success End Condition	Il venditore ha creato e pubblicato una nuova asta.		
Failed End Condition	Il venditore ha annullato la creazione; Si verifica un errore in fase di creazione.		
Primary Actor	Venditore		
Trigger	Il Venditore preme su "Crea Asta" dalla Tab "Vendi" **M6B_Vendi**.		
Descrizione	**Step #**	**Venditore**	**Sistema**
	1	Preme su "Crea asta" in **M6B_Vendi**	
	2		Mostra schermata **M6B_CreaAstaBase**
	3	Inserisce i dati obbligatori: Titolo, descrizione e categoria	
	4		Abilita il pulsante "Avanti"
	5	Preme "Avanti"	
	6		Mostra **M6B_CreaAstaDettagli**
	7	Compila i campi necessari	
	8		Abilita il pulsante "Avanti"
	9	Preme "Avanti"	
	10		Mostra **M6B_AnteprimaAsta**
	11	Visualizza riepilogo e preme "Pubblica Annuncio"	
	12		Mostra **M6b_AstaCreata** in caso di pubblicazione avvenuta con successo.

	13	Scorre per chiudere la modale	
	14		Chiude la schermata e torna su **M6B_Vendi**
Extension	**Step #**	**Venditore**	**Sistema**
#1: Errore in fase di pubblicazione	12E.1.1		Mostra finestra di dialogo con descrizione errore in fase di pubblicazione
	12E.1.2	Preme “Chiudi”	
	12E.1.3		Chiude la finestra di dialogo
#2: Venditore annulla la creazione	3E.2.1	Preme “Annulla”	
	3E.2.2		Chiude la schermata e torna alla tab Vendi
Subvariations	**Step #**	**Venditore**	**Sistema**
#1: Inserimento immagini	3V.1	Preme su “Aggiungi Foto”	
	3V.2		Mostra schermata **Picker Galleria di sistema**
	3V.3	Seleziona le foto	
	3V.4		Abilita pulsante Aggiungi
	3V.5	Preme su “Aggiungi”	
	3V.6		Carica le immagini e ritorna alla schermata aggiornata **M6B_CreaAstaBase**
#2: Cambio Tipologia Asta	7V.1	Seleziona tipologia asta	
	7V.2		Mostra campi relativi alla tipologia
	7V.3	Ricompila eventuale campo vuoto	

USE CASE	**#2 - Visualizza dettagli Asta**		
Goal in Context	Visualizzare i dettagli di un Asta		
Precondition	È presente sulla schermata una card o una notifica relativa ad un Asta		
Success End Condition	L'utente visualizza i dettagli dell'asta desiderata		
Failed End Condition	Viene presentato un alert.		
Primary Actor	Utente		
Trigger	Pressione sulla card dell'Asta o sulla notifica relativa ad un asta.		
Description	Step #	Utente	Sistema
	1	L'utente preme sulla card di un asta	
	2		Carica i dettagli dell'asta e mostra **M10A_DettaglioAsta**
Extensions	Step #	Utente	System
#1: Errore in fase di caricamento dati asta	E1.2.1		Mostra alert di errore
	E1.2.2	Preme sul pulsante "Chiudi"	
	E1.2.3		Chiude l'alert
#2: L'utente accede al dettaglio asta tramite Notifica	E2.1.1	L'utente preme sulla notifica relativa ad un Asta	
	E2.1.2		Carica i dettagli dell'asta e mostra **M10A_DettaglioAsta**
Subvariations	Step #	Utente	Sistema

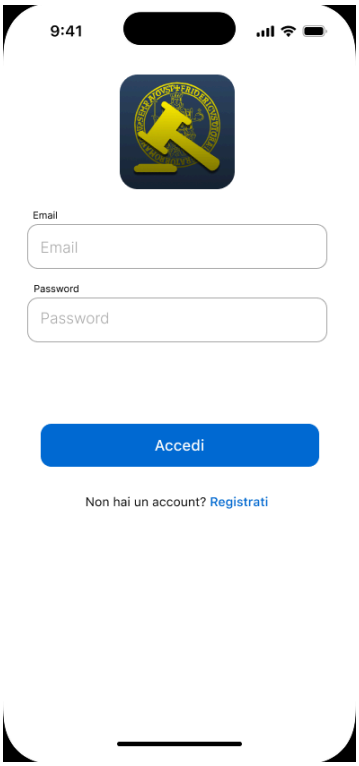
USE CASE	**#3 - Ricerca Aste Attive**		
Goal in Context	Visualizzare le aste attualmente in corso di validità.		
Precondition	Essere autenticati, visibilità della schermata Esplora o Cerca		
Success End Condition	L'utente visualizza le aste in corso di validità.		
Failed End Condition	Non ci sono aste attive oppure non viene restituita correttamente la lista delle aste.		
Primary Actor	Utente		
Trigger	L'utente entra nelle schermate Esplora (**M4_Esplora**) o Cerca (**M5A_Cerca**)		
Description	**Step #**	**User**	**System**
	1	Tap sulla tab Esplora o Cerca	
	2		Mostra la Tab corrispondente
	3	Preme sulla barra di ricerca.	
	4		Mostra la tastiera .
	5	Digita il termine di ricerca.	
	6		Mostra **M5B_Cerca_Risultati**.
Extensions	**Step #**	**User**	**System**
#1 Applica Filtro / Ordinamento	6E.1	Preme su una tipologia di filtro.	
	6E.2		Mostra **M5C_ModaleFiltri**.
	6E.3	Seleziona un filtro o ordinamento dalla schermata modale.	
	6E.4	Preme Applica.	
	6E.5		Mostra **M5B_Cerca_Risultati**.

Subvariations	Step #	Utente	Sistema
#1 Effettua ricerca su categoria	1V.1	Nella Tab di Cerca tappa su una delle categorie senza termine di ricerca	
	1V.2		Mostra **M5B_Cerca_Risultati**.

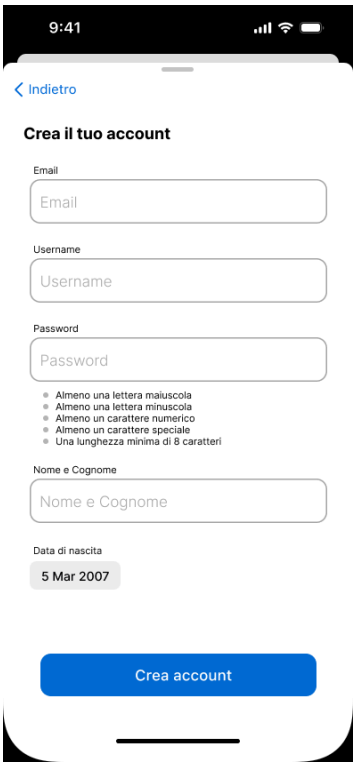
USE CASE	#3 - Presenta Offerta		
Goal in Context	Presentare un'offerta per l'asta.		
Precondition	L'utente visualizza un asta non di sua proprietà.		
Success End Condition	L'utente riesce a presentare correttamente l'offerta.		
Failed End Condition	Si ha un errore durante l'invio dell'offerta.		
Primary Actor	Utente non proprietario dell'asta		
Trigger	Premere il bottone "Fai un'offerta".		
Description	**Step #**	**User**	**System**
	1	Preme il pulsante Fai un offerta	
	2		Mostra **M10B_PresentaOfferta**.
	3	Inserisce l'offerta nel campo "La tua offerta".	
	4		Abilita pulsante offri
	5	Preme "Offri".	
	6		Notifica venditore della nuova offerta e Mostra Dettaglio asta aggiornato
Extensions	**Step #**	**User**	**System**

#1 Errore Offerta	6E.1		Mostra “PopUp Errore”.
	6E.2	Preme “Chiudi”.	
	6E.3		Mostra Dettaglio asta aggiornato.
Subvariations	**Step #**	**Utente**	**Sistema**
#1 Asta al Ribasso	3V.1	Preme “Offri”.	
	3V.2		Manda Notifica vittoria e conclusione asta e mostra Dettaglio Asta Aggiornato

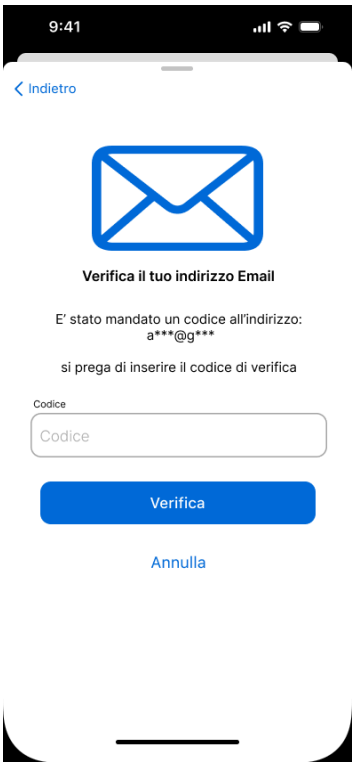
vi. Mock-up Interfaccia Utente



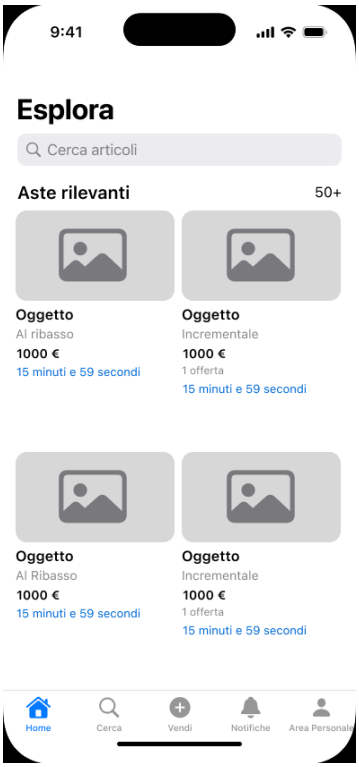
M1_Accedi



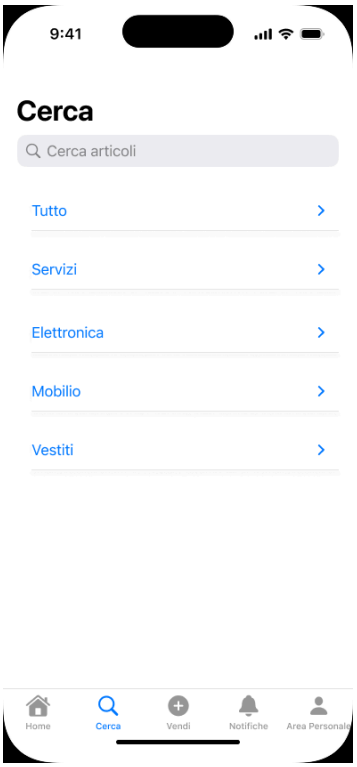
M2_CreaAccount



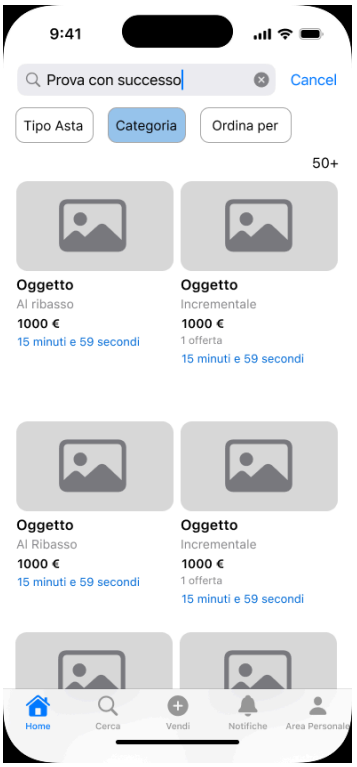
M3_ConfermaEmail



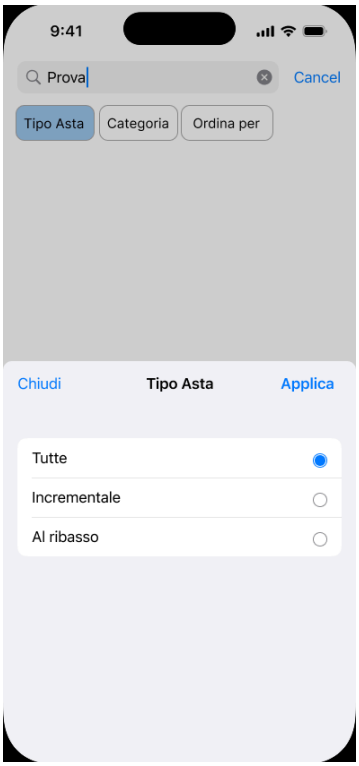
M4_Esplora



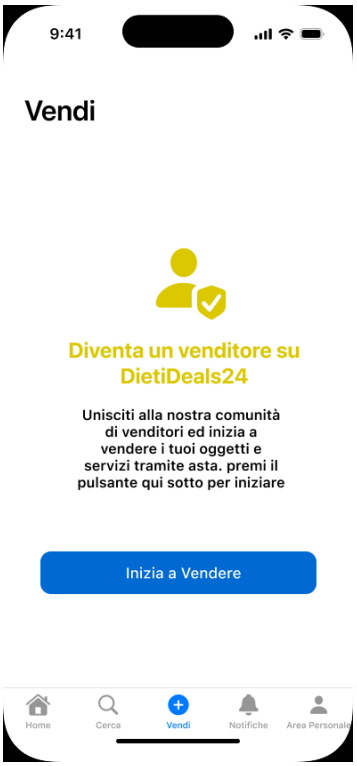
M5A_Cerca



M5B_Cerca_Risultati



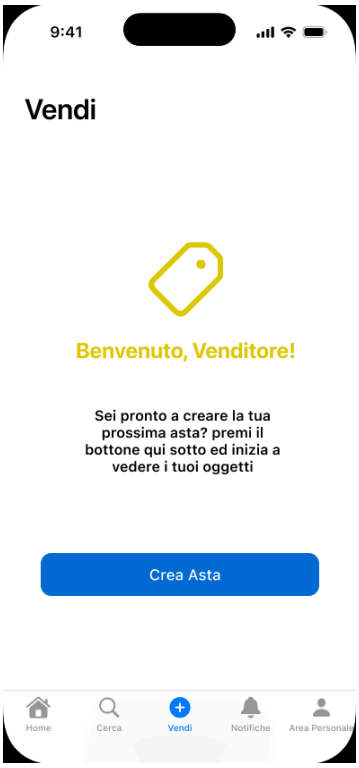
M5C_ModaleFiltri



M6A_DiventaVenditore



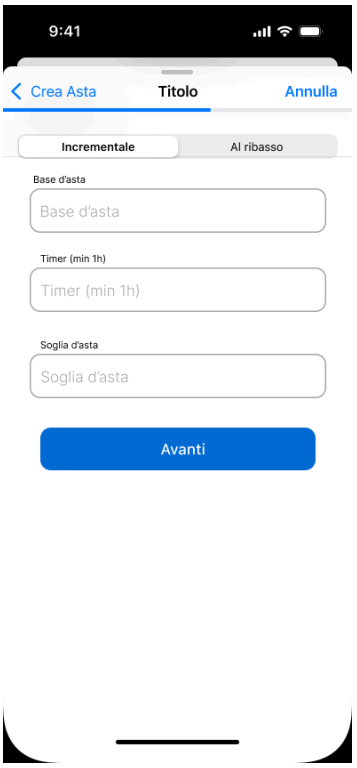
M6A_InserisciDatiVenditore



M6B_Vendi



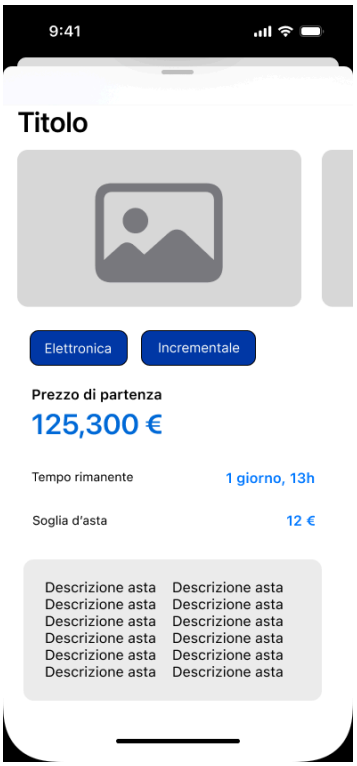
M6B_CreaAstaBase



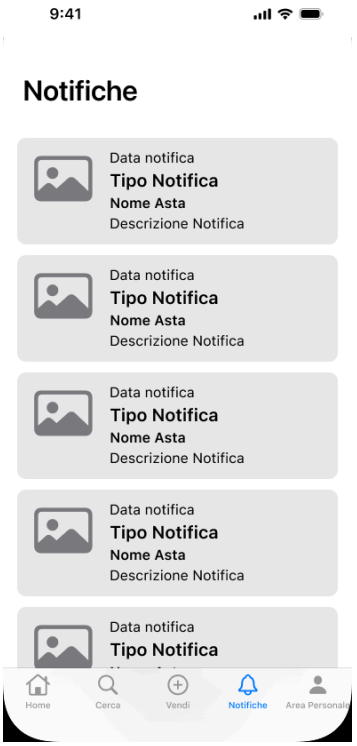
M6B_CreaAstaDettagli



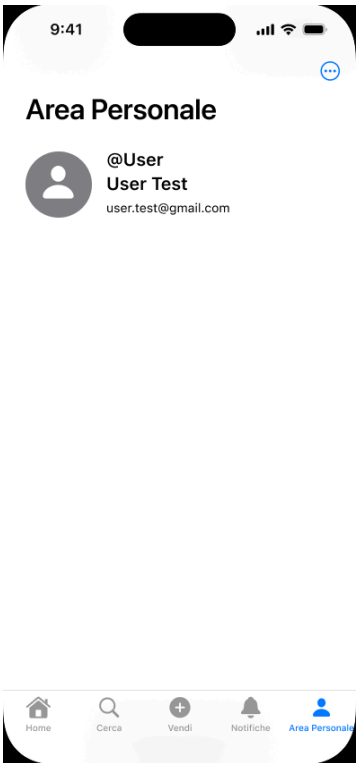
M6B_AnteprimaAsta



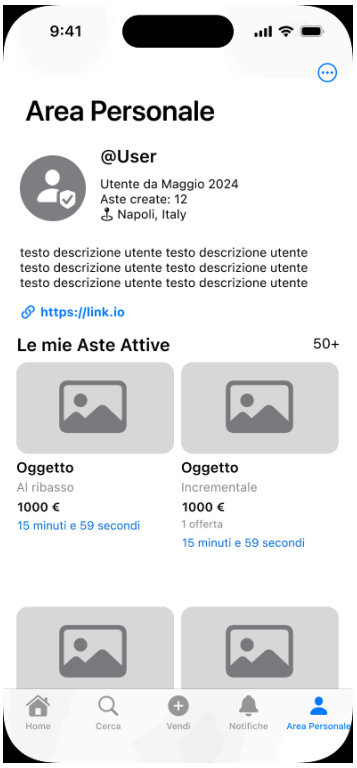
M6B_AstaCreata



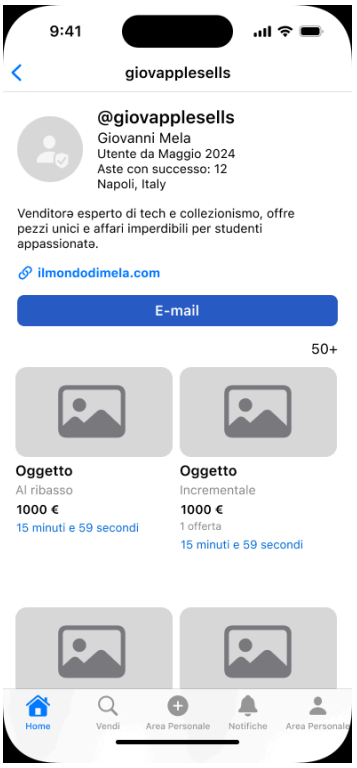
M7_Notifiche



M8A_AreaPersonaleCompratore



M8B_AreaPersonaleVenditore



M9_ProfiloVenditore_Pubblico



M10A_DettaglioAsta



M10A_DettaglioAsta2



M10B_PresentaOfferta

vii. Glossario

Termine	Significato
ACID	Un insieme di proprietà (Atomicità, Coerenza, Isolamento, Durabilità) che garantiscono l'affidabilità delle transazioni nei database, assicurando che vengano eseguite in modo sicuro e corretto anche in caso di guasti.
API (Application Programming Interface)	Un insieme di definizioni e protocolli per la costruzione e l'integrazione di software applicativo.
APNS (Apple Push Notification Service)	Il servizio di notifica push di Apple che permette alle applicazioni iOS, iPadOS, macOS, watchOS e tvOS di ricevere notifiche da server remoti. APNS funge da intermediario tra il server dell'app e il dispositivo dell'utente, garantendo la consegna sicura e ottimizzata delle notifiche. Per inviare notifiche tramite APNS, è necessario registrare l'app con Apple, ottenere i relativi certificati di autenticazione e utilizzare i token di dispositivo univoci per indirizzare correttamente i messaggi.

Termine	Significato
Architettura del Sistema	Struttura organizzativa di un sistema software, comprensiva di componenti software, relazioni tra componenti e proprietà di entrambe.
Asta al Ribasso	Un tipo di asta in cui il prezzo di partenza di un bene o servizio è elevato e viene progressivamente ridotto a intervalli regolari fino a quando un partecipante accetta il prezzo corrente, acquistando così l'oggetto. Se il prezzo diventa inferiore al minimo segreto impostato dal venditore, l'asta si conclude e viene considerata fallita.
Asta all'inglese (Incrementale)	Un tipo di asta in cui il prezzo di un bene o servizio viene progressivamente aumentato attraverso offerte incrementali da parte dei partecipanti entro un certo periodo di tempo. L'asta continua finché non scade il timer e l'oggetto viene aggiudicato all'offerente con la somma più alta. L'asta si svolge in tempo reale, e i partecipanti possono vedere le offerte degli altri, aumentando gradualmente il prezzo offerto.
Autenticazione	Il processo di verifica dell'identità di un utente, dispositivo o altro sistema.
Autorizzazione	Il processo di concedere o negare l'accesso alle risorse del sistema basato sull'identità autenticata.
AWS (Amazon Web Services)	Una piattaforma di cloud computing fornita da Amazon che offre una vasta gamma di servizi di infrastruttura cloud, come calcolo, archiviazione e rete.
Backend	La parte di un sistema software che gestisce la logica dell'applicazione, l'elaborazione dei dati e l'interazione con il database. Espone API per il frontend e altri servizi, eseguendo operazioni come autenticazione, archiviazione e gestione delle richieste degli utenti.
Categoria d'asta	Un'etichetta o classificazione assegnata a ciascuna asta per facilitare la navigazione e la ricerca (e.g., elettronica, giocattoli, alimentari).
CI/CD (Continuous Integration/Continuous Deployment)	Un insieme di pratiche per automatizzare il processo di test, integrazione e deployment del software, migliorando l'efficienza dello sviluppo.
CloudWatch	Un servizio di monitoraggio e gestione di AWS che fornisce dati e approfondimenti sulla performance delle applicazioni, sull'utilizzo delle risorse e altro.
Container	Un'unità software standardizzata che include il codice dell'applicazione e tutte le sue dipendenze (librerie, runtime, configurazioni) in un ambiente isolato e portabile. I container garantiscono che

Termine	Significato
	l’applicazione funzioni in modo coerente su qualsiasi infrastruttura, indipendentemente dal sistema operativo sottostante. Sono leggeri, avviabili rapidamente e ideali per il deployment in ambienti cloud e architetture microservizi.
Database	Un insieme organizzato di dati, solitamente gestito e accessibile tramite un sistema di gestione di database (DBMS).
Deployment	Il processo di rilascio e distribuzione di un'applicazione o servizio in un ambiente di produzione, rendendolo accessibile agli utenti finali.
Device Token	Un identificatore univoco assegnato da Apple Push Notification Service (APNS) a un dispositivo iOS, iPadOS, macOS, watchOS o tvOS quando un'app registra il supporto per le notifiche push. Il device token è un insieme di dati binari che il backend dell'app deve memorizzare e utilizzare per indirizzare correttamente le notifiche push a un dispositivo specifico. Poiché il device token può cambiare nel tempo (ad esempio, dopo un aggiornamento del sistema operativo o la reinstallazione dell’app), è importante aggiornarlo periodicamente nel backend.
Docker	Una piattaforma software che permette di creare, distribuire e gestire applicazioni all'interno di container, isolando le applicazioni dal loro ambiente.
EC2 (Elastic Compute Cloud)	Un servizio AWS che fornisce capacità di calcolo scalabile nel cloud, consentendo di avviare e gestire server virtuali (istanze) in base alle esigenze.
Event Bridge	Servizio di gestione degli eventi completamente gestito di AWS che permette di connettere diverse applicazioni utilizzando eventi in tempo reale. Consente di instradare eventi da sorgenti AWS, applicazioni SaaS o applicazioni personalizzate verso servizi AWS come Lambda, SQS, SNS e altri, facilitando l'implementazione di architetture event-driven senza dover gestire l'infrastruttura sottostante.
Frontend	La parte di un sistema software con cui l'utente interagisce direttamente, responsabile dell'interfaccia utente e dell'esperienza utente. Comunica con il backend per recuperare e inviare dati.
IAM (Identity and Access Management)	Un servizio AWS che consente di gestire in modo sicuro l'accesso ai servizi e alle risorse AWS.

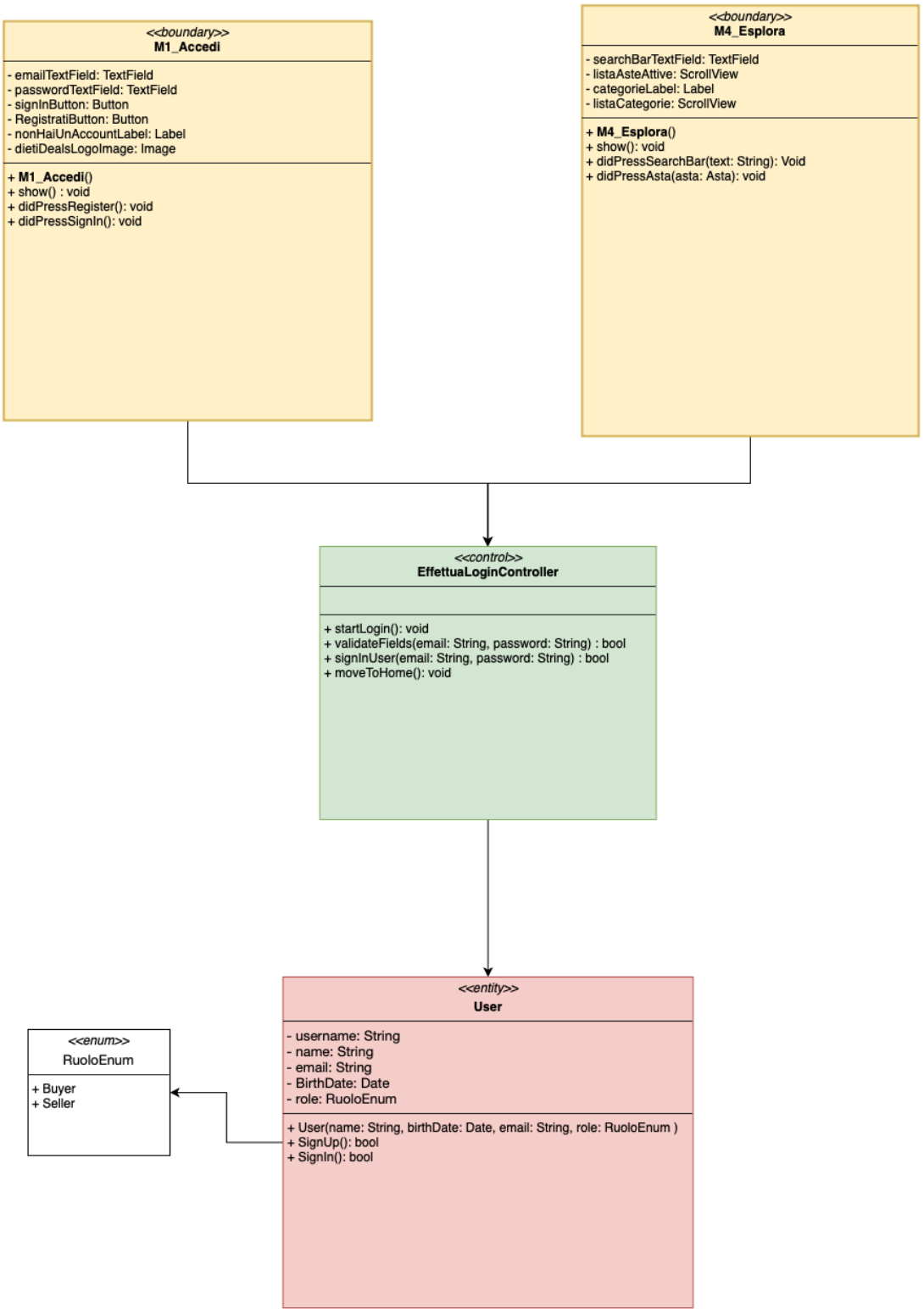
Termine	Significato
Interfaccia Utente (UI - User Interface)	La parte del software che permette agli utenti di interagire con il sistema.
JSON (JavaScript Object Notation)	Un formato leggero per lo scambio di dati, basato su testo leggibile, strutturato in coppie chiave-valore e liste, ampiamente utilizzato nelle API web.
Keychain	Un servizio di Apple che consente di archiviare e sincronizzare in modo sicuro dati sensibili, come password, chiavi crittografiche e token di autenticazione, tra i dispositivi di un utente tramite iCloud. Utilizza la crittografia end-to-end per garantire che solo il proprietario dell'account possa accedere ai dati memorizzati. In applicazioni iOS, il Keychain può essere utilizzato per conservare credenziali di accesso e altre informazioni riservate, riducendo il rischio di esposizione a minacce esterne.
Lambda	Un servizio di AWS che consente di eseguire codice in risposta a eventi senza dover gestire server, utilizzando un modello di calcolo serverless.
Notifica	Un messaggio di avviso che viene inviato agli utenti per informarli su eventi rilevanti, come l'aggiudicazione di un'asta o l'annullamento di un'asta al ribasso al raggiungimento del prezzo minimo segreto.
Push Notification	Un meccanismo che consente a un'applicazione di ricevere messaggi o aggiornamenti da un server remoto anche quando l'app non è attiva. Le notifiche push possono essere utilizzate per informare gli utenti di nuovi eventi, messaggi o cambiamenti nello stato dell'applicazione. Su iOS, le notifiche push vengono gestite da Apple Push Notification Service (APNS), che consegna i messaggi ai dispositivi registrati in modo affidabile ed efficiente.
RDS (Relational Database Service)	Un servizio AWS che semplifica l'impostazione, l'operazione e il ridimensionamento di database relazionali nel cloud.
RESTful	Un'architettura per servizi web basata sui principi REST (Representational State Transfer), che utilizza metodi HTTP standard per operazioni su risorse, garantendo scalabilità e semplicità.
S3 (Simple Storage Service)	Un servizio di archiviazione di oggetti di AWS che offre scalabilità, disponibilità e sicurezza per l'archiviazione di dati non strutturati nel cloud.

Termine	Significato
Soglia	L’importo minimo di incremento del prezzo che un acquirente deve superare per fare una nuova offerta valida in un’asta all’inglese.
Timer	Un conto alla rovescia che determina il tempo rimanente per presentare offerte in un’asta all’inglese o per il decremento del prezzo in un’asta al ribasso.
VPC (Virtual Private Cloud)	Un servizio AWS che permette di lanciare risorse AWS in una rete virtuale isolata, fornendo controllo su ambiente di rete virtuale, inclusi intervalli di indirizzi IP.

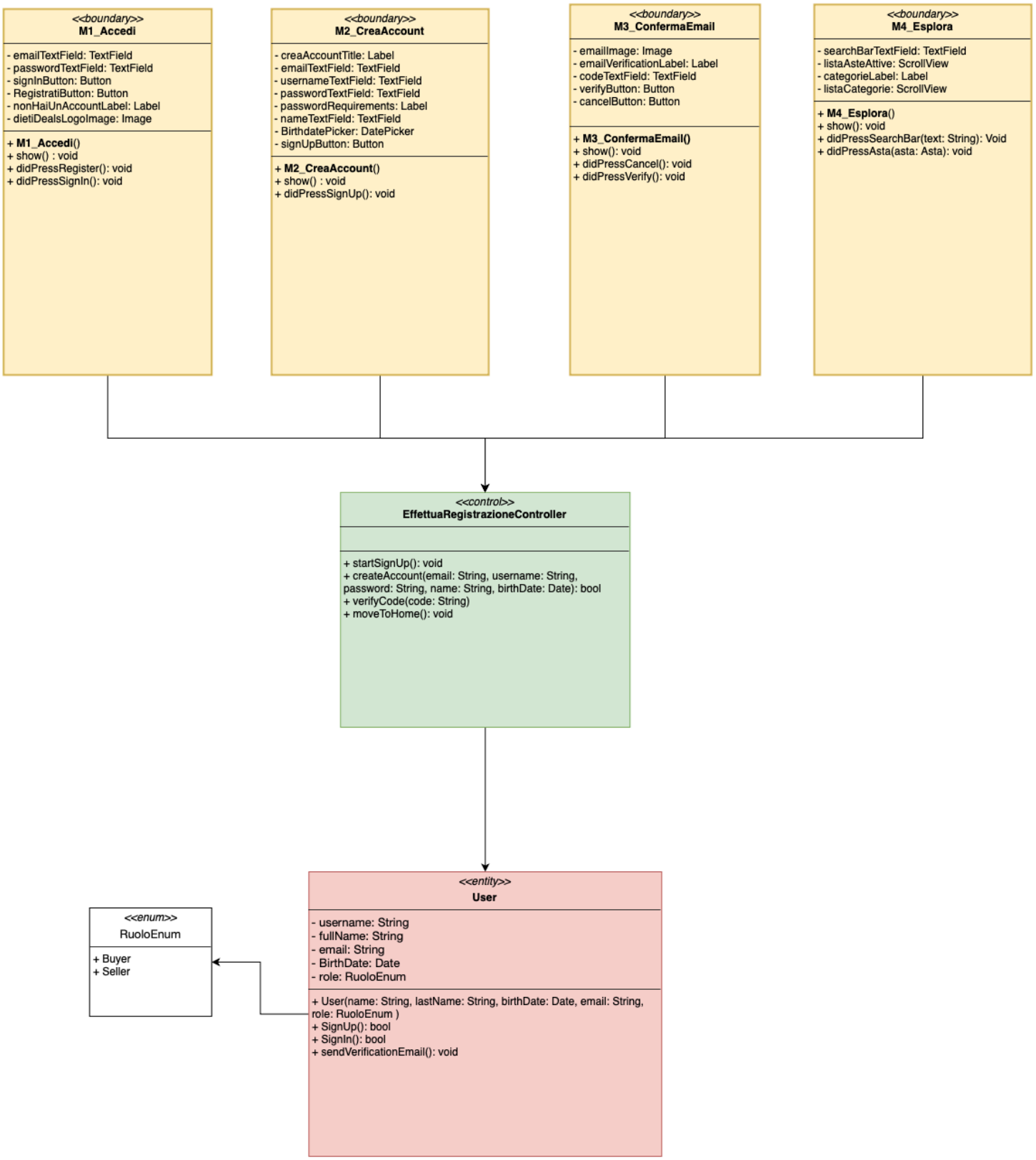
b. Specifica dei Requisiti

i. UML Class Diagram di Analisi

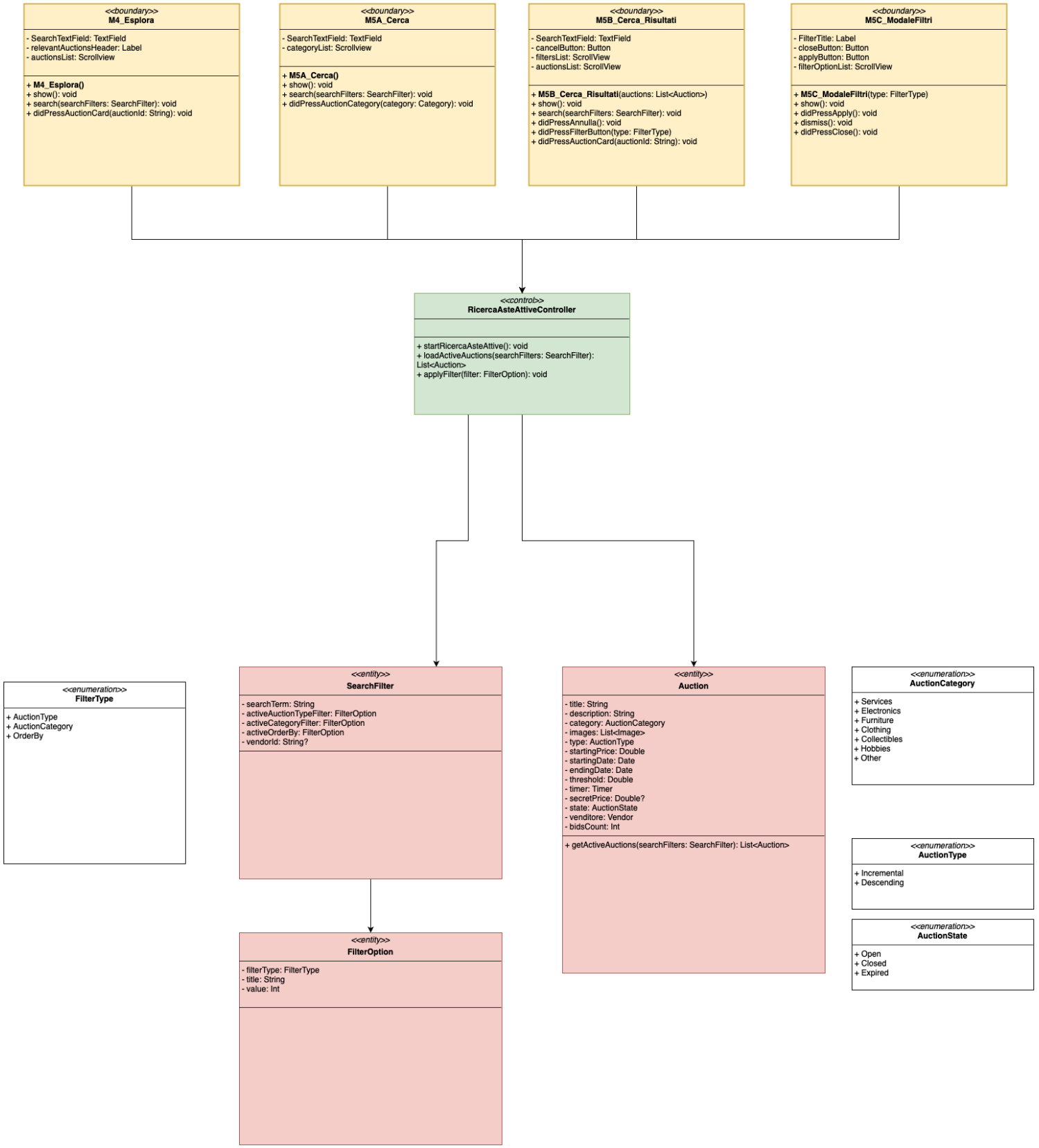
1. Use case - Effettua Login



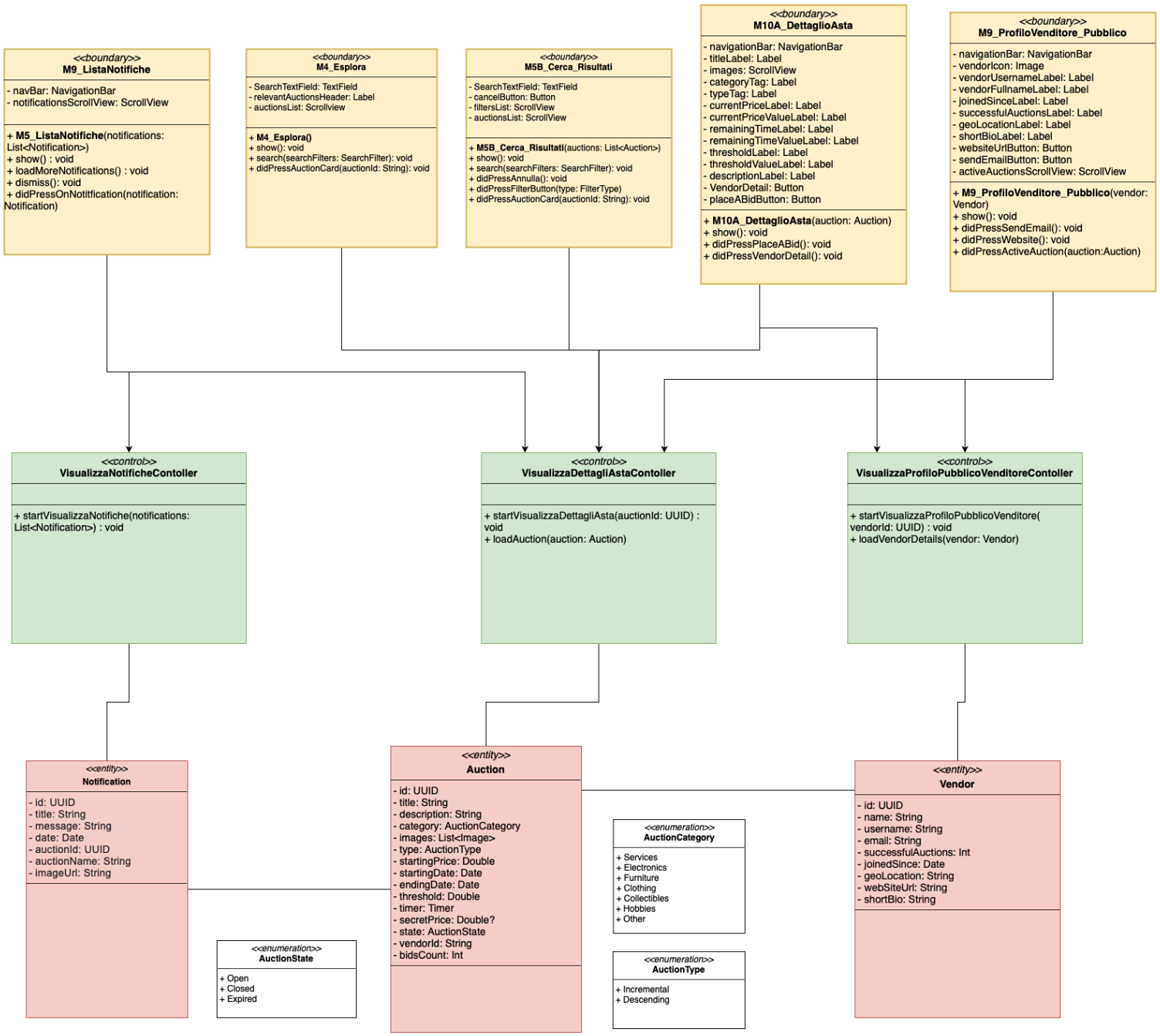
2. Use case - Effettua Registrazione



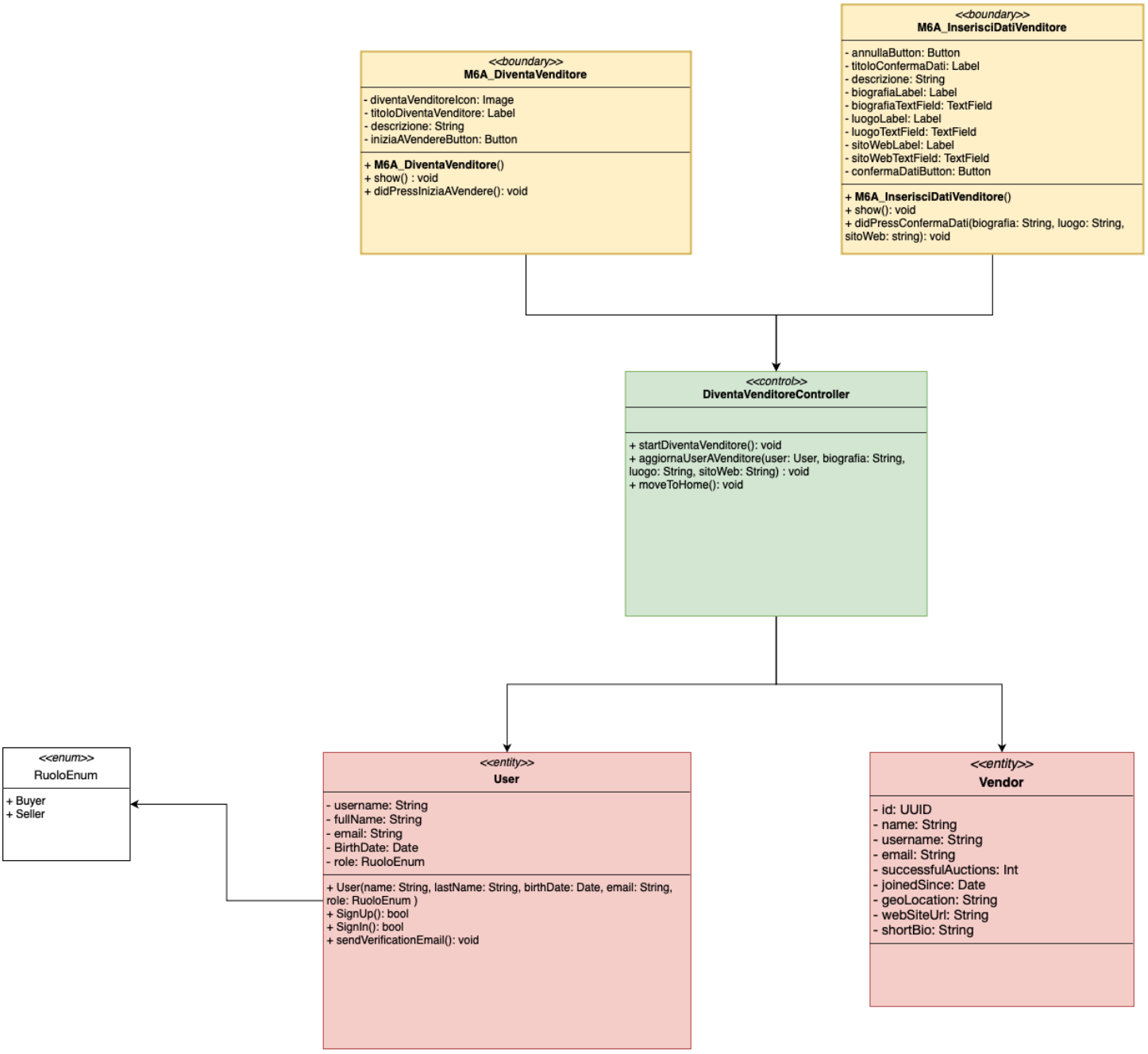
3. Use Case - Ricerca Aste Attive



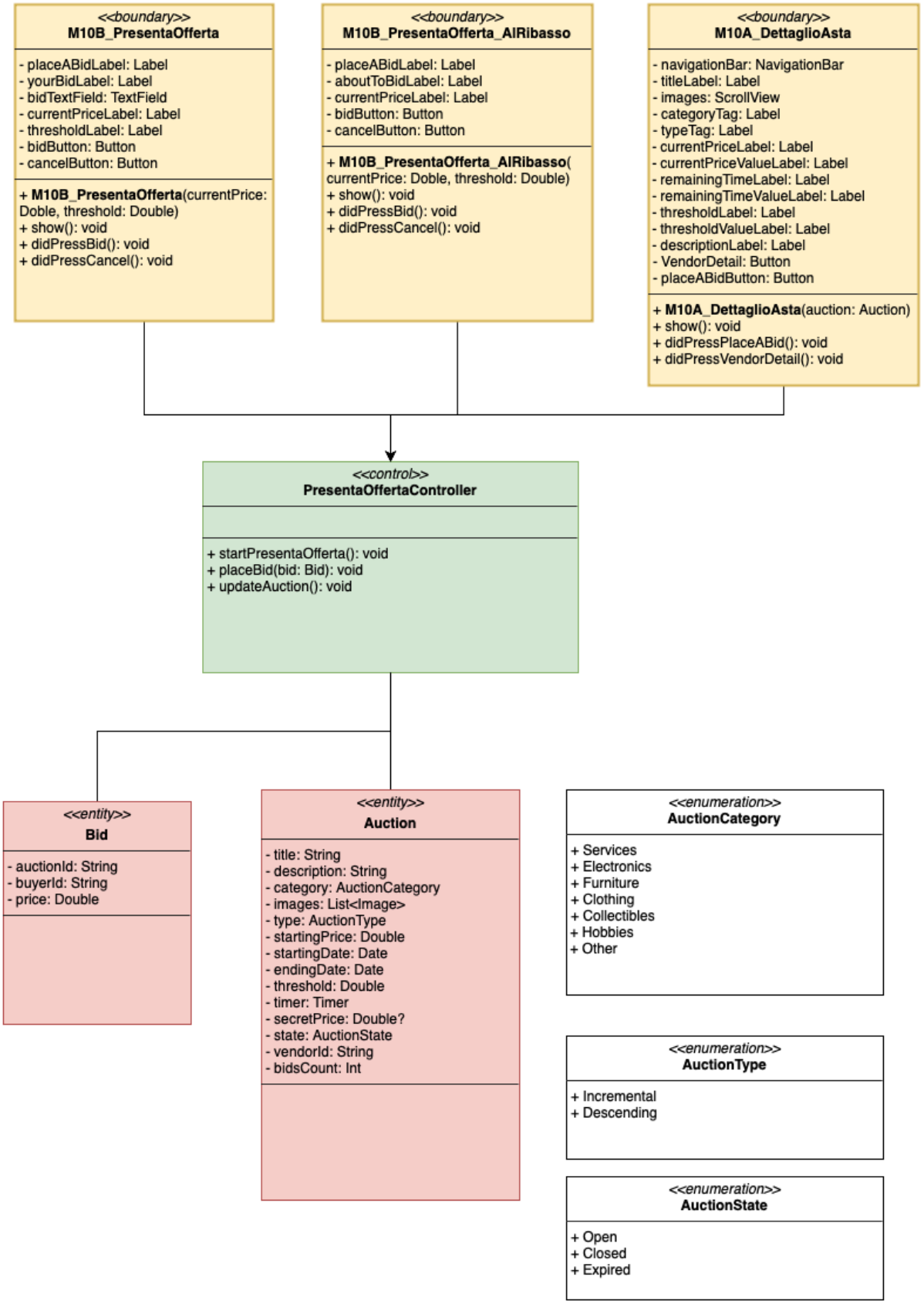
4. Use Cases -Visualizza Dettagli Asta / Visualizza Notifiche / Visualizza profilo pubblico venditore



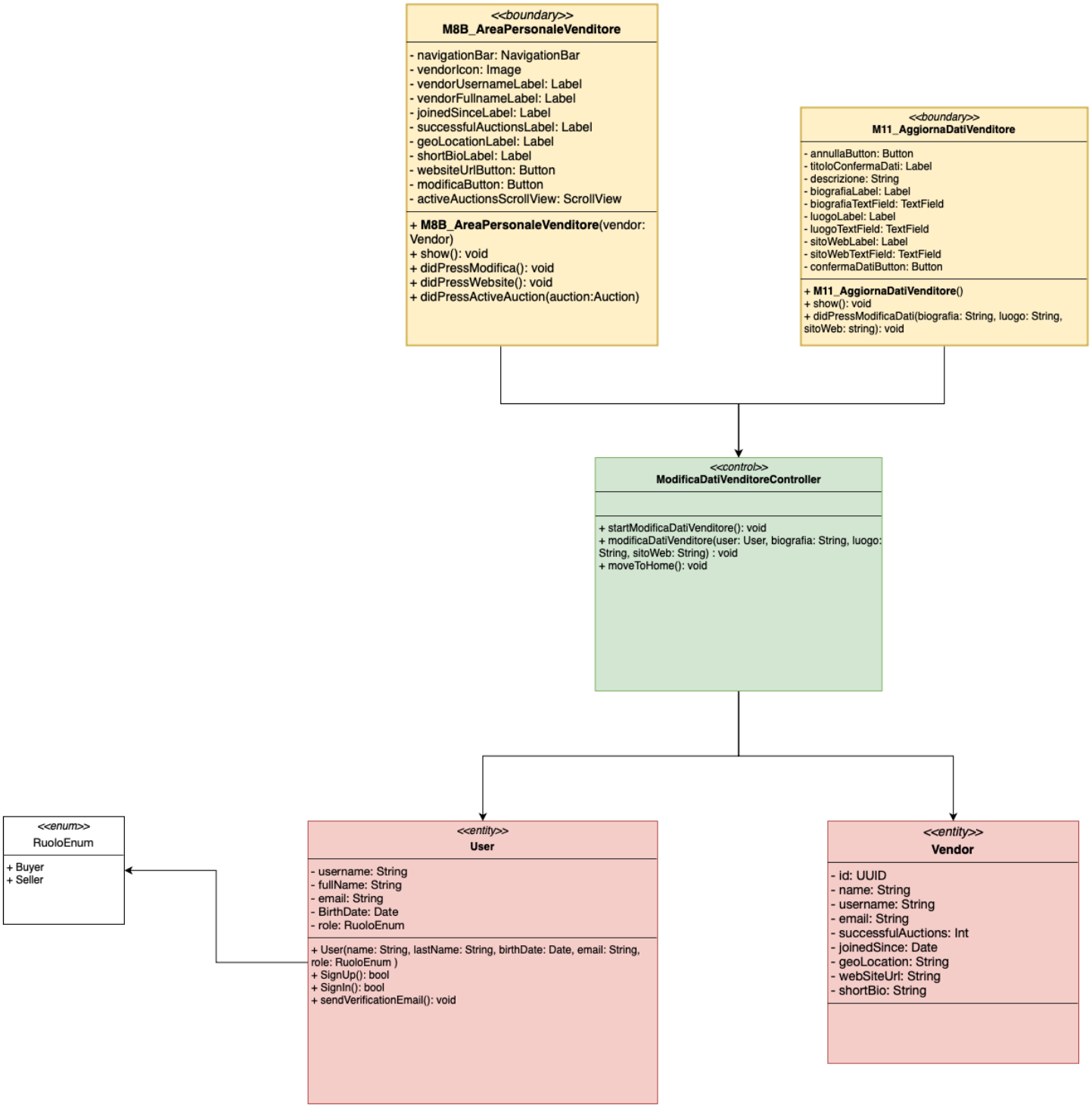
5. Use Case - Diventa Venditore



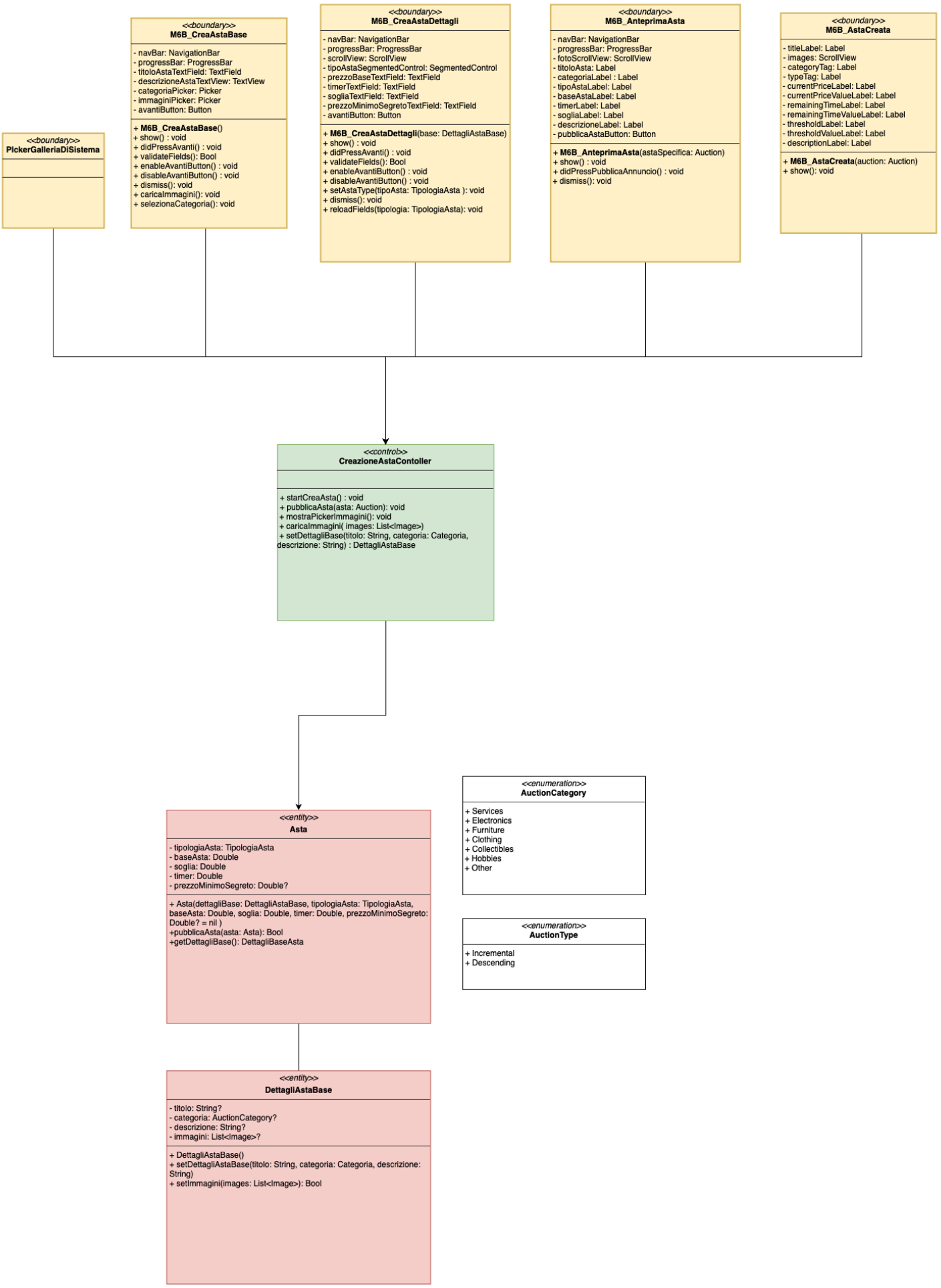
6. Use Case - Presenta Offerta



7. Use Case - Personalizza Profilo Pubblico

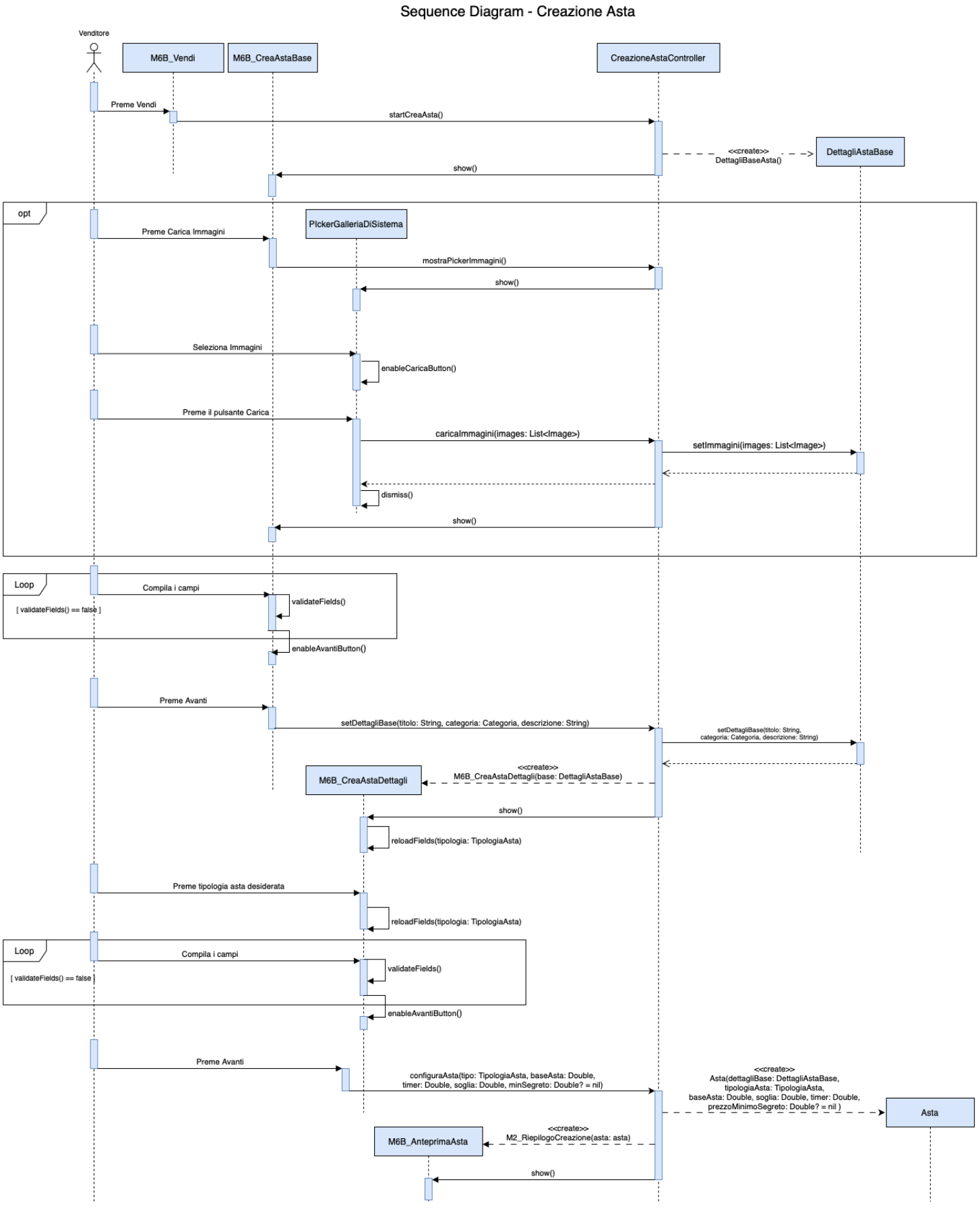


8. Use Case - Crea Asta

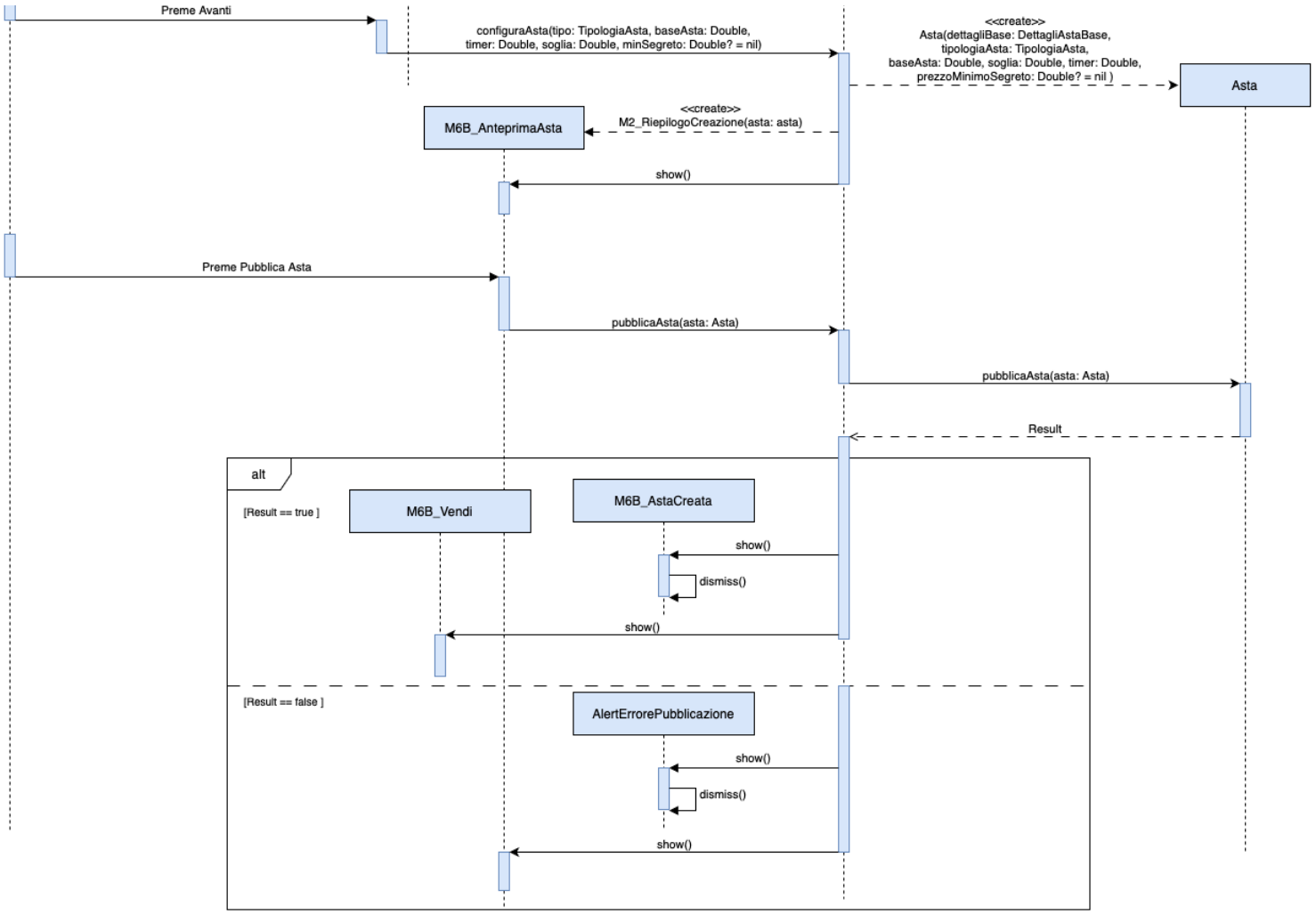


ii. UML Sequence Diagram di Analisi

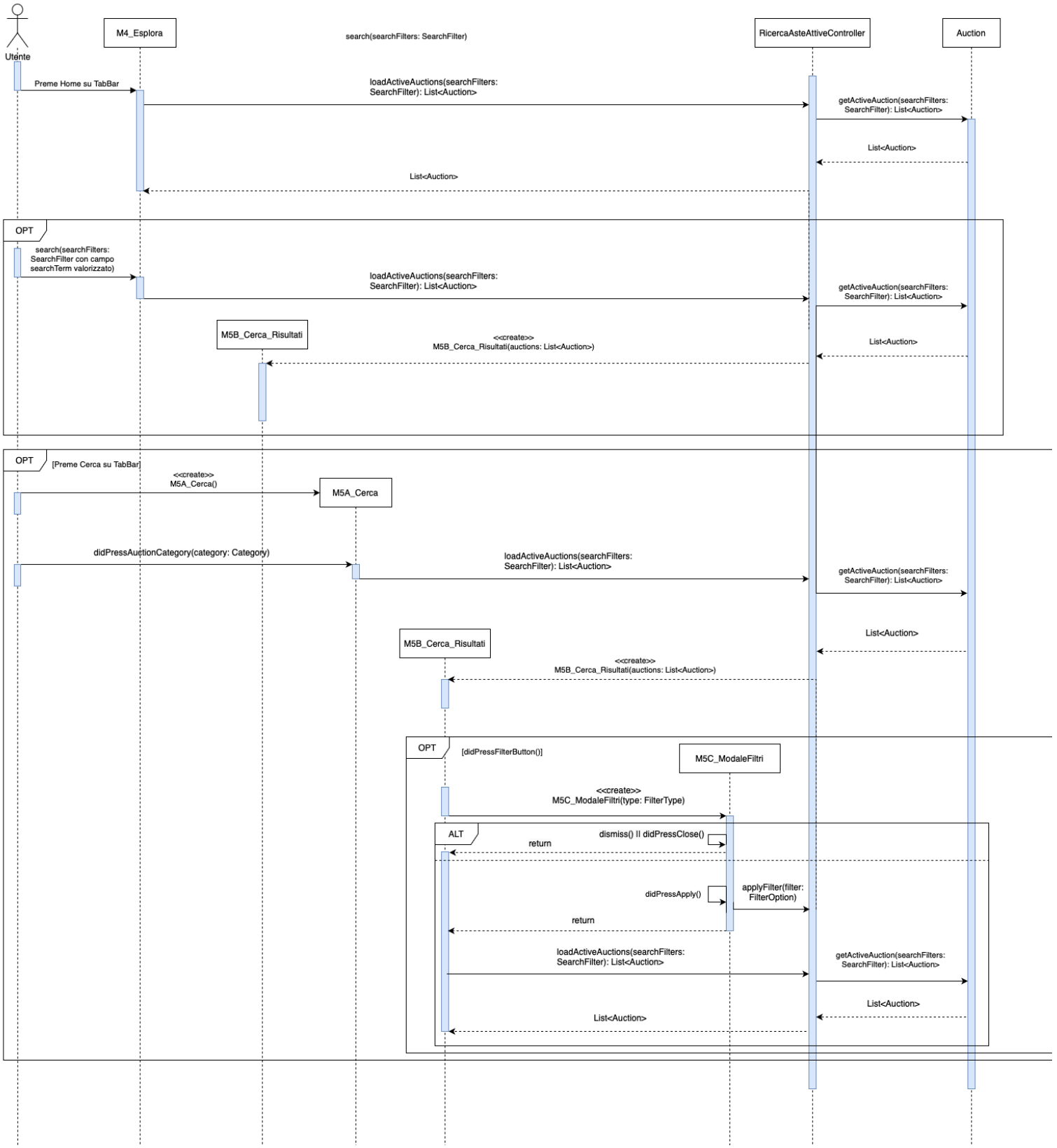
1. Creazione Asta



continuo di Creazione Asta

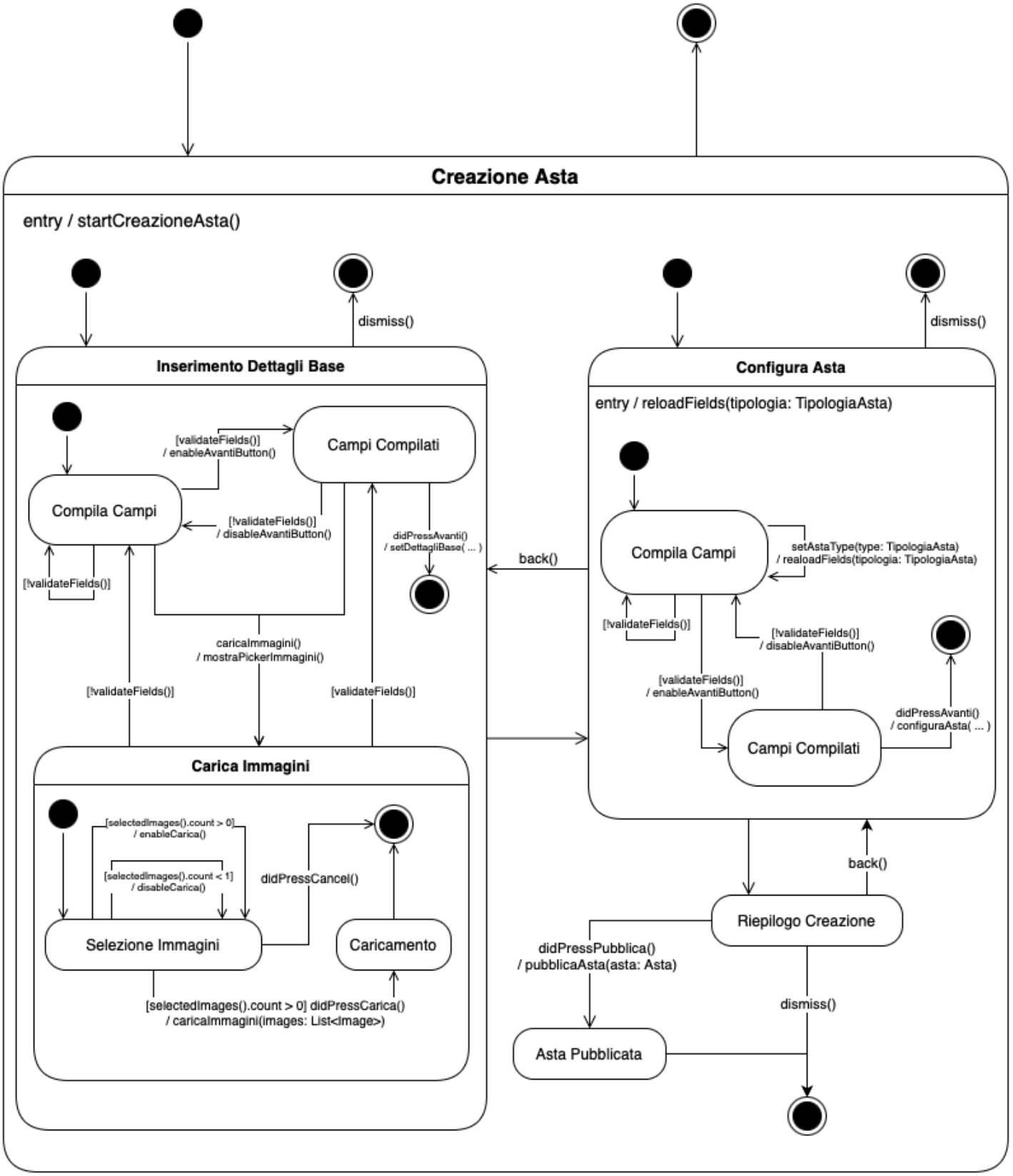


2. Ricerca Aste Attive

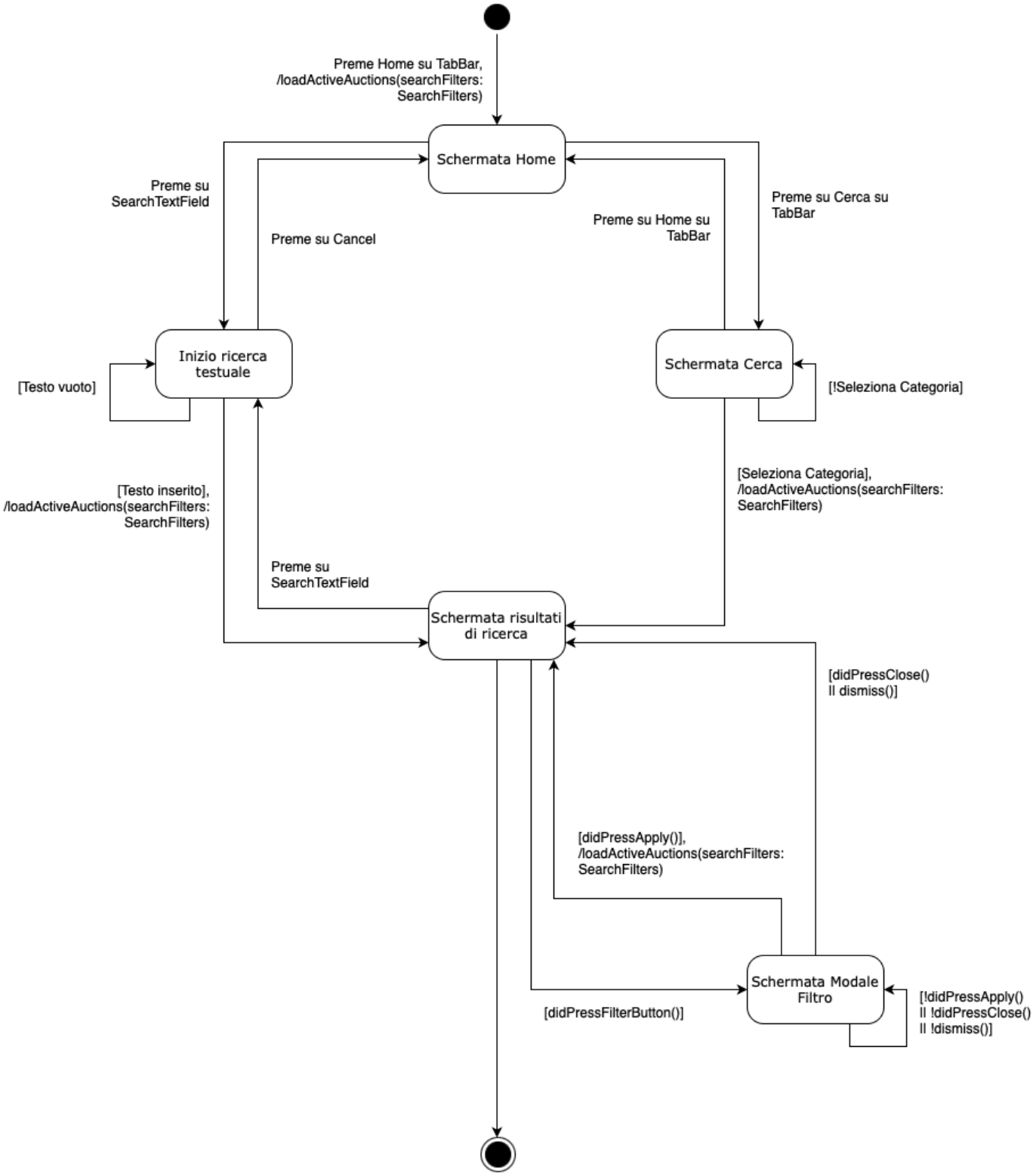


iii. UML Statechart Diagram di Analisi

1. Creazione Asta



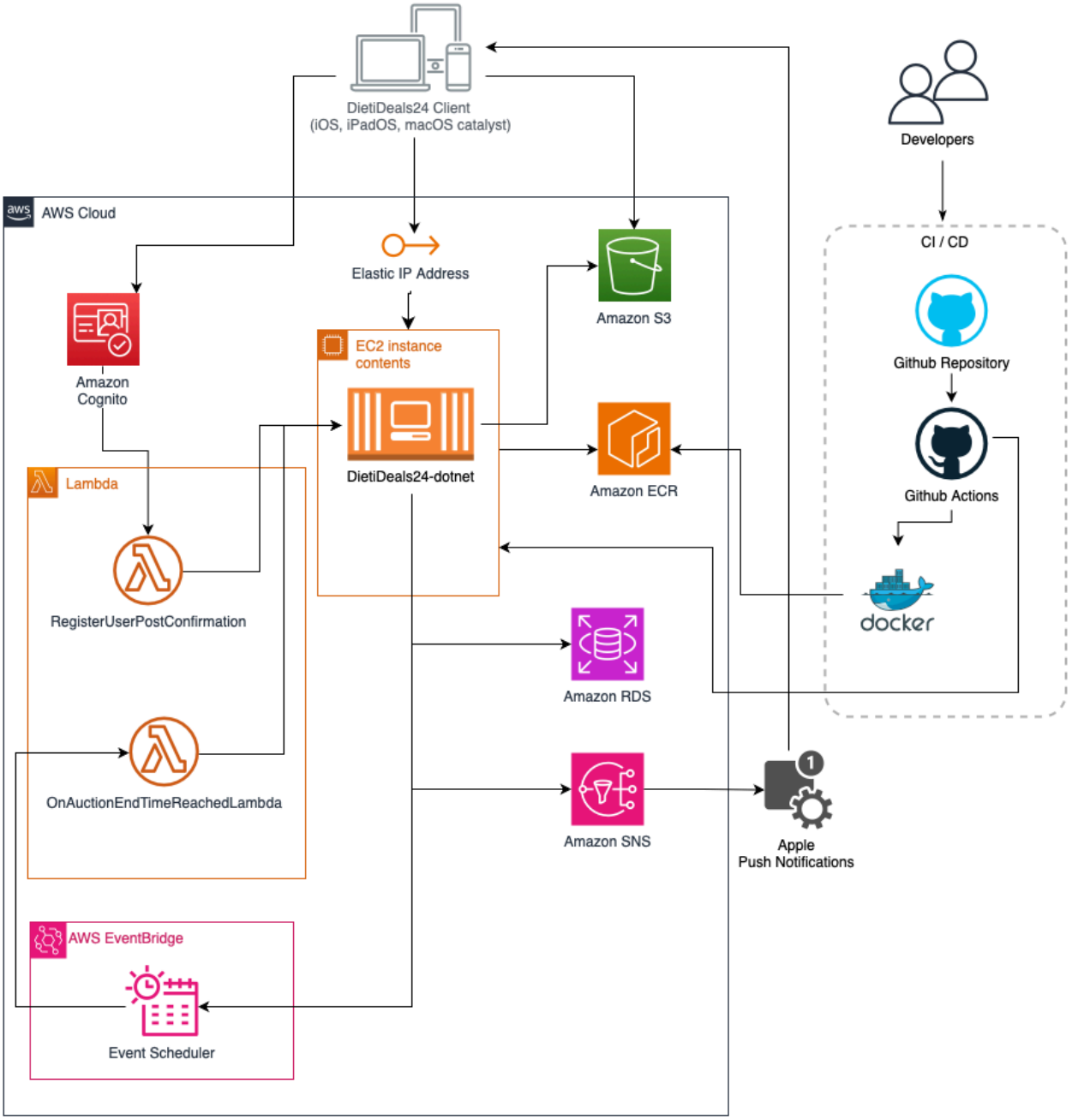
2. Ricerca Aste attive



5. Documento di Design del Sistema

a. Analisi dell'Architettura e scelte Tecnologiche

i. Architettura Generale e Tecnologie utilizzate



L'architettura di sistema per **DietiDeals24** è stata progettata in linea con i requisiti del committente, realizzando le funzionalità assegnate tramite un sistema distribuito con una macro componente front-end per l'utilizzo da parte

degli utenti, e una parte back-end che espone delle api utilizzate dal precedente, messo in atto come container Docker e che utilizzi un approccio orientato al Cloud Computing, accessibile tramite rete internet.

DietiDeals24 è una piattaforma moderna, scalabile e resiliente, costruita su un'architettura cloud-native che sfrutta **Amazon Web Services (AWS)** per gestire l'infrastruttura, l'autenticazione, il database, le notifiche e il deployment. Sono state impiegate tecnologie all'avanguardia come Swift e SwiftUI per la realizzazione dell'applicativo client mobile su piattaforme Apple, e .NET e ASP.NET Core per la realizzazione dell'applicativo Server distribuito tramite container Docker.

1. **DietiDeals24-dotnet**:
   Il backend è sviluppato utilizzando **ASP.NET Core**, **.NET 8** e **C#** per ottenere una moderna **REST API** in grado di gestire le richieste in arrivo dal client, forte di un'ampia presenza di librerie di supporto allo sviluppo, della forte tipizzazione, gestione efficiente della memoria e supporto per la programmazione asincrona concorrente. La distribuzione tramite container **Docker** permette all'applicativo di essere eseguito in modo coerente su qualsiasi sistema che supporti Docker, in quanto lo stesso e tutte le sue dipendenze sono impacchettate in unità standard, astraendo dalla piattaforma sottostante. L'applicazione server è inoltre progettata utilizzando un'architettura monolitica, organizzata in modo service-oriented. All'interno del monolite, ogni servizio è responsabile di una funzionalità specifica (ad esempio, gestione degli utenti, gestione delle aste e notifiche). Questa struttura consente di centralizzare la logica applicativa e semplificare il flusso delle comunicazioni interne. L'approccio monolitico permette di velocizzare lo sviluppo e ridurre la complessità di distribuzione, mantenendo comunque una separazione logica tra i vari servizi per facilitare la manutenzione e l'eventuale scalabilità futura.
2. **Client Front-End DietiDeals24 (Applicazione iOS)**:
   Il front-end dell'applicazione è stato realizzato come applicazione mobile per piattaforme **Apple** quali **iOS**, **iPadOS** e, attraverso la tecnologia Catalyst, utilizzabile anche su **macOS** e **visionOS** in versione iPad. Lo sviluppo tramite **Swift** e **SwiftUI** permette la creazione di interfacce utente moderne e reattive con tempistiche minori e con minor codice

boilerplate grazie alla sua natura dichiarativa, e una maggiore compatibilità con l'intero ecosistema offrendo un'esperienza utente coerente, fluida e intuitiva a prescindere dal dispositivo utilizzato.

Servizi AWS

3. **Hosting (AWS EC2):**
   Il container Docker è ospitato su un'istanza Amazon EC2 (Elastic Compute Cloud). EC2 fornisce macchine virtuali scalabili nel cloud AWS

4. **Amazon Cognito:**
   Gestisce l'autenticazione e l'autorizzazione degli utenti. Cognito fornisce funzionalità come la registrazione, l'accesso, la gestione delle identità e il controllo degli accessi. Cognito permette di intercettare specifici eventi in relazione alle fasi di autenticazione dell'utente, funzionalità che sfruttiamo ad esempio con il trigger che viene lanciato dopo la conferma dell'indirizzo email per poter aggiungere l'utente alla tabella degli utenti del Database.

5. **AWS Lambda:**
   Un servizio di AWS che consente di eseguire codice in risposta a eventi senza dover gestire server, utilizzando un modello di calcolo serverless. In particolare, sono utilizzate per due scenari specifici:
   - RegisterUserPostConfirmation: Inserisce l'utente che ha confermato la registrazione nella tabella corrispondente del db, permettendo di legare l'utenza ad operazioni legate alle aste e alla gestione del ruolo.
   - **OnAuctionEndTimeReachedLambda:** Funzione che viene invocata dallo scheduler di AWS EventBridge quando un'asta raggiunge la sua data di scadenza, la quale a sua volta informa il backend dell'evento.

6. **AWS EventBridge - Event Scheduler:**
   Servizio di AWS utilizzato per pianificare l'esecuzione di eventi. In questo caso, il Backend crea o elimina uno scheduler in seguito alla creazione di un asta, alla creazione di una nuova offerta, o alla riprogrammazione di una data di scadenza per un asta. Lo scheduler invoca la funzione Lambda **OnAuctionEndTimeReachedLambda,** passando come parametro di input l'id dell'asta una volta raggiunta la data impostata in fase di creazione. Questo approccio consente una

gestione coerente, reattiva e affidabile degli eventi in tempo reale, senza introdurre complessità o impatti computazionali sul Backend o sul Database, lasciando al Backend ogni responsabilità sulla gestione dei dati.

7. **Amazon RDS - PostgreSQL:**
   RDS semplifica la configurazione, la gestione e la scalabilità di database relazionali nel cloud. PostgreSQL è un database relazionale open-source noto per la sua conformità agli standard SQL e ACID per le transazioni, il che assicura che le operazioni del database siano affidabili e sicure anche in caso di errore del sistema. Amazon RDS gestirà il provisioning, l'aggiornamento, il backup e la scalabilità automatica, permettendo al team di concentrarsi sullo sviluppo delle funzionalità senza preoccuparsi della gestione dell'infrastruttura del database.
8. **Amazon S3:**
   Servizio utilizzato per l'archiviazione di file multimediali, come le foto associate alle aste. Al momento della creazione dell'asta, il client chiede al Backend di interfacciarsi con S3 per generare dei url coi quali potrà tramite una semplice chiamata caricare le immagini.
9. **Amazon ECR:**
   Elastic Container Registry è un registro privato di immagini Docker gestito da AWS. Qui vengono archiviate le immagini Docker del backend DietiDeals24, generate tramite pipeline su Github Actions, prima del deployment su EC2.
10. **Amazon SNS:**
    Simple Notification Service è un servizio di messaggistica pub/sub (publish/subscribe) completamente gestito. Viene utilizzato per gestire il flusso di notifiche push in risposta ad eventi legati alle aste. Esso si interfaccia con l'APNS di Apple per permettere la corretta ricezione delle notifiche push per l'app.

A coronamento dell'architettura di DietiDeals24, un processo di integrazione continua e deployment continuo (CI/CD), basato su GitHub Actions e Amazon ECR, assicura che ogni aggiornamento, inclusi quelli relativi al design system, venga rilasciato rapidamente e in modo affidabile. Questo flusso automatizzato, che culmina con il deployment dell'immagine Docker del backend su EC2,

garantisce coerenza tra il design system definito e l'esperienza utente finale, accelerando il ciclo di rilascio e mantenendo elevata la qualità del software. Il CI/CD abilita quindi un design system dinamico, integrato nel ciclo di vita dello sviluppo e in costante evoluzione.

ii. Criteri di Design

Questa sezione delinea i principi guida e le scelte progettuali fondamentali adottate nello sviluppo dei componenti backend e client della piattaforma DietiDeals24. L'obiettivo è garantire coerenza, manutenibilità, scalabilità e un'esperienza utente ottimale.

a. Criteri di design dell'applicazione Backend

Il backend di DietiDeals24 è stato progettato seguendo un'architettura monolitica modulare, con l'obiettivo primario di accelerare lo sviluppo iniziale, pur mantenendo una struttura che faciliti una futura evoluzione verso un'architettura a microservizi. Questa scelta rappresenta un compromesso tra la semplicità di un monolite e la flessibilità dei microservizi. I criteri di design chiave sono:

1. **Architettura Monolitica Modulare:**
    - L'intera applicazione backend è contenuta in un singolo container Docker, semplificando il deployment e la gestione in questa fase iniziale.
    - Internamente, il codice è organizzato in moduli distinti (Rest API, Data Access Layer, logica di business nei Worker), promuovendo una separazione delle responsabilità e una maggiore manutenibilità.
    - Questa modularità interna facilita una potenziale migrazione futura a microservizi, in cui ciascun modulo potrebbe diventare un servizio indipendente.
2. **API RESTful (JSON):**
    - Il backend espone le sue funzionalità tramite API RESTful che utilizzano il formato JSON per lo scambio di dati.

3. **Autenticazione e Autorizzazione (con Cognito):**
    - L'autenticazione degli utenti è gestita da Amazon Cognito, che fornisce un sistema di gestione delle identità sicuro e scalabile.
    - Le API sono progettate per essere protette tramite JWT (JSON Web Token) forniti da Cognito.
    - I token JWT contengono informazioni aggiuntive (claims), come il ruolo dell'utente (“custom:role”, con valori 0 o 1 per indicare compratore/venditore), per consentire un controllo degli accessi granulare a livello di API (autorizzazione).
4. **Containerizzazione (Docker e Docker Compose):**
    - L'utilizzo di Docker garantisce la portabilità e la coerenza dell'ambiente di esecuzione tra sviluppo, test e produzione.
    - Docker Compose viene utilizzato per semplificare lo sviluppo locale, consentendo di eseguire l'applicazione e le sue dipendenze (es., database) senza richiedere una configurazione complessa dell'ambiente di sviluppo.
5. **Struttura Interna a Layer:**
    - **Rest API Layer (ASP.NET Core Controllers):**
        - Punto di ingresso per le richieste HTTP.
        - Utilizza ASP.NET Core Controllers per gestire le richieste, instradarle ai Worker appropriati e restituire le risposte.
        - Si appoggia a FluentValidation per la validazione dei dati in input.
    - **Business Logic Layer (Worker):**
        - Incapsulano la logica di business specifica dell'applicazione.
        - Sono implementati come classi che implementano interfacce specifiche (per ogni Controller).
        - Non hanno conoscenza dei Controller, ma accedono ai dati tramite i Service del DAL.
        - Utilizzano la Dependency Injection per ottenere le dipendenze necessarie.

- **Data Access Layer (DAL):**
    - Responsabile dell'interazione con il database (PostgreSQL tramite Amazon RDS o collegamento in locale con Docker Compose).
    - Utilizza Entity Framework Core per la mappatura oggetto-relazionale (ORM) e la gestione delle entità.
    - Implementa i pattern Repository e Unit of Work per astrarre l'accesso ai dati e garantire la consistenza transazionale.
    - Espone le funzionalità tramite "Service" che forniscono un'interfaccia di alto livello per le operazioni sui dati.

6. **Dependency Injection (DI):**
    - Viene utilizzato il container DI integrato di .NET (Microsoft.Extensions.DependencyInjection).
    - La maggior parte dei servizi sono registrati con un lifetime *Scoped*, garantendo che una singola istanza venga utilizzata per tutta la durata di una richiesta HTTP.
    - Sono state create estensioni custom per semplificare la registrazione dei Worker e dei Service.
7. **Mapping tra Oggetti (DTO):**
    - Vengono utilizzati DTO (Data Transfer Object) ad hoc per trasferire dati tra i layer e per personalizzare la risposta inviata al client. Il mapping dalle entity ai DTO e viceversa è presumibilmente gestito manualmente all'interno dei service o dei worker, in base alla complessita' della trasformazione.
8. **Validazione (FluentValidation):**
    - FluentValidation viene utilizzato per definire regole di validazione complesse per i dati in input.
    - La validazione è integrata nel flusso di elaborazione delle richieste ASP.NET Core.
9. **Gestione della Configurazione:**
    - In ambiente di sviluppo locale, le configurazioni (es., stringa di connessione al database) vengono gestite tramite Docker Compose (variabili d'ambiente).

- In ambiente di produzione, le configurazioni vengono gestite tramite i secret di GitHub Actions e iniettate nell'applicazione durante il deployment.

b. Criteri di design dell'applicazione Client

L'applicazione client di DietiDeals24, sviluppata per le piattaforme iOS, iPadOS e macOS (tramite Catalyst), è stata progettata seguendo una serie di principi chiave per garantire un'esperienza utente coerente, un codice manutenibile e una buona scalabilità.

1. **UI Dichiarativa (SwiftUI):**
   - L'interfaccia utente è costruita utilizzando SwiftUI, il framework dichiarativo di Apple. Questo approccio permette di descrivere l'aspetto e il comportamento della UI in base allo stato dell'applicazione, semplificando lo sviluppo e la manutenzione.
   - Viene sfruttata la macro @Observable, introdotta in iOS 17, per la gestione dello stato e la propagazione dei cambiamenti all'interno dell'interfaccia utente, migliorando la reattività dell'app.
   - Viene fatto un uso estensivo di componenti personalizzati in combinazione a quelli messi a disposizione dal framework.
2. **Architettura MVVM + Coordinator:**
   - L'architettura dell'applicazione segue il pattern Model-View-ViewModel (**MVVM**), che promuove una chiara separazione delle responsabilità:
     - **Model:** Rappresenta i dati dell'applicazione.
     - **View:** Definisce l'interfaccia utente e visualizza i dati forniti dal ViewModel.
     - **ViewModel:** Agisce da intermediario tra la View e il Model, esponendo i dati in un formato adatto alla visualizzazione, gestendo le interazioni dell'utente e le integrazioni con servizi esterni. I ViewModels vengono resi osservabili tramite la macro @Observable.
   - Per la gestione della navigazione e del flusso dell'applicazione, è stato introdotto il pattern **Coordinator**. Questo componente astrae

la logica di navigazione dalla View e dai ViewModel, rendendo il codice più modulare e testabile.
    - È stata sviluppata una libreria di navigazione personalizzata per gestire in modo ottimale il flusso di navigazione senza legarlo strettamente alle API di SwiftUI, offrendo maggiore flessibilità e controllo.

3. **Dependency Injection (DI):**
    - Per disaccoppiare i componenti dell'applicazione e favorire la testabilità, è stato implementato un sistema di Dependency Injection (DI) personalizzato. Questo container DI a livello applicativo gestisce la registrazione e la risoluzione delle dipendenze (servizi, ViewModel, ecc.).
    - Questo approccio si discosta dall'uso di Environment Object che renderebbero gli oggetti registrati, legati ed unicamente accessibili all'interno delle View.

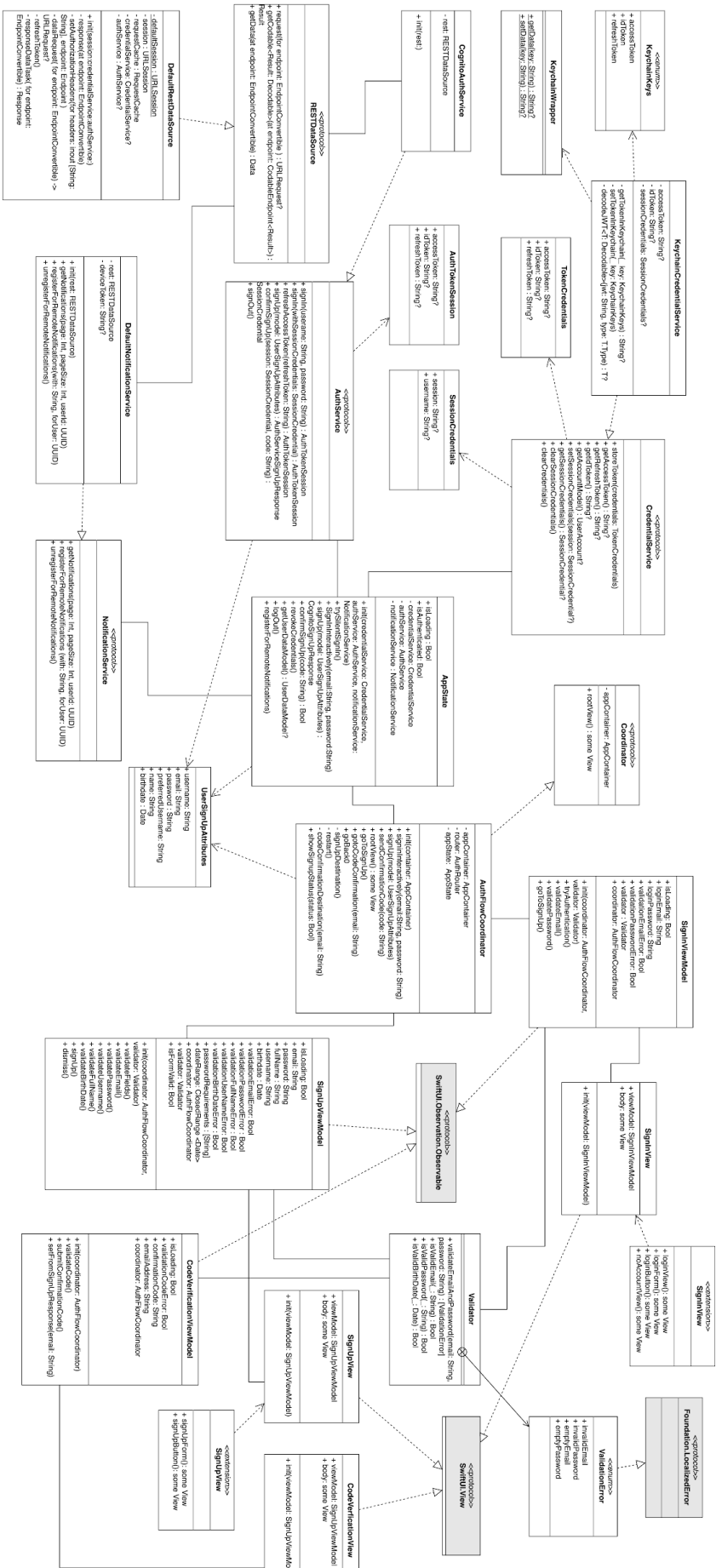
4. **Networking Asincrono (Async/Await):**
    - Per le operazioni di rete (comunicazione con il backend), viene utilizzata una libreria personalizzata basata su URLSession di Apple.
    - Le funzionalità async/await di Swift, disponibili grazie al supporto di una versione recente di iOS, vengono sfruttate per gestire le chiamate di rete in modo asincrono e concorrente, semplificando il codice e migliorando la reattività dell'interfaccia utente.

5. **Struttura del Progetto e Organizzazione del Codice:**
    - Il progetto Xcode è organizzato in cartelle, separate per funzionalità, che riflettono l'architettura MVVM + Coordinator. Questo include cartelle per Models, Views, ViewModels, Coordinators, Services (con sottocartelle per interfacce e implementazioni) e altre componenti comuni.
    - Vengono utilizzati protocolli in modo estensivo per astrarre le implementazioni dei servizi, promuovendo il disaccoppiamento e la testabilità.

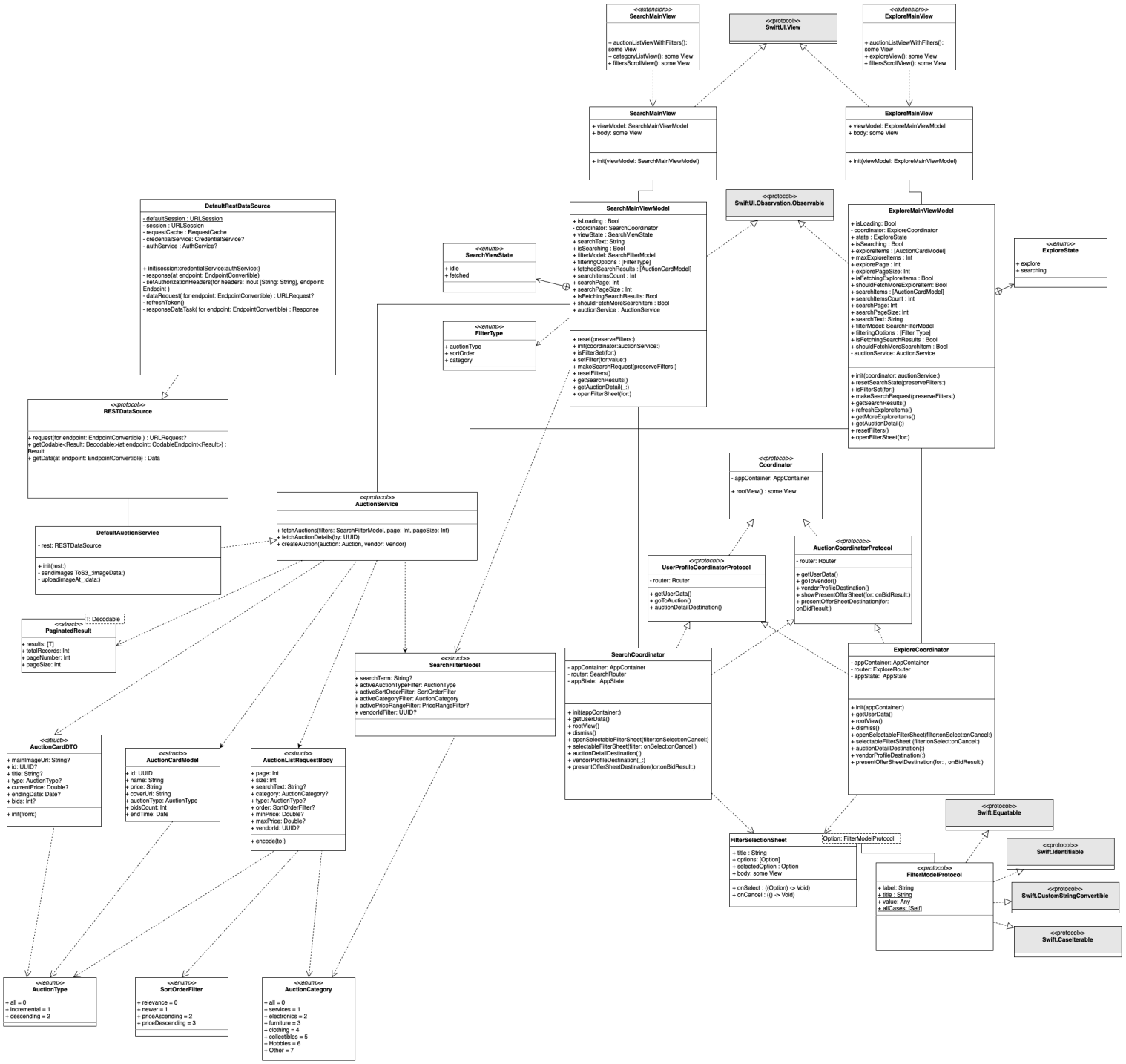
6. **Localizzazione:**
    - L'applicazione supporta la localizzazione in inglese e italiano, utilizzando il sistema String Catalog introdotto da Xcode, facilitando la gestione di testi tradotti, pluralizzazioni e variazioni di genere.
7. **Gestione delle Configurazioni per Ambienti Multipli (xcconfig, plist):**
    - Il progetto include target Xcode separati per gli ambienti di sviluppo (Development) e produzione (Production).
    - Le differenze di configurazione tra i target (es., URL del backend) sono gestite tramite file .xcconfig (non versionati nel controllo sorgenti) e file .plist specifici per ciascun target.
8. **Testing:**
    - Sono stati implementati test di unità (unit test) utilizzando il framework Swift Testing di Apple, nuova aggiunta basata su xUnit che sostituisce la verbosità del framework XCTest sfruttando le macro.
9. **Distribuzione:**
    - L'app è predisposta alla distribuzione, in versioni di test, tramite il push su piattaforma TestFlight direttamente da XCode.
10. **Sicurezza**:
    - Per garantire la massima sicurezza delle informazioni sensibili dell'utente (come token di autenticazione o altre credenziali), l'applicazione utilizza il Portachiavi iOS (Keychain). Questo approccio offre benefici significativi: i dati vengono crittografati a livello di sistema operativo, spesso protetti da meccanismi hardware (come il Secure Enclave), e sono isolati dalle altre applicazioni presenti sul dispositivo.

b. UML Class Diagram di Design

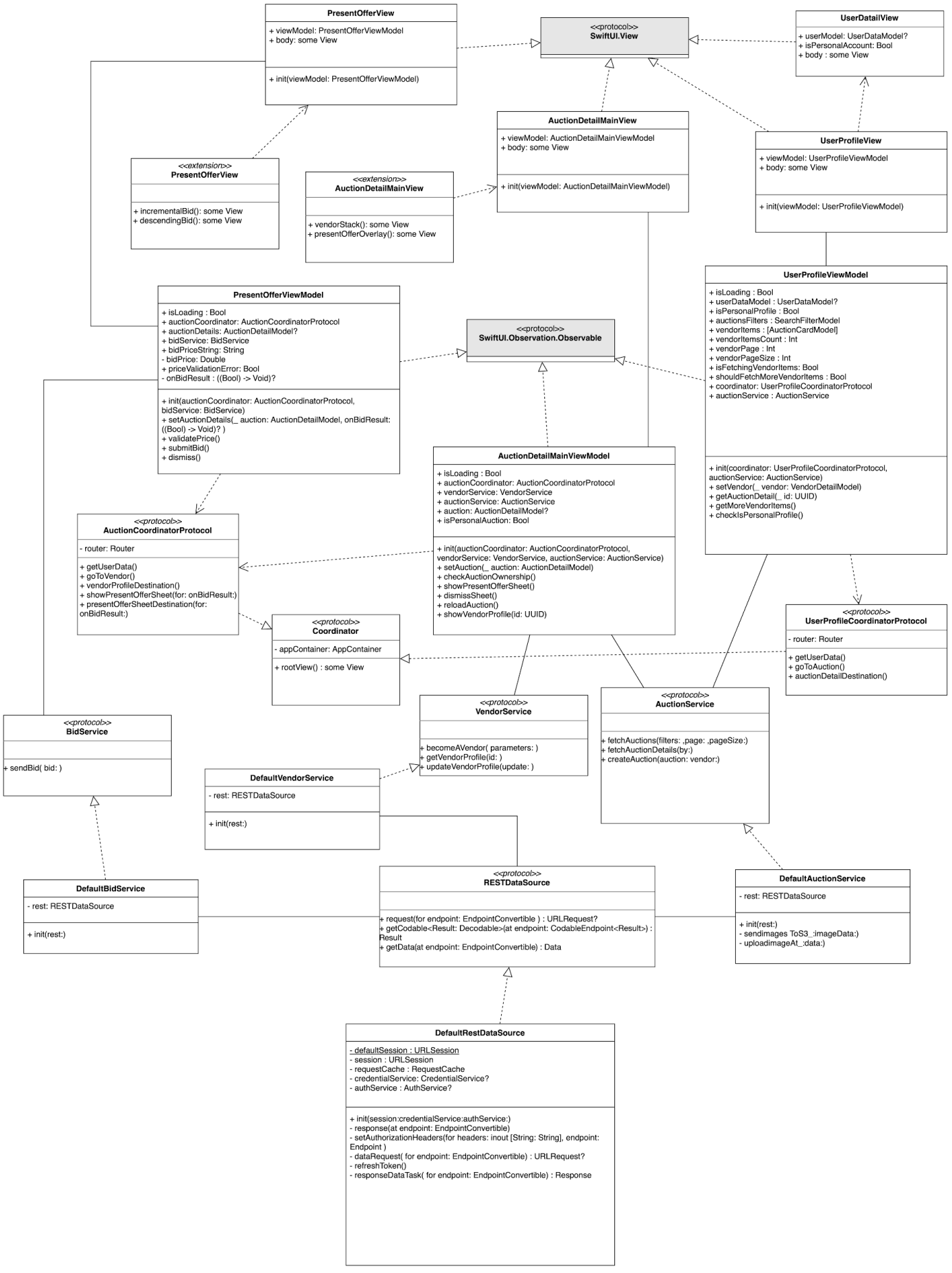
i. Effettua Login / Registrazione



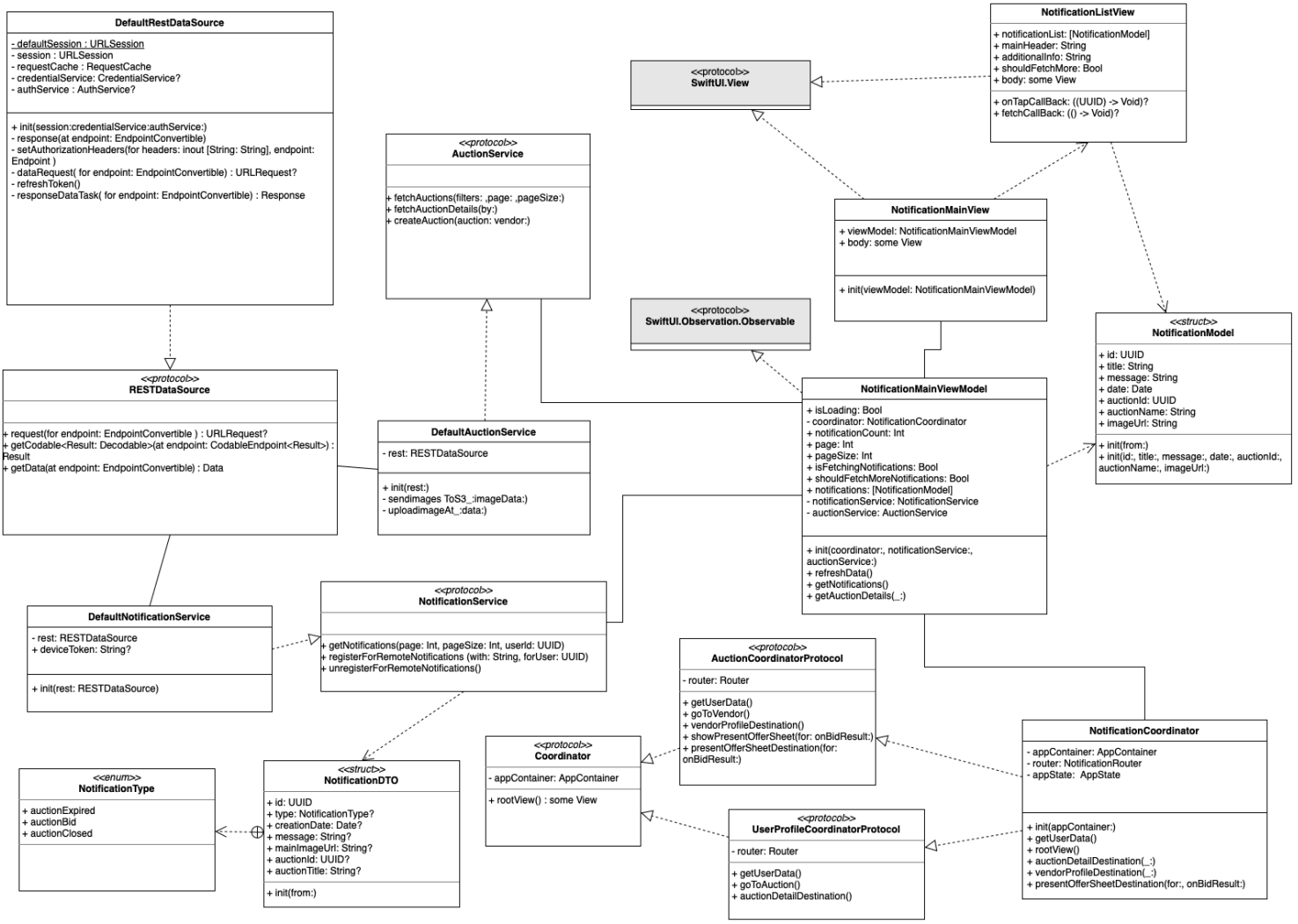
ii. Ricerca Aste Attive



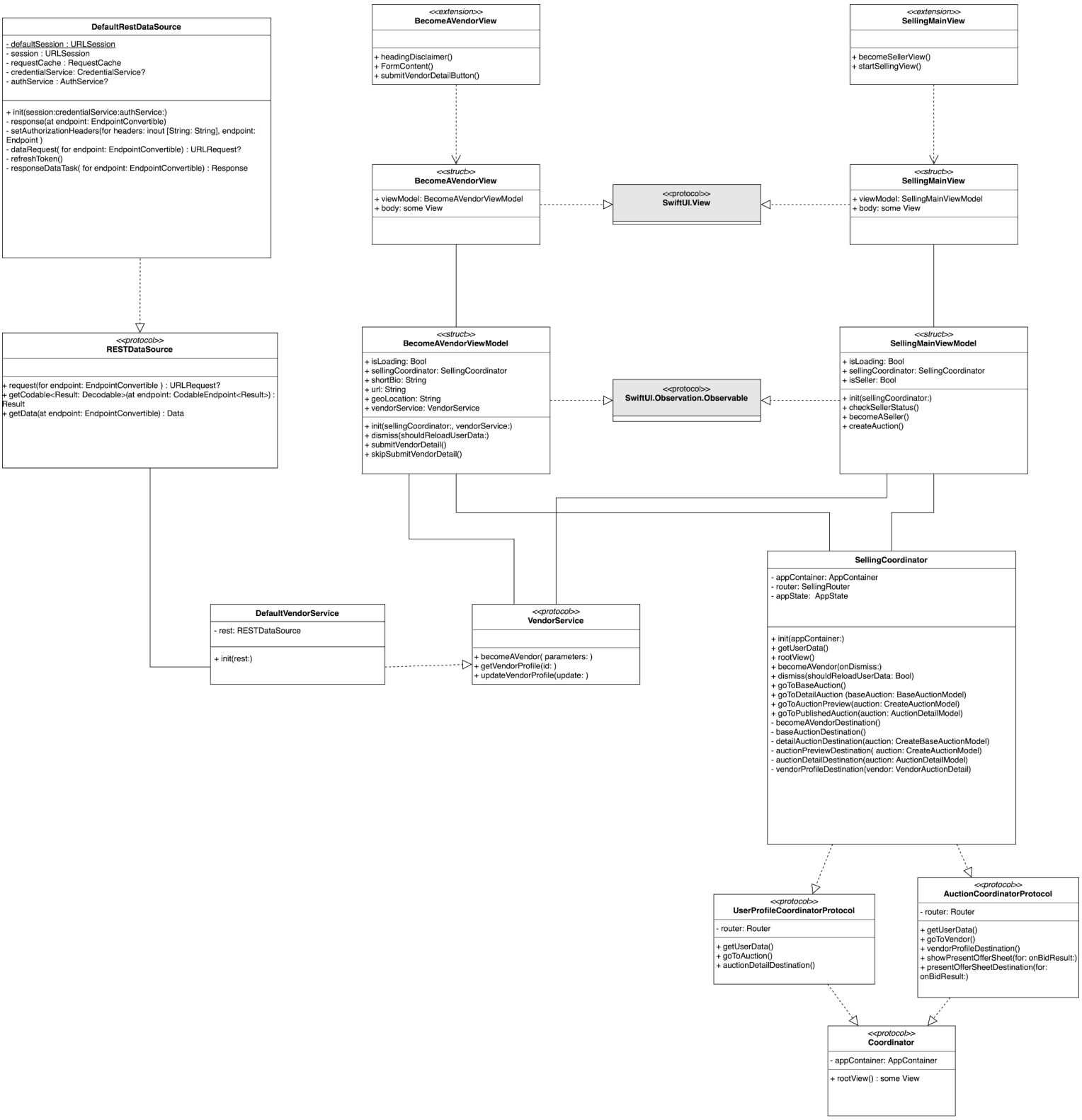
iii. Visualizza Dettagli Asta / Presenta Offerta / Visualizza Profilo Pubblico Venditore



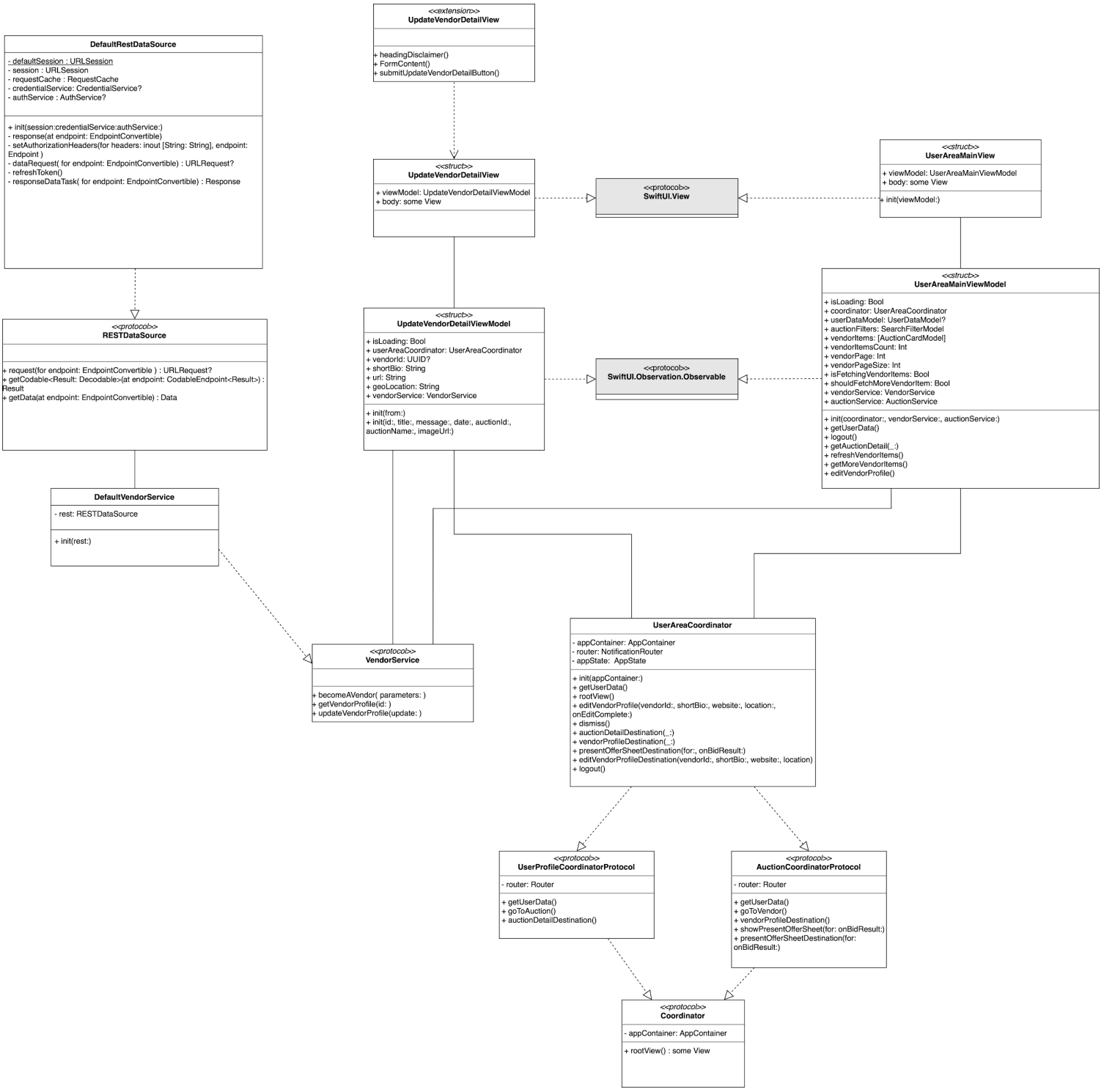
iv. Visualizza Notifiche



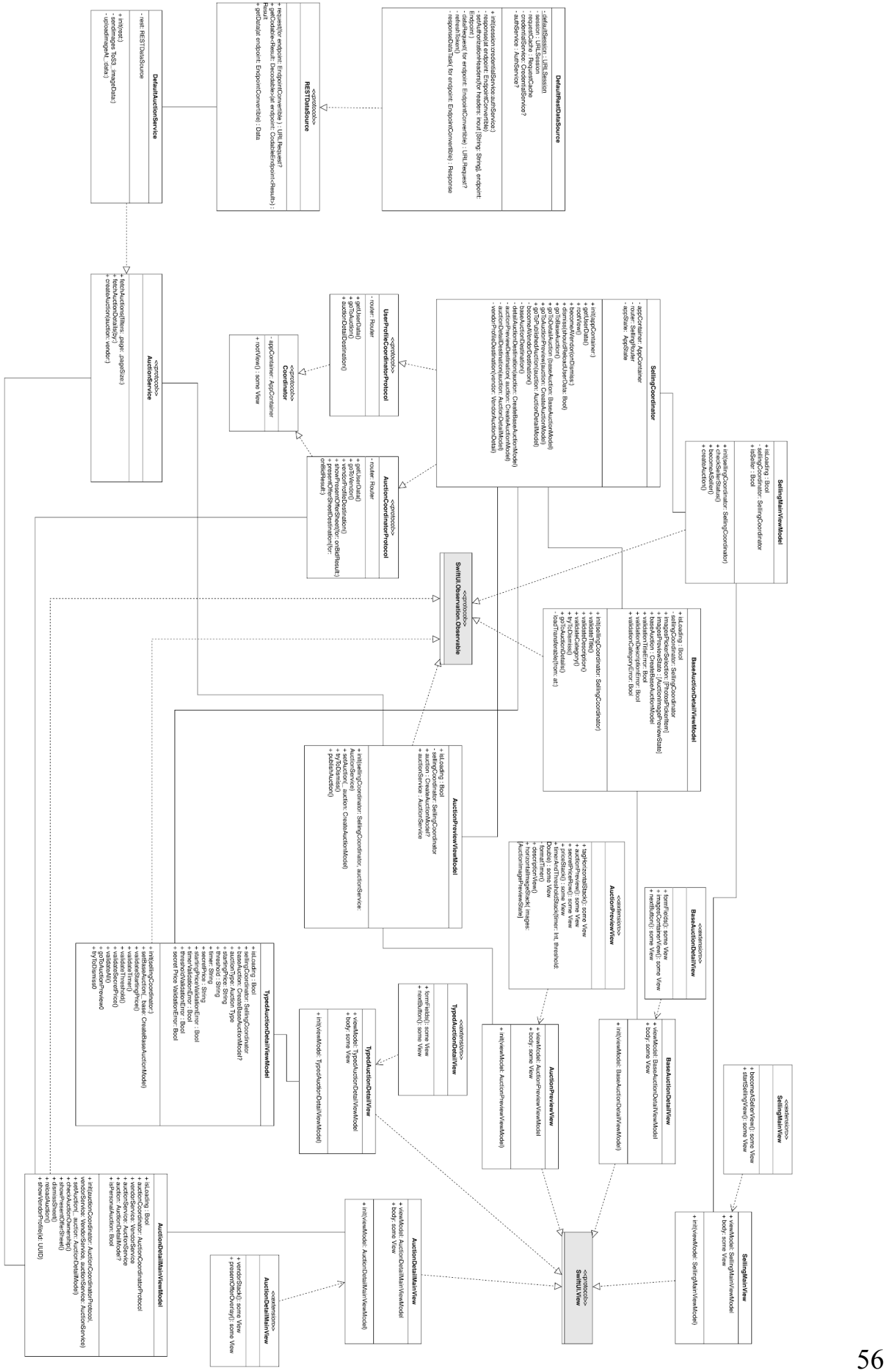
v. Diventa Venditore



vi. Personalizza Profilo pubblico

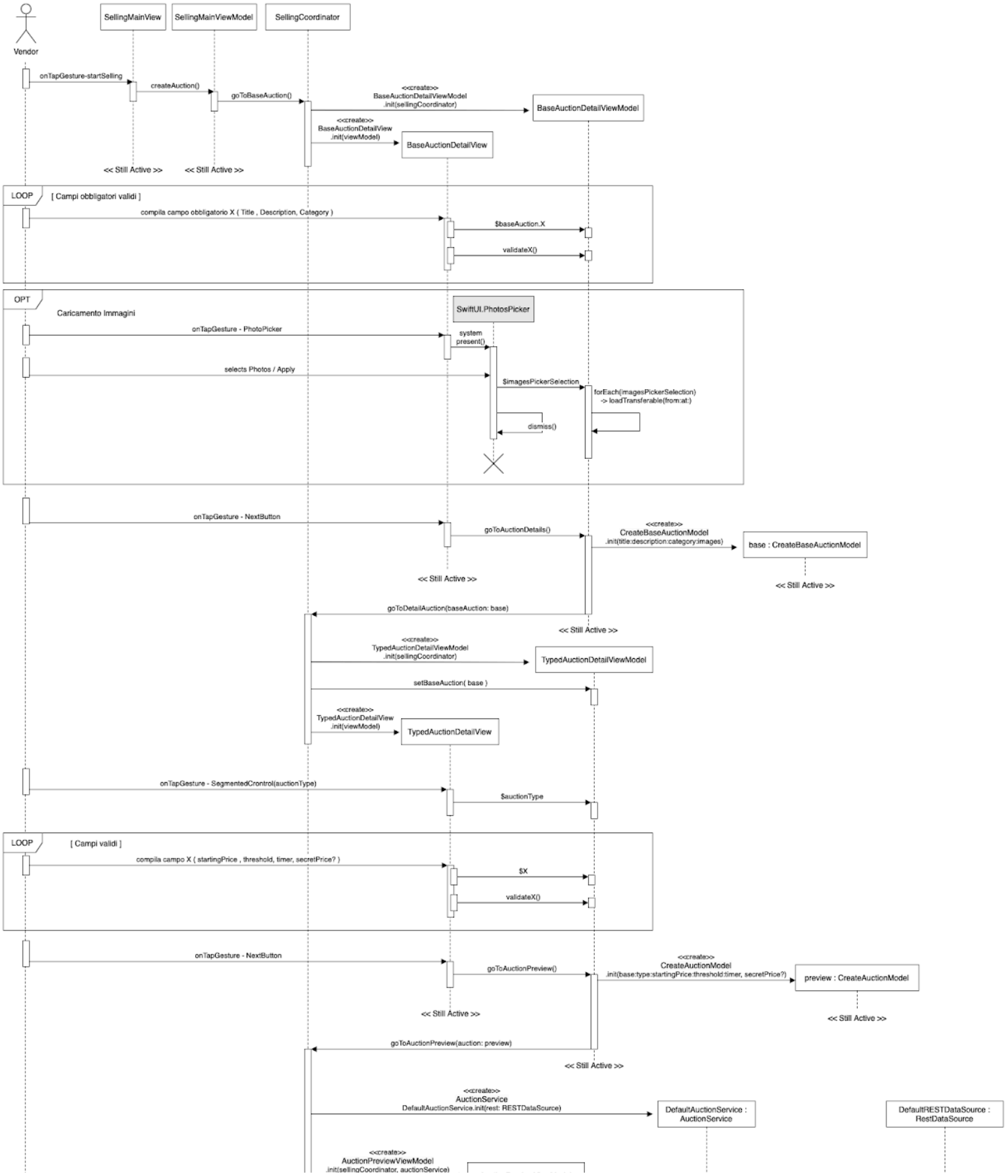


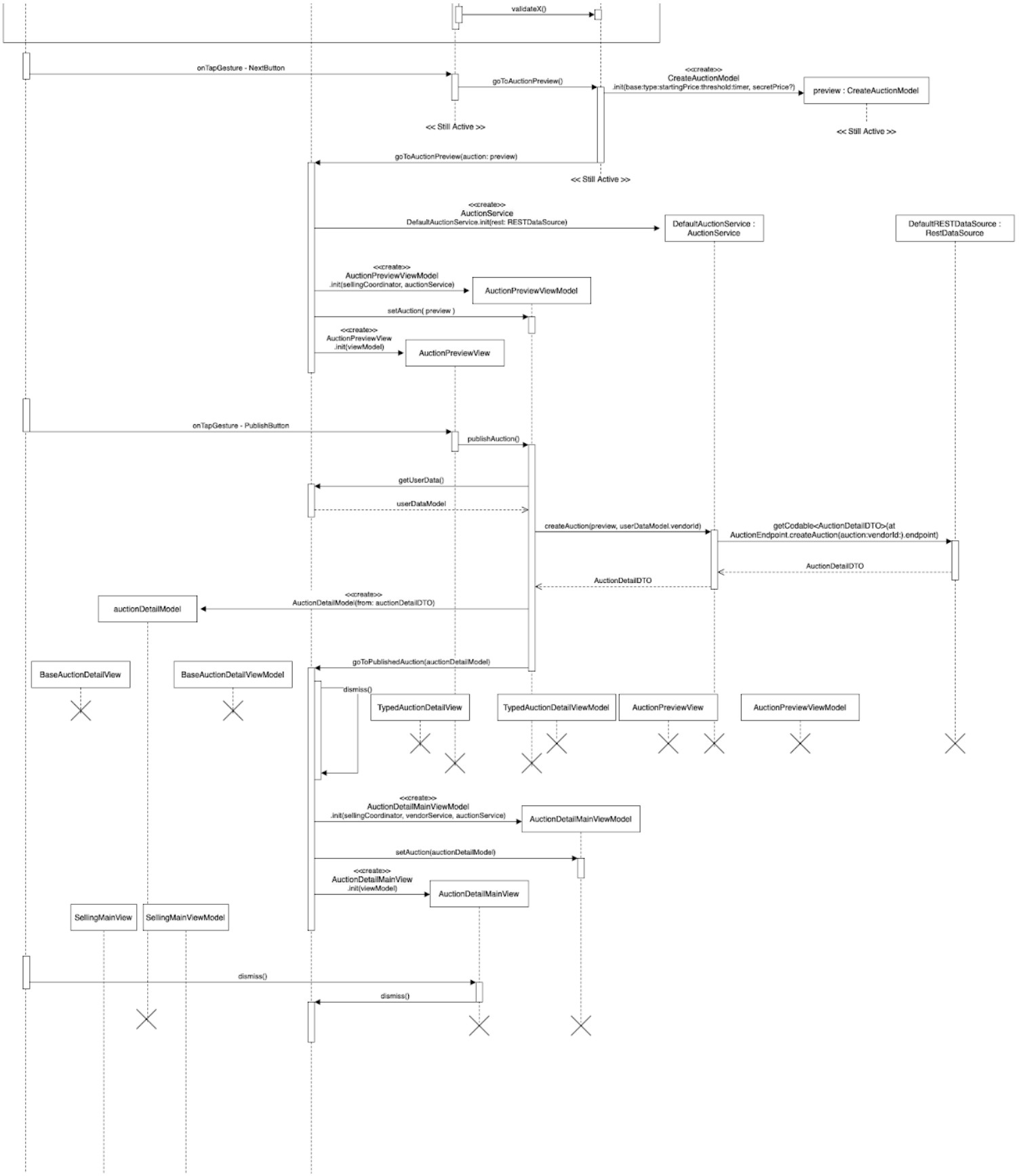
vii. Crea Asta



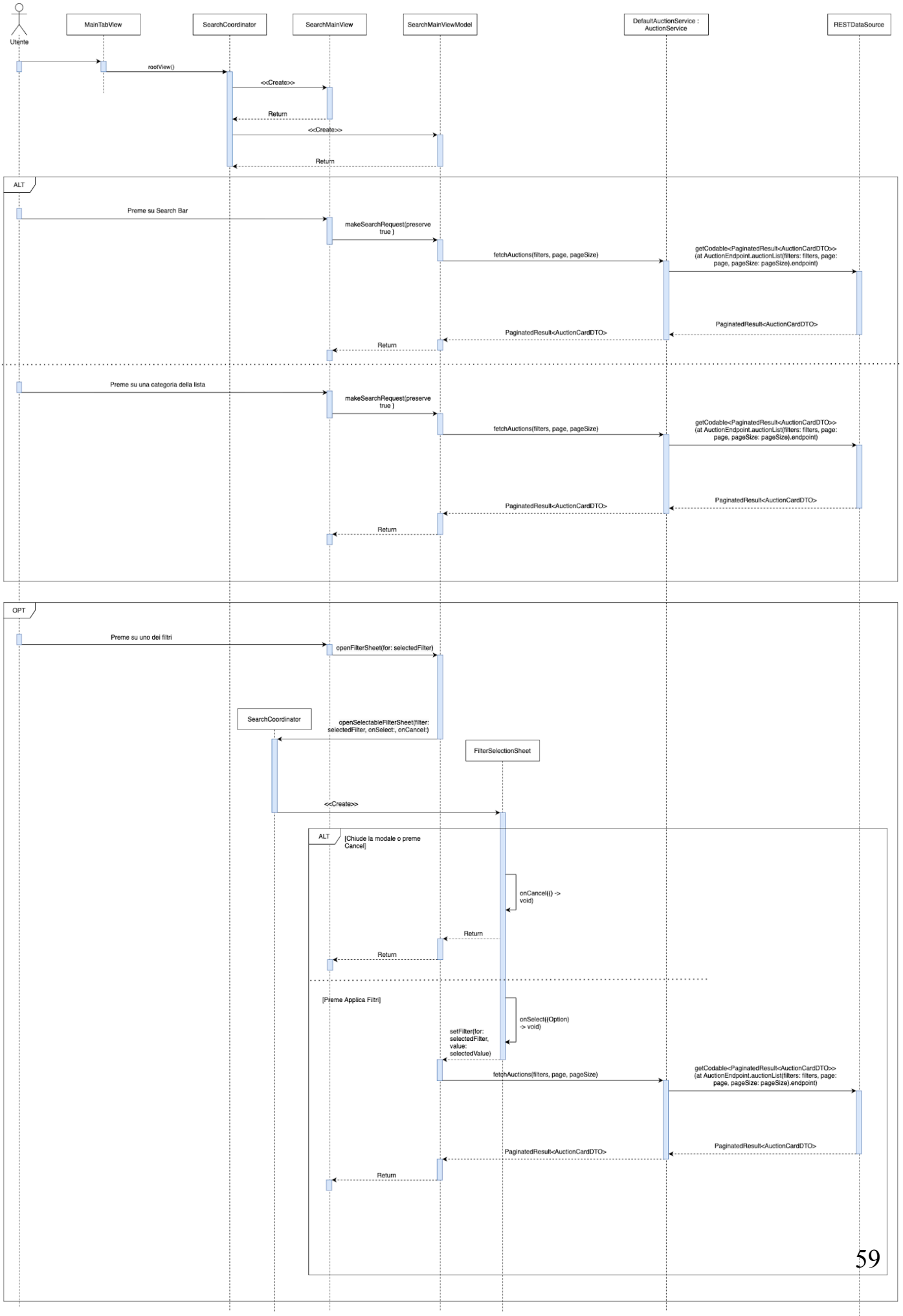
c. UML Sequence Diagram di Design

i. Creazione Asta



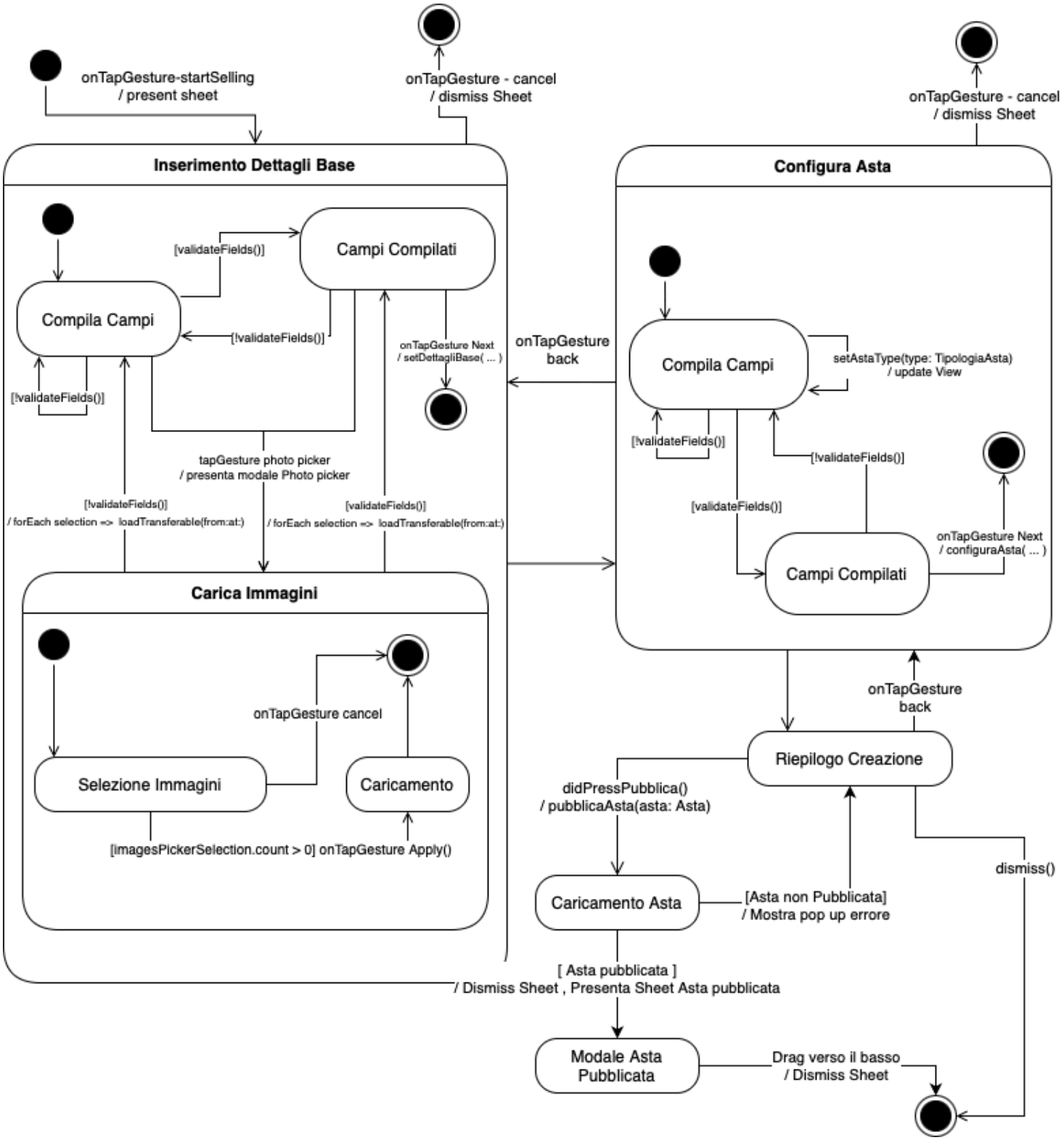

validateX()
onTapGesture - NextButton
goToAuctionPreview()
<<create>>
CreateAuctionModel
.init(base:type:startingPrice:threshold:timer, secretPrice?)
preview : CreateAuctionModel
<< Still Active >>
<< Still Active >>
goToAuctionPreview(auction: preview)
<< Still Active >>
<<create>>
AuctionService
DefaultAuctionService.init(rest: RESTDataSource)
DefaultAuctionService : AuctionService
DefaultRESTDataSource : RestDataSource
<<create>>
AuctionPreviewViewModel
.init(sellingCoordinator, auctionService)
AuctionPreviewViewModel
setAuction(preview)
<<create>>
AuctionPreviewView
.init(viewModel)
AuctionPreviewView
onTapGesture - PublishButton
publishAuction()
getUserData()
userDataModel
createAuction(preview, userDataModel.vendorId)
getCodable<AuctionDetailDTO>(at AuctionEndpoint.createAuction(auction:vendorId:).endpoint)
AuctionDetailDTO
AuctionDetailDTO
<<create>>
AuctionDetailModel(from: auctionDetailDTO)
auctionDetailModel
goToPublishedAuction(auctionDetailModel)
BaseAuctionDetailView
BaseAuctionDetailViewModel
dismiss()
TypedAuctionDetailView
TypedAuctionDetailViewModel
AuctionPreviewView
AuctionPreviewViewModel
<<create>>
AuctionDetailMainViewModel
.init(sellingCoordinator, vendorService, auctionService)
AuctionDetailMainViewModel
setAuction(auctionDetailModel)
<<create>>
AuctionDetailMainView
.init(viewModel)
AuctionDetailMainView
SellingMainView
SellingMainViewModel
dismiss()
dismiss()

ii. Ricerca Aste Attive

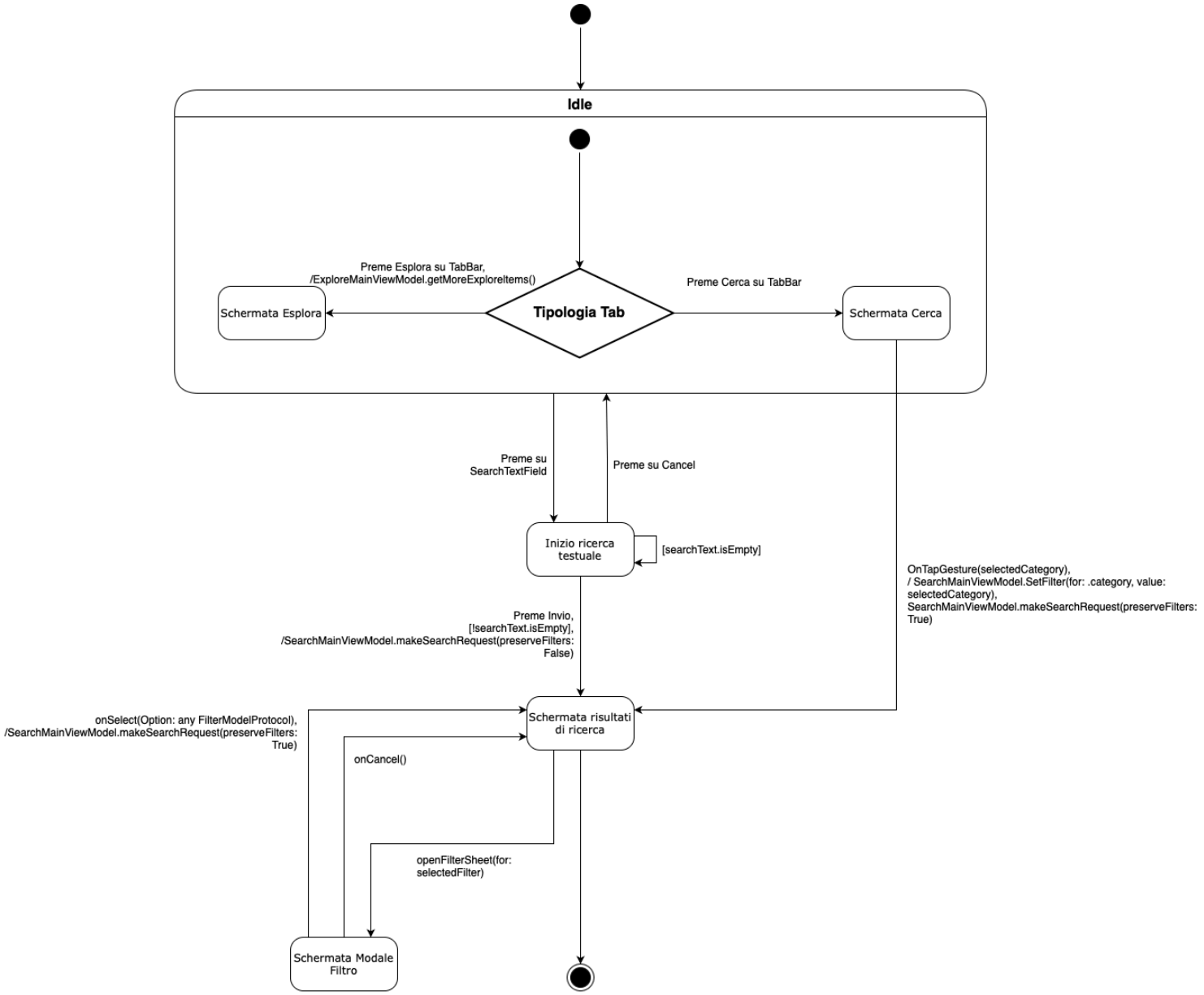


d. UML Statechart Diagram di Design

i. Creazione Asta



ii. Ricerca Aste Attive



6. Documento di Testing del Sistema

a. Codice xUnit

i. Validator.validateEmailAndPassword

Il metodo **validateEmailAndPassword** è responsabile della validazione di un indirizzo email e di una password forniti come input. Fa parte della struct **Validator**, che incapsula la logica di validazione degli input testuali nel progetto.

```
1 func validateEmailAndPassword(email: String, password: String) ->
  [ValidationError] {
2         var errors: [ValidationError] = []
3
4         if email.isEmpty {
5             errors.append(.emptyEmail)
6         } else if !isValidEmail(email) {
7             errors.append(.invalidEmail)
8         }
9
10        if password.isEmpty {
11            errors.append(.emptyPassword)
12        } else if !validatePassword(password) {
13            errors.append(.invalidPassword)
14        }
15
16        return errors
17    }
```

Input:

email: Una stringa (String) che rappresenta l'indirizzo email da validare.

password: Una stringa (String) che rappresenta la password da validare.

Output:

Un array di **ValidationError**. Questo array è vuoto se sia l'email che la password sono valide. Se vengono rilevati errori, l'array contiene uno o più valori dell'enum **ValidationError**, che rappresentano i tipi specifici di errore (ad esempio, emptyEmail, invalidEmail, emptyPassword, invalidPassword).

Comportamento:

Il metodo esegue una serie di controlli di validazione sia sull'email che sulla password.

Per l'email, verifica che non sia vuota e che sia conforme a un formato email valido (tramite l'utilizzo di una regular expression nel metodo di supporto isValidEmail).

Per la password, verifica che non sia vuota e che soddisfi determinati criteri di complessità (tramite l'utilizzo del metodo di supporto validatePassword):

- Lunghezza minima di 8 caratteri.
- Presenza di almeno una lettera maiuscola.
- Presenza di almeno una lettera minuscola.
- Presenza di almeno un carattere numerico.
- Presenza di almeno un carattere speciale.

Se vengono individuati uno o più errori, questi vengono aggiunti all'array errors.

L'array errors (vuoto o contenente gli errori) viene restituito.

Dipendenze (Esterne): Il metodo non ha dipendenze esterne *dirette*, ma si appoggia ai metodi interni **isValidEmail** e **validatePassword** per l'effettiva validazione.

Strategia

La strategia di testing adottata è di tipo **black-box**. Ciò significa che i test sono stati progettati esclusivamente sulla base delle specifiche del metodo **validateEmailAndPassword** (input, output, comportamento atteso), *senza* considerare la sua implementazione interna.

email : String

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
E1	Stringa email vuota	Non Valido	""	[.emptyEmail]
E2	Email priva del simbolo "@"	Non Valido	"testexample.com"	[.invalidEmail]
E3	Email priva della parte di dominio (dopo "@")	Non Valido	"test@"	[.invalidEmail]
E4	Email con caratteri non validi (prima di "@")	Non Valido	"!test@example.com"	[.invalidEmail]
E5	Email con caratteri non validi (dopo "@")	Non Valido	"test@example!.com"	[.invalidEmail]
E6	Email con più simboli "@"	Non Valido	"test@@example.com"	[.invalidEmail]
E7	Formato email valido, base	Valido	"test@example.com"	[]
E8	Email valida con sotto-dominio	Valido	"test@sub.example.com"	[]
E9	Email valida con caratteri speciali (+, ., _)	Valido	"test.name+alias@example.co.uk"	[]

password: String

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
P1	Stringa password vuota	Non Valido	""	[.emptyPassword]
P2	Password più corta di 8 caratteri	Non Valido	"short"	[.invalidPassword]

P3	Password con 8 o più caratteri, tutte minuscole	Non Valido	"password123"	[.invalidPassword]
P4	Password con 8 o più caratteri, tutte maiuscole	Non Valido	"PASSWORD123"	[.invalidPassword]
P5	Password con 8 o più caratteri, senza numeri	Non Valido	"Password!!"	[.invalidPassword]
P6	Password con 8 o più caratteri, senza caratteri speciali	Non Valido	"Password123"	[.invalidPassword]
P7	Password valida (>= 8 caratteri, maiuscola, minuscola, numero, carattere speciale)	Valido	"Pass123!"	[]
P8	Password valida, lunga (>= 200 caratteri)	Valido	"\(String(repeating: "a", count: 197))Abc1"	[]

Casi di test

Metodo di test	Classi Email	Classi Password	Note
validEmailAndPassword	E7	P7	Caso base: entrambi validi.
emptyEmail	E1	P7	Email vuota, password valida.
invalidEmail	E2,E4,E5,E6	P7	Email invalida (generico, copre più casistiche di formato non valido), password valida.
invalidEmail_MissingAtSymbol	E2	P7	Email invalida (manca "@"), password valida.
invalidEmail_MissingDomain	E3	P7	Email invalida (manca il dominio), password valida.
emptyPassword	E7	P1	Email valida, password vuota.
invalidPassword_TooShort	E7	P2	Email valida, password troppo corta.
invalidPassword_NoUppercase	E7	P3	Email valida, password senza maiuscole. (Nota: P3 include implicitamente la lunghezza >= 8, quindi copre anche il caso di password solo minuscole).

invalidPassword_NoLowercase	E7	P4	Email valida, password senza minuscole. (Nota: P4 include implicitamente la lunghezza >= 8, quindi copre anche il caso di password solo maiuscole).
invalidPassword_NoNumber	E7	P5	Email valida, password senza numeri.
invalidPassword_NoSpecialChar	E7	P6	Email valida, password senza caratteri speciali.
emptyEmailAndEmptyPassword	E1	P1	Sia email che password vuote.
invalidEmailAndInvalidPassword	E2,E4,E5,E6	P2,P3,P4,P5,P6	Sia email che password invalide (varie combinazioni, utile per testare interazioni).
longPassword	E7	P8	Email valida, password molto lunga (test di robustezza/caso limite).
validEmailWithSpecialChars	E9	P7	Email valida con caratteri speciali, password valida.
passwordWithSpecialChars	E7	P7	Email valida, password valida con caratteri speciali
invalidCases	E1,E2,E3,E4,E5,E6,E7	P1,P2,P3,P4,P5,P6	Questo metodo copre in un ciclo diverse combinazioni di classi non valide

Codice Swift Testing

```swift
@Suite("Validator Tests")
struct ValidatorTests{

    static let validator = Validator()

    struct EmailPasswordValidation_BlackBox_Tests {

        //MARK: test valid email and password
        @Test func validEmailAndPassword() {
            let errors = validator.validateEmailAndPassword(email: "test@example.com", password: "ValidPassword123!")
            #expect(errors.isEmpty)
        }

        //MARK: test empty email
        @Test func emptyEmail() {
            let errors = validator.validateEmailAndPassword(email: "", password: "ValidPassword123!")
            #expect(errors == [.emptyEmail])
        }

        //MARK: test invalid email
        @Test func invalidEmail() {
            let errors = validator.validateEmailAndPassword(email: "invalid-email", password: "ValidPassword123!")
            #expect(errors == [.invalidEmail])
        }

        @Test func invalidEmail_MissingAtSymbol()  {
            let errors = validator.validateEmailAndPassword(email: "testexample.com", password: "ValidPassword123!")
            #expect(errors == [.invalidEmail])
        }

        @Test func invalidEmail_MissingDomain() {
            let errors = validator.validateEmailAndPassword(email: "test@", password: "ValidPassword123!")
            #expect(errors == [.invalidEmail])
        }

        //MARK: test empty password
        @Test func emptyPassword() {
            let errors = validator.validateEmailAndPassword(email: "test@example.com", password: "")
            #expect(errors == [.emptyPassword])
        }

        //MARK: test invalid password
          @Test func invalidPassword_TooShort() {
            let errors = validator.validateEmailAndPassword(email: "test@example.com", password: "short")
            #expect(errors == [.invalidPassword])
        }

        @Test func invalidPassword_NoUppercase() {
            let errors = validator.validateEmailAndPassword(email: "test@example.com", password: "password123!")
            #expect(errors == [.invalidPassword])
        }

        @Test func invalidPassword_NoLowercase() {
            let errors = validator.validateEmailAndPassword(email: "test@example.com", password: "PASSWORD123!")
            #expect(errors == [.invalidPassword])
        }
```

```
57
58          @Test func invalidPassword_NoNumber() {
59              let errors = validator.validateEmailAndPassword(email: "test@example.com",
    password: "Password!")
60              #expect(errors == [.invalidPassword])
61          }
62
63          @Test func invalidPassword_NoSpecialChar() {
64              let errors = validator.validateEmailAndPassword(email: "test@example.com",
    password: "Password123")
65              #expect(errors == [.invalidPassword])
66          }
67          //MARK: test all invalid
68          @Test func emptyEmailAndEmptyPassword() {
69              let errors = validator.validateEmailAndPassword(email: "", password: "")
70              #expect(errors == [.emptyEmail, .emptyPassword])
71          }
72
73          @Test func invalidEmailAndInvalidPassword() {
74              let errors = validator.validateEmailAndPassword(email: "invalid", password:
    "short")
75              #expect(errors.count == 2)
76              #expect(errors.contains(.invalidEmail))
77              #expect(errors.contains(.invalidPassword))
78          }
79
80          //MARK: Test long password
81          @Test func longPassword() {
82              let longPassword = String(repeating: "a", count: 200) + "A1!" // Assicura che
    sia valida anche se lunga
83              let errors = validator.validateEmailAndPassword(email: "test@example.com",
    password: longPassword)
84              #expect(errors.isEmpty)
85          }
86
87          //MARK: test valid email with special characters
88          @Test func validEmailWithSpecialChars() {
89              let errors = validator.validateEmailAndPassword(email:
    "test.name+alias@example.co.uk", password: "ValidPassword123!")
90              #expect(errors.isEmpty)
91          }
92
93          @Test func passwordWithSpecialChars() {
94              let errors = validator.validateEmailAndPassword(email: "test@example.com",
    password: "P@$$wOrd1")
95              #expect(errors.isEmpty)
96          }
97
98          //MARK: Test invalid cases, all together.
99          @Test func invalidCases() {
100             let testCases: [(email: String, pass: String, expected:
    [Validator.ValidationError])] = [
101                 ("", "", [.emptyEmail, .emptyPassword]),
102                 ("invalid", "short", [.invalidEmail, .invalidPassword]),
103                 ("test@", "Password123!", [.invalidEmail]),
104                 ("test@example.com", "short", [.invalidPassword]),
105                 ("test@example.com", "password123!", [.invalidPassword]), // No uppercase
106                 ("test@example.com", "PASSWORD123!", [.invalidPassword]), // No lowercase
107                 ("test@example.com", "Password!", [.invalidPassword]),  // No number
108                 ("test@example.com", "Password123", [.invalidPassword]),  // No special char
109                 ("testexample.com", "ValidPassword123!", [.invalidEmail]), // Invalid email
    format
110             ]
111
112             for (email, password, expectedErrors) in testCases {
113                 let errors = validator.validateEmailAndPassword(email: email, password:
    password)
114                 #expect(errors == expectedErrors, "Failed for email: \(email), password: \
    (password)")
115             }
116         }
117     }
118 }
```

ii. AuctionService.CreateAuctionAsync

Il metodo CreateAuctionAsync è responsabile della creazione di una nuova entità di tipo "Auction" e del suo salvataggio nel database. Si occupa anche di formattare correttamente la data di inizio dell'asta prima di inserirla nell'entità.

```
public async Task<Auction> CreateAuctionAsync(CreateAuctionDTO auction, Vendor vendor)
{
    try
    {
        if (auction == null) throw new ArgumentNullException(nameof(auction), "[SERVICE]
CreateAuctionDTO is null.");
        if (vendor == null) throw new ArgumentNullException(nameof(vendor), "[SERVICE] Vendor does not
exist.");

        DateTime now = DateTime.Now;
        DateTime startingDate = new DateTime(now.Year, now.Month, now.Day, now.Hour, now.Minute,
now.Second);

        var newAuction = new Auction
        {
            Title = auction.Title,
            AuctionDescription = auction.Description,
            StartingPrice = auction.StartingPrice,
            CurrentPrice = auction.StartingPrice,
            AuctionType = auction.Type,
            Threshold = auction.Threshold,
            Timer = auction.ThresholdTimer,
            SecretPrice = auction.SecretPrice,
            VendorId = vendor.Id,
            AuctionState = AuctionState.Open,
            StartingDate = startingDate,
            EndingDate = startingDate.AddHours(auction.ThresholdTimer),
            Category = auction.Category
        };

        _unitOfWork.BeginTransaction();
        await _unitOfWork.AuctionRepository.Add(newAuction);
        _unitOfWork.Commit();
        await _unitOfWork.Save();

        return newAuction;
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, $"[SERVICE] Creating new auction failed. Exception occurred::
{ex.Message}");
        throw new Exception($"[SERVICE] Creating new auction failed. Exception occurred: {ex.Message}",
ex);
    }
}
```

Input:

- **auction**: Un Data Transfer Object (CreateAuctionDTO) contenente i dati per la creazione dell'asta:
    - Title (string): Titolo dell'asta.
    - Description (string): Descrizione dell'asta.

    - Type (AuctionType): Tipo di asta (custom).
    - Category (AuctionCategory): Categoria dell'asta (custom).
    - StartingPrice (decimal): Prezzo di partenza dell'asta.
    - Threshold (decimal): Soglia per le offerte.
    - ThresholdTimer (int): Durata dell'asta in ore.
    - ImagesIdentifiers (List<Guid>, opzionale): Identificatori delle immagini dell'asta.
    - SecretPrice (decimal, opzionale): Prezzo segreto dell'asta.
    - VendorId (Guid): Identificatore del venditore.
- **vendor**: Un'entità Vendor dal database, contenente:
    - Id (Guid): Identificatore del venditore.
    - UserId (Guid): Identificatore dell'utente associato al venditore.
    - GeoLocation (string, opzionale): Geolocalizzazione del venditore.
    - WebSiteUrl (string, opzionale): URL del sito web del venditore.
    - ShortBio (string, opzionale): Breve biografia del venditore.
    - StartingDate (DateTime): Data di inizio attività del venditore.
    - SuccessfulAuctions (int): Numero di aste concluse con successo dal venditore.
    - Proprietà di navigazione: User, Auctions.

Output:

- L'entità Auction appena creata, se l'operazione ha successo.
- Il metodo può lanciare ArgumentNullException se auction o vendor sono null.
- Il metodo può lanciare Exception per altri errori durante l'esecuzione.

Comportamento:

1. **Validazione dei parametri:**
    - Il metodo verifica che auction e vendor non siano null.
    - Se uno dei parametri non è valido, viene lanciata un'eccezione ArgumentNullException e viene registrato un errore nel logger.
2. **Preparazione dei dati:**
    - Il metodo ottiene la data e l'ora correnti e le formatta per l'inizio dell'asta.
    - Calcola la data di fine dell'asta aggiungendo la durata specificata (ThresholdTimer) alla data di inizio.
3. **Creazione dell'entità Auction:**

    - Il metodo crea una nuova entità Auction con i dati forniti nel CreateAuctionDTO e le date calcolate.
    - Imposta lo stato dell'asta su "Open".
4. **Gestione della transazione:**
    - Il metodo avvia una transazione con _unitOfWork.BeginTransaction().
    - Aggiunge la nuova entità Auction al repository delle aste (_unitOfWork.AuctionRepository.Add(newAuction)).
    - Esegue il commit della transazione (_unitOfWork.Commit()).
    - Salva le modifiche nel database (_unitOfWork.Save()).
5. **Restituzione del risultato:**
    - Il metodo restituisce l'entità Auction appena creata.
6. **Gestione degli errori:**
    - Se si verifica un'eccezione durante il processo di creazione dell'asta, viene registrato un errore nel logger e viene rilanciata un'eccezione generica.

Dipendenze Esterne:

- **_logger**: Un'istanza di un logger per la registrazione di messaggi informativi e di errore.
- **_unitOfWork**: Un'istanza di UnitOfWork per gestire le transazioni e l'accesso alla repository delle aste.

Strategia di Testing:

La strategia di testing adottata è di tipo black-box. Ciò significa che i test sono stati progettati esclusivamente sulla base delle specifiche del metodo CreateAuctionAsync (input, output, comportamento atteso), senza considerare la sua implementazione interna.

auction : CreateAuctionDTO

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
E1	auction è nullo	Non Valido	null	Exception
E2	auction ha valori minimi per i suoi campi	Valido	Title = "", Description = "", Type = AuctionType.Incremental, Category = AuctionCategory.Services, StartingPrice = 0, Threshold = 0, ThresholdTimer = 0, ImagesIdentifiers = new List<Guid>(), SecretPrice = null, VendorId = 78afb6a7-2e31-42a7-a758-7732e592dba4	[]
E3	auction ha valori massimi per i suoi campi	Valido	Title = new string('A', 500), Description = new string('B', 1000), Type = AuctionType.Incremental, Category = AuctionCategory.Services, StartingPrice = decimal.MaxValue, Threshold = int.MaxValue, ThresholdTimer = 8760, ImagesIdentifiers = new List<Guid>(Enumerable.Range(0,	[]

			100).Select(_ => Guid.NewGuid())), SecretPrice = decimal.MaxValue, VendorId = 78afb6a7-2e31-42a7-a758-7732e592dba4	
E4	auction ha valori compresi tra il range minimo e massimo per i suoi campi	Valido	Title = "iPhone 12 Usato - Buone condizioni", Description = "Vendo il mio iPhone 12. Nella scatola è incluso il caricatore.", Type = AuctionType.Incremental, Category = AuctionCategory.Services, StartingPrice = 200, Threshold = 25, ThresholdTimer = 3, ImagesIdentifiers = new List<Guid> { 384af91c-0d3a-4fa2-ac17-d5a9042e43ad, 184ef91c-0d3a-4fa2-ac17-d5a2042e43af, 124af91c-0d3a-4fa2-ac16-d5a9242e43ad }, SecretPrice = null, VendorId = 78afb6a7-2e31-42a7-a758-7732e592dba4	[]

vendor : Vendor

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
E1	vendor è nullo	Non Valido	null	Exception
E2	vendor ha valori minimi per i suoi campi	Valido	“Id = 78afb6a7-2e31-42a7-a758-7732e592dba4, UserId = 72159444-d071-7025-645c-45e0fd6fed46, GeoLocation = “”, WebSiteUrl = “”, ShortBio = "", StartingDate = DateTime.MinValue, SuccessfulAuctions = 0”	[]
E3	vendor ha valori massimi per i suoi campi	Valido	“Id = 78afb6a7-2e31-42a7-a758-7732e592dba4, UserId = 72159444-d071-7025-645c-45e0fd6fed46, GeoLocation = new string('X', 500), WebSiteUrl = "https://" + new string('Y', 490) + ".com", ShortBio = new string('Z', 1000), StartingDate = DateTime.MaxValue, SuccessfulAuctions = int.MaxValue”	[]
E4	vendor ha valori compresi	Valido	“Id =	[]

	tra il range minimo e massimo per i suoi campi		78afb6a7-2e31-42a7-a758-7732e592dba4, UserId = 72159444-d071-7025-645c-45e0fd6fed46, GeoLocation = "Napoli", WebSiteUrl = "www.unsempliceve nditore.com", ShortBio = "Un semplice venditore di Napoli.", StartingDate = 2025-03-12, SuccessfulAuctions = 2”	

Codice xUnit Test

```
public class AuctionServiceTests
{
    private readonly AuctionService _auctionService;
    private readonly Mock<IRepository<Auction, Guid>> _mockAuctionRepository;
    private readonly Mock<IUnitOfWork> _mockUnitOfWork;

    public AuctionServiceTests()
    {
        var mockLogger = new Mock<ILogger<AuctionService>>();
        _mockUnitOfWork = new Mock<IUnitOfWork>();
        _mockAuctionRepository = new Mock<IRepository<Auction, Guid>>();

        _mockUnitOfWork.Setup(unit => unit.AuctionRepository).Returns(_mockAuctionRepository.Object);
        _auctionService = new AuctionService(_mockUnitOfWork.Object, mockLogger.Object);
    }

    private static DateTime GetValidDate()
    {
        DateTime now = DateTime.Now;
        DateTime validDate = new DateTime(now.Year, now.Month, now.Day, now.Hour, now.Minute,
now.Second);
        return validDate;
    }

    private static Vendor GetValidInputForVendor()
    {
        var vendor = new Vendor
        {
            Id = Guid.NewGuid(),
            UserId = Guid.NewGuid(),
            GeoLocation = "Napoli",
            WebSiteUrl = "www.unsempliceveditore.com",
            ShortBio = "Un semplice venditore di Napoli.",
            StartingDate = GetValidDate(),
            SuccessfulAuctions = 2
        };

        return vendor;
    }

    private static CreateAuctionDTO GetValidInputForCreateAuctionDTO()
    {
        var createAuctionDto = new CreateAuctionDTO
        {
            Title = "iPhone 12 Usato - Buone condizioni",
            Description = "Vendo il mio iPhone 12. Nella scatola è incluso il caricatore.",
            Type = AuctionType.Incremental,
            Category = AuctionCategory.Services,
            StartingPrice = 200,
            Threshold = 25,
            ThresholdTimer = 3,
            ImagesIdentifiers = new List<Guid>
            {
                Guid.NewGuid(),
                Guid.NewGuid(),
                Guid.NewGuid()
            },
            SecretPrice = null,
            VendorId = Guid.NewGuid(),
        };

        return createAuctionDto;
    }
```

```
[Fact]
public async Task CreateAuctionAsync_TestWithValidInut()
{
    // Arrange
    var vendor = GetValidInputForVendor();
    var auctionDTO = GetValidInputForCreateAuctionDTO();
    auctionDTO.VendorId = vendor.Id;

    _mockAuctionRepository.Setup(repo => repo.Add(It.IsAny<Auction>()));
    _mockUnitOfWork.Setup(unit => unit.Save()).ReturnsAsync(1);

    // Act
    var result = await _auctionService.CreateAuctionAsync(auctionDTO, vendor);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(auctionDTO.Title, result.Title);
    Assert.Equal(auctionDTO.VendorId, result.VendorId);
}

[Fact]
public async Task CreateAuctionAsync_TestWithMinValueDTO_ValidVendor()
{
    // Arrange
    var vendor = GetValidInputForVendor();
    var auctionDTO = new CreateAuctionDTO
    {
        Title = "",
        Description = "",
        Type = AuctionType.Incremental,
        Category = AuctionCategory.Services,
        StartingPrice = 0,
        Threshold = 0,
        ThresholdTimer = 0,
        ImagesIdentifiers = new List<Guid>(),
        SecretPrice = null,
        VendorId = vendor.Id
    };

    _mockAuctionRepository.Setup(repo => repo.Add(It.IsAny<Auction>()));
    _mockUnitOfWork.Setup(unit => unit.Save()).ReturnsAsync(1);

    // Act
    var result = await _auctionService.CreateAuctionAsync(auctionDTO, vendor);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(auctionDTO.Title, result.Title);
}
```

```
[Fact]
public async Task CreateAuctionAsync_TestWithMaxValueDTO_ValidVendor()
{
    // Arrange
    var vendor = GetValidInputForVendor();
    var auctionDTO = new CreateAuctionDTO
    {
        Title = new string('A', 500),
        Description = new string('B', 1000),
        Type = AuctionType.Incremental,
        Category = AuctionCategory.Services,
        StartingPrice = decimal.MaxValue,
        Threshold = int.MaxValue,
        ThresholdTimer = 8760, //1 anno
        ImagesIdentifiers = new List<Guid>(Enumerable.Range(0, 100).Select(_ => Guid.NewGuid())),
        SecretPrice = decimal.MaxValue,
        VendorId = vendor.Id
    };

    _mockAuctionRepository.Setup(repo => repo.Add(It.IsAny<Auction>()));
    _mockUnitOfWork.Setup(unit => unit.Save()).ReturnsAsync(1);

    // Act
    var result = await _auctionService.CreateAuctionAsync(auctionDTO, vendor);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(auctionDTO.Title, result.Title);
}

[Fact]
public async Task CreateAuctionAsync_TestWithValidDTO_MinValueVendor()
{
    // Arrange
    var vendor = new Vendor
    {
        Id = Guid.NewGuid(),
        UserId = Guid.NewGuid(),
        GeoLocation = "",
        WebSiteUrl = "",
        ShortBio = "",
        StartingDate = DateTime.MinValue,
        SuccessfulAuctions = 0
    };

    var auctionDTO = GetValidInputForCreateAuctionDTO();
    auctionDTO.VendorId = vendor.Id;

    _mockAuctionRepository.Setup(repo => repo.Add(It.IsAny<Auction>()));
    _mockUnitOfWork.Setup(unit => unit.Save()).ReturnsAsync(1);

    // Act
    var result = await _auctionService.CreateAuctionAsync(auctionDTO, vendor);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(auctionDTO.Title, result.Title);
}
```

```
    [Fact]
    public async Task CreateAuctionAsync_TestWithValidDTO_MaxValueVendor()
    {
        // Arrange
        var vendor = new Vendor
        {
            Id = Guid.NewGuid(),
            UserId = Guid.NewGuid(),
            GeoLocation = new string('X', 500),
            WebSiteUrl = "https://" + new string('Y', 490) + ".com",
            ShortBio = new string('Z', 1000),
            StartingDate = DateTime.MaxValue,
            SuccessfulAuctions = int.MaxValue
        };

        var auctionDTO = GetValidInputForCreateAuctionDTO();
        auctionDTO.VendorId = vendor.Id;

        _mockAuctionRepository.Setup(repo => repo.Add(It.IsAny<Auction>()));
        _mockUnitOfWork.Setup(unit => unit.Save()).ReturnsAsync(1);

        // Act
        var result = await _auctionService.CreateAuctionAsync(auctionDTO, vendor);

        // Assert
        Assert.NotNull(result);
        Assert.Equal(auctionDTO.Title, result.Title);
    }

    [Fact]
    public async Task CreateAuctionAsync_TestWithNullDTO()
    {
        // Arrange
        var vendor = GetValidInputForVendor();
        CreateAuctionDTO auctionDTO = null;

        // Act & Assert
        await Assert.ThrowsAsync<Exception>(() => _auctionService.CreateAuctionAsync(auctionDTO,
vendor));
    }

    [Fact]
    public async Task CreateAuctionAsync_TestWithNullVendor()
    {
        // Arrange
        Vendor vendor = null;
        var auctionDTO = GetValidInputForCreateAuctionDTO();

        // Act & Assert
        await Assert.ThrowsAsync<Exception>(() => _auctionService.CreateAuctionAsync(auctionDTO,
vendor));
    }
}
```

iii. NotificationWorker.AddNotificationAsync

Il metodo AddNotificationTokenAsync è responsabile della registrazione di un token del dispositivo a cui inviare notifiche (device token) per un utente specifico, consentendo l'invio di notifiche push tramite AWS SNS (Simple Notification Service).

```
public async Task<Auction> CreateAuctionAsync(CreateAuctionDTO auction, Vendor vendor)
{
    try
    {
        if (auction == null) throw new ArgumentNullException(nameof(auction), "[SERVICE]
CreateAuctionDTO is null.");
        if (vendor == null) throw new ArgumentNullException(nameof(vendor), "[SERVICE] Vendor does not
exist.");

        DateTime now = DateTime.Now;
        DateTime startingDate = new DateTime(now.Year, now.Month, now.Day, now.Hour, now.Minute,
now.Second);

        var newAuction = new Auction
        {
            Title = auction.Title,
            AuctionDescription = auction.Description,
            StartingPrice = auction.StartingPrice,
            CurrentPrice = auction.StartingPrice,
            AuctionType = auction.Type,
            Threshold = auction.Threshold,
            Timer = auction.ThresholdTimer,
            SecretPrice = auction.SecretPrice,
            VendorId = vendor.Id,
            AuctionState = AuctionState.Open,
            StartingDate = startingDate,
            EndingDate = startingDate.AddHours(auction.ThresholdTimer),
            Category = auction.Category
        };

        _unitOfWork.BeginTransaction();
        await _unitOfWork.AuctionRepository.Add(newAuction);
        _unitOfWork.Commit();
        await _unitOfWork.Save();

        return newAuction;
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, $"[SERVICE] Creating new auction failed. Exception occurred::
{ex.Message}");
        throw new Exception($"[SERVICE] Creating new auction failed. Exception occurred: {ex.Message}",
ex);
    }
}
```

Input:

- **userId**: Un Guid che identifica l'utente per cui si sta registrando il token di notifica.
- **deviceToken**: Una stringa che rappresenta il token del dispositivo (ottenuto da Apple Push Notification Service).

Output:

- Il metodo non restituisce un valore esplicito (void).
- Il metodo può lanciare un'eccezione di tipo Exception se uno dei parametri di input è vuoto o se si verificano errori durante la registrazione del token con AWS SNS.

Comportamento:

1. **Validazione dei parametri:**
    - Il metodo verifica che userId e deviceToken non siano vuoti o null.
    - Se uno dei parametri non è valido, viene registrato un errore nel logger e viene lanciata un'eccezione.
2. **Verifica della registrazione esistente:**
    - Il metodo utilizza _notificationService.GetEndPointArnFromDeviceTokenAsync (deviceToken) per verificare se il deviceToken è già stato registrato.
    - Se il token è già registrato, viene registrato un messaggio informativo nel logger e il metodo termina.
3. **Registrazione del token con AWS SNS:**
    - Il metodo utilizza _snsClient.CreatePlatformEndpointAsync per registrare il deviceToken con AWS SNS.
    - Viene fornito l'ARN dell'applicazione della piattaforma (_platformApplicationArn), il deviceToken e l'ID dell'utente (userId) come dati personalizzati.
    - L'ARN dell'endpoint creato viene memorizzato nella variabile endpointArn.
    - Viene registrato un messaggio informativo nel logger con l'endpointArn creato.
4. **Memorizzazione dell'ARN dell'endpoint:**
    - Il metodo utilizza _notificationService.AddNotificationTokenAsync(userId, deviceToken, endpointArn) per memorizzare l'ARN dell'endpoint nel sistema, associandolo all'utente e al token del dispositivo.
    - Viene registrato un messaggio informativo nel logger di avvenuto salvataggio.

5. **Gestione degli errori:**
    - Se si verifica un'eccezione durante la registrazione del token con AWS SNS, viene registrato un errore nel logger e viene rilanciata l'eccezione.

Dipendenze Esterne:

- **_logger**: Un'istanza di un logger per la registrazione di messaggi informativi e di errore.
- **_notificationService**: Un servizio che fornisce metodi per interagire con il sistema di notifiche (ad esempio, per verificare la registrazione esistente e memorizzare l'ARN dell'endpoint).
- **_snsClient**: Un client per interagire con AWS SNS.
- **_platformApplicationArn**: L'ARN dell'applicazione della piattaforma AWS SNS.

Strategia Black Box

La prima strategia di testing adottata è di tipo **Black Box**. Ciò significa che i test sono stati progettati esclusivamente sulla base delle specifiche del metodo **AddNotificationAsync**(input, output, comportamento atteso), *senza* considerare la sua implementazione interna.

userId : Guid

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
E1	userId è nullo	Non Valido	null	Exception
E2	userId ha un valore che non sia nullo.	Valido	“78afb6a7-2e31-42a7-a758-7732e592dba4”	[]

deviceToken : string

ID Classe di	Descrizione	Valido/Non	Esempio Input	Output

Equivalenza		Valido		Previsto (Errori)
E1	deviceToken è nullo	Non Valido	null	Exception
E2	deviceToken è una stringa vuota	Non Valido	“”	Exception
E3	deviceToken è una stringa non vuota e sotto la lunghezza massima	Valido	“359015081997370 ”	[]
E4	deviceToken ha una lunghezza di 300 caratteri	Valido	“new string(‘a’, 300)”	[]

Codice xUnit Black Box Test

```
public class BlackBoxTests
        {
            private readonly NotificationWorker _notificationWorker;

            public BlackBoxTests()
            {
                var mockLogger = new Mock<ILogger<NotificationWorker>>();
                var mockNotificationService = new Mock<INotificationService>();
                var mockNotificationClient = new Mock<IAmazonSimpleNotificationService>();
                mockNotificationClient.Setup(client =>
                        client.CreatePlatformEndpointAsync(It.IsAny<CreatePlatformEndpointRequest>(),
                            It.IsAny<CancellationToken>()))
                    .ReturnsAsync(new CreatePlatformEndpointResponse
                        { EndpointArn = "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone" });
                _notificationWorker = new NotificationWorker(mockLogger.Object,
                    mockNotificationService.Object,
                    mockNotificationClient.Object);
            }

            [Fact]
            public async Task AddNotificationTokenAsync_TestWithValidInput()
            {
                // Arrange
                var userId = GetValidUserId();
                var deviceToken = GetValidDeviceToken();

                //Act
                var result = _notificationWorker.AddNotificationTokenAsync(userId, deviceToken);
                await result;

                //Assert
                Assert.True(result.IsCompletedSuccessfully);
            }

            [Fact]
            public async Task AddNotificationTokenAsync_TestWithEmptyUserId()
            {
                // Arrange
                var userId = Guid.Empty;
                var deviceToken = GetValidDeviceToken();

                // Act & Assert
                await Assert.ThrowsAsync<Exception>(() =>
                    _notificationWorker.AddNotificationTokenAsync(userId, deviceToken));
            }
```

```
        }

        [Fact]
        public async Task AddNotificationTokenAsync_TestWithEmptyDeviceToken()
        {
            // Arrange
            var userId = GetValidUserId();
            var deviceToken = "";

            // Act & Assert
            await Assert.ThrowsAsync<Exception>(() =>
                _notificationWorker.AddNotificationTokenAsync(userId, deviceToken));
        }

        [Fact]
        public async Task AddNotificationTokenAsync_TestWithNullDeviceToken()
        {
            // Arrange
            var userId = GetValidUserId();
            string deviceToken = null;

            //Act & Assert
            await Assert.ThrowsAsync<Exception>(() =>
                _notificationWorker.AddNotificationTokenAsync(userId, deviceToken));
        }

        [Fact]
        public async Task AddNotificationTokenAsync_TestWithLongDeviceToken()
        {
            // Arrange
            var userId = GetValidUserId();
            var deviceToken = new string('a', 300);

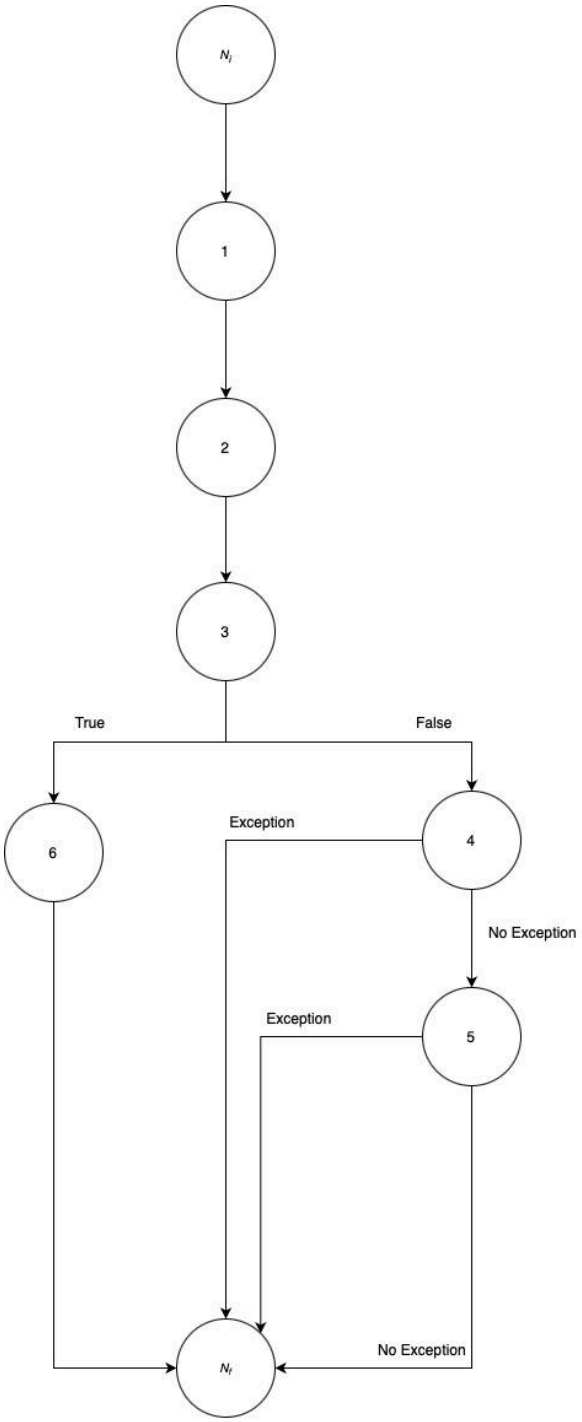
            //Act
            var result = _notificationWorker.AddNotificationTokenAsync(userId, deviceToken);
            await result;

            //Assert
            Assert.True(result.IsCompletedSuccessfully);
        }
    }
```

Strategia White Box

La strategia di Black Box precedentemente descritta non riesce a coprire l'interezza del metodo precedentemente esaminato, dunque risulta necessaria una strategia White Box, per la quale si fornisce il seguente Grafo dei flussi di controllo e i test sui possibili percorsi di esecuzione.

Grafo



Codice xUnit White Box Test

```
public class WhiteBoxTests
        {
            private readonly NotificationWorker _notificationWorker;
            private readonly Mock<ILogger<NotificationWorker>> _mockLogger;
            private readonly Mock<INotificationService> _mockNotificationService;
            private readonly Mock<IAmazonSimpleNotificationService> _mockNotificationClient;

            public WhiteBoxTests()
            {
                _mockLogger = new Mock<ILogger<NotificationWorker>>();
                _mockNotificationService = new Mock<INotificationService>();
                _mockNotificationClient = new Mock<IAmazonSimpleNotificationService>();
                _notificationWorker = new NotificationWorker(_mockLogger.Object,
                    _mockNotificationService.Object,
                    _mockNotificationClient.Object);
            }

            [Fact]
            public async Task TestPath_1_2_3_4_5_Success()
            {
                //Arrange
                var userId = GetValidUserId();
                var deviceToken = GetValidDeviceToken();
                var endpointArn = "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone";

                _mockNotificationService.Setup(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken));
                _mockNotificationClient.Setup(client =>
                        client.CreatePlatformEndpointAsync(It.IsAny<CreatePlatformEndpointRequest>(),
                            It.IsAny<CancellationToken>()))
                    .ReturnsAsync(new CreatePlatformEndpointResponse
                        { EndpointArn = endpointArn });
                _mockNotificationService.Setup(s => s.AddNotificationTokenAsync(userId, deviceToken,
                        endpointArn))
                    .ReturnsAsync(new UserPushToken());

                //Act
                await _notificationWorker.AddNotificationTokenAsync(userId, deviceToken);

                //Assert
                _mockLogger.Verify(
                    x => x.Log(
                        LogLevel.Information,
                        It.IsAny<EventId>(),
                        It.IsAny<It.IsAnyType>(),
                        It.IsAny<Exception>(),
                        (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
                    Times.Exactly(3));
                _mockNotificationService.Verify(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken),
Times.Once);
                _mockNotificationClient.Verify(client => client.CreatePlatformEndpointAsync(
                    It.Is<CreatePlatformEndpointRequest>(req => req.Token == deviceToken),
                    It.IsAny<CancellationToken>()
                ), Times.Once);
                _mockNotificationService.Verify(
                    s => s.AddNotificationTokenAsync(userId, deviceToken,
                        endpointArn), Times.Once);
            }
```

```
            [Fact]
            public async Task TestPath1_2_3_4_5_Exception()
            {

                //Arrange
                var userId = GetValidUserId();
                var deviceToken = GetValidDeviceToken();
                var endpointArn = "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone";
                var exception = new Exception("AddNotificationTokenAsync Exception.");

                _mockNotificationService.Setup(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken));
                _mockNotificationClient.Setup(client =>
                        client.CreatePlatformEndpointAsync(It.IsAny<CreatePlatformEndpointRequest>(),
                            It.IsAny<CancellationToken>()))
                    .ReturnsAsync(new CreatePlatformEndpointResponse
                        { EndpointArn = endpointArn });
                _mockNotificationService.Setup(s => s.AddNotificationTokenAsync(userId, deviceToken,
                        endpointArn))
                    .ThrowsAsync(exception);

                //Act
                await Assert.ThrowsAsync<Exception>(() =>
                    _notificationWorker.AddNotificationTokenAsync(userId, deviceToken));

                //Assert
                _mockLogger.Verify(
                    x => x.Log(
                        LogLevel.Error,
                        It.IsAny<EventId>(),
                        It.IsAny<It.IsAnyType>(),
                        It.IsAny<Exception>(),
                        (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
                    Times.AtLeastOnce);
                _mockNotificationService.Verify(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken),
Times.Once);
                _mockNotificationClient.Verify(client => client.CreatePlatformEndpointAsync(
                    It.Is<CreatePlatformEndpointRequest>(req => req.Token == deviceToken),
                    It.IsAny<CancellationToken>()
                ), Times.Once);
                _mockNotificationService.Verify(
                    s => s.AddNotificationTokenAsync(userId, deviceToken,
                        endpointArn), Times.Once);
            }

            [Fact]
            public async Task TestPath_1_2_3_4()
            {
                //Arrange
                var userId = GetValidUserId();
                var deviceToken = GetValidDeviceToken();
                var exception = new Exception("CreatePlatformEndpoint Exception.");

                _mockNotificationService.Setup(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken));
                _mockNotificationClient.Setup(client =>
                        client.CreatePlatformEndpointAsync(It.IsAny<CreatePlatformEndpointRequest>(),
                            It.IsAny<CancellationToken>()))
                    .ThrowsAsync(exception);

                //Act
                await Assert.ThrowsAsync<Exception>(() =>
                    _notificationWorker.AddNotificationTokenAsync(userId, deviceToken));

                //Assert
                _mockLogger.Verify(
                    x => x.Log(
                        LogLevel.Error,
                        It.IsAny<EventId>(),
                        It.IsAny<It.IsAnyType>(),
                        It.IsAny<Exception>(),
                        (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
                    Times.AtLeastOnce);
                _mockNotificationService.Verify(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken),
Times.Once);
                _mockNotificationClient.Verify(client => client.CreatePlatformEndpointAsync(
                    It.Is<CreatePlatformEndpointRequest>(req => req.Token == deviceToken),
                    It.IsAny<CancellationToken>()
                ), Times.Once);
                _mockNotificationService.Verify(
                    s => s.AddNotificationTokenAsync(userId, deviceToken,
                        It.IsAny<string>()), Times.Never);
            }
```

```
            [Fact]
            public async Task TestPath_1_2_3_6()
            {
                //Arrange
                var userId = GetValidUserId();
                var deviceToken = GetValidDeviceToken();
                var alreadyRegisteredArn = "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone";

                _mockNotificationService.Setup(s => s.GetEndPointArnFromDeviceTokenAsync(deviceToken))
                    .ReturnsAsync(alreadyRegisteredArn);

                //Act
                await _notificationWorker.AddNotificationTokenAsync(userId, deviceToken);

                // Assert
                _mockLogger.Verify(l => l.Log(
                    LogLevel.Information,
                    It.IsAny<EventId>(),
                    It.Is<It.IsAnyType>((state, type) => state.ToString().Contains($"[WORKER] Adding
notification token failed for user {userId}: Device token {deviceToken} was already registered.")),
                    It.IsAny<Exception>(),
                    (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()
                ), Times.Once);
                _mockNotificationService.Verify(
                    s => s.AddNotificationTokenAsync(userId, deviceToken, alreadyRegisteredArn),
                    Times.Never
                );
            }
        }
```

iv. NotificationWorker.SendNotificationAsync

Il metodo SendNotificationAsync è responsabile dell'invio di una notifica push a un utente specifico, memorizzando la notifica e inviandola a tutti i dispositivi registrati dell'utente tramite AWS SNS.

```
public async Task SendNotificationAsync(Guid userId, NotificationDTO notificationDto)
{
    if (userId == Guid.Empty || notificationDto == null)
    {
        _logger.LogError("[WORKER] One or more parameter are empty.");
        throw new Exception($"[WORKER] One or more parameter are empty.");
    }

    try
    {
        _logger.LogInformation($"[WORKER] Storing notification for user {userId}.");
        await _notificationService.AddNotificationAsync(notificationDto, userId);
        _logger.LogInformation($"[WORKER] Notification for user {userId} was stored.");
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, $"[WORKER] Storing notification failed for user {userId}: {ex.Message}");
        return;
    }

    List<string> endpointArns;
    // Retrieve the user's endpoint ARNs from the database
    try
    {
        _logger.LogInformation($"[WORKER] Sending notification to user {userId}.");

        endpointArns = await _notificationService.GetEndPointArnFromUserIdAsync(userId);
        if (!endpointArns.Any())
        {
            _logger.LogWarning($"[WORKER] No endpoints found for user {userId}.");
            return;
        }
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, $"[WORKER] Error in retrieving endpoints for user {userId}:
{ex.Message}");
        return;
    }

    string notificationTitle;

    switch (notificationDto.Type)
    {
        case NotificationType.AuctionExpired:
            notificationTitle = "Auction expired";
            break;
        case NotificationType.AuctionBid:
            notificationTitle = "New Bid";
            break;
        case NotificationType.AuctionClosed:
            notificationTitle = "Auction Closed";
            break;
        default:
            notificationTitle = "Unknown Notification Type";
            _logger.LogWarning($"[WORKER] Unknown notification type: {notificationDto.Type}");
            break;
    }

    var jsonPayload = PushNotificationTemplate.GetPushString(notificationTitle,
notificationDto.Message,
        notificationDto.AuctionTitle, notificationDto.AuctionId);

    foreach (var endpointArn in endpointArns)
        try
        {
            _logger.LogInformation($"[WORKER] Creating Push Configuration for endpoint
{endpointArn}.");
            // Send the notification
            var publishRequest = new PublishRequest
            {
                TargetArn = endpointArn,
                MessageStructure = "json",
                Message = JsonSerializer.Serialize(new
                {
                    APNS = jsonPayload // Wrap the APNs payload in APNS
                })
            };
            await _snsClient.PublishAsync(publishRequest);
            _logger.LogInformation($"[WORKER] Notification sent to endpoint {endpointArn}.");
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, $"[WORKER] Sending notification failed for user {userId}:
{ex.Message}");
        }

    _logger.LogInformation($"[WORKER] Notifications sent to every endpoint for user {userId}.");
}
```

Input:

- **userId**: Un Guid che identifica l'utente a cui inviare la notifica.
- **notificationDto**: Un Data Transfer Object contenente i dati per la creazione di una notifica:
    - Id (Guid): Identificatore univoco della notifica.
    - Type (NotificationType): Tipo di notifica (custom).
    - CreationDate (DateTime): Data e ora di creazione della notifica.
    - Message (string): Messaggio di testo della notifica.
    - MainImageUrl (string): URL dell'immagine principale associata alla notifica.
    - AuctionId (Guid): Identificatore dell'asta associata alla notifica.
    - AuctionTitle (string): Titolo dell'asta associata alla notifica.

Output:

- Il metodo non restituisce un valore esplicito (void).
- Il metodo può lanciare un'eccezione di tipo Exception se uno dei parametri di input è vuoto.

Comportamento:

1. **Validazione dei parametri:**
    - Il metodo verifica che userId non sia vuoto e che notificationDto non sia null.
    - Se uno dei parametri non è valido, viene registrato un errore nel logger e viene lanciata un'eccezione.
2. **Memorizzazione della notifica:**
    - Il metodo utilizza _notificationService.AddNotificationAsync(notificationDto, userId) per memorizzare la notifica nel sistema, associandola all'utente.
    - Viene registrato un messaggio informativo nel logger di avvenuto salvataggio.
    - Se il salvataggio genera un errore, l'errore viene loggato, ed il metodo termina.

3. **Recupero degli endpoint ARN dell'utente:**
    - Il metodo utilizza _notificationService.GetEndPointArnFromUserIdAsync(userId) per recuperare tutti gli endpoint ARN associati all'utente.
    - Se non vengono trovati endpoint, viene registrato un avviso nel logger e il metodo termina.
    - Se il recupero degli endpoint genera un errore, l'errore viene loggato, ed il metodo termina.
4. **Determinazione del titolo della notifica:**
    - Il metodo determina il titolo della notifica in base al tipo di notifica (notificationDto.Type).
    - Viene utilizzato uno statement switch per mappare il tipo di notifica a un titolo predefinito.
    - Se il tipo di notifica è sconosciuto, viene utilizzato un titolo predefinito e viene registrato un avviso nel logger.
5. **Creazione del payload JSON della notifica:**
    - Il metodo utilizza PushNotificationTemplate.GetPushString per creare il payload JSON della notifica, formattando il titolo, il messaggio e i dettagli dell'asta.
    - Il payload JSON è specifico per APNs (Apple Push Notification service).
6. **Invio della notifica a ciascun endpoint ARN:**
    - Il metodo itera attraverso tutti gli endpoint ARN recuperati.
    - Per ciascun endpoint, crea un oggetto PublishRequest per AWS SNS.
    - Il PublishRequest include l'ARN dell'endpoint di destinazione, la struttura del messaggio come JSON e il payload JSON della notifica.
    - Il metodo utilizza _snsClient.PublishAsync per inviare la notifica all'endpoint.
    - Viene registrato un messaggio informativo nel logger per ciascuna notifica inviata.
    - Se l'invio di una notifica genera un errore, l'errore viene registrato nel logger, ma il metodo continua con gli altri endpoint.

7. **Completamento:**
    - viene registrato un messaggio informativo nel logger quando tutte le notifiche sono state inviate.

Dipendenze Esterne:

- **_logger**: Un'istanza di un logger per la registrazione di messaggi informativi e di errore.
- **_notificationService**: Un servizio che fornisce metodi per interagire con il sistema di notifiche (ad esempio, per memorizzare la notifica e recuperare gli endpoint ARN).
- **_snsClient**: Un client per interagire con AWS SNS.
- **PushNotificationTemplate**: Una classe o un servizio che fornisce metodi per formattare il payload JSON delle notifiche push.
- **NotificationDTO**: un Data Transfer Object che contiene le informazioni della notifica.
- **NotificationType**: un enum che definisce i tipi di notifiche.
- **JsonSerializer**: una classe per serializzare oggetti in formato json.

Strategia Black Box

La prima strategia di testing adottata è di tipo **Black Box**. Ciò significa che i test sono stati progettati esclusivamente sulla base delle specifiche del metodo **SendNotificationAsync**(input, output, comportamento atteso), *senza* considerare la sua implementazione interna.

userId : Guid

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
E1	userId è nullo	Non Valido	null	Exception
E2	userId ha un valore che non sia nullo.	Valido	“78afb6a7-2e31-42a7-a758-7732e592dba4”	[]

notificationDto : NotificationDTO

ID Classe di Equivalenza	Descrizione	Valido/Non Valido	Esempio Input	Output Previsto (Errori)
E1	notificationDto è nullo	Non Valido	null	Exception
E2	notificationDto ha valori minimi per i suoi campi	Valido	“Id = 46afc5a7-2e31-42a7-a758-7332e592dba5, Type = NotificationType.AuctionClosed, CreationDate = new DateTime(2025,01,01), Message = "", MainImageUrl = "", AuctionId = 92afc5a7-2e31-42a7-a758-4632e592dba2, AuctionTitle = "" ”	[]
E3	notificationDto ha valori massimi per i suoi campi	Valido	“Id = 46afc5a7-2e31-42a7-a758-7332e592dba5, Type = NotificationType.AuctionBid, CreationDate = new DateTime(2025,01,01), Message = new string('a', 300), MainImageUrl = new string('a', 300), AuctionId = 92afc5a7-2e31-42a7-a758-4632e592dba2, AuctionTitle = new string('a', 300)	[]

			”	
E4	notificationDto ha valori compresi tra il range minimo e massimo per i suoi campi	Valido	“Id = default, Type = NotificationType.AuctionClosed, CreationDate = new DateTime(2025,01,01), Message = "L'asta è terminata.", MainImageUrl = "www.mainimage.com", AuctionId = Guid.NewGuid(), AuctionTitle = "iPhone 12 - Usato" ”	[]

Codice xUnit Test

```
public class BlackBoxTests
{
    private readonly NotificationWorker _notificationWorker;
    private readonly Mock<ILogger<NotificationWorker>> _mockLogger;
    private readonly Mock<INotificationService> _mockNotificationService;
    private readonly Mock<IAmazonSimpleNotificationService> _mockSnsClient;

    public BlackBoxTests()
    {
        _mockLogger = new Mock<ILogger<NotificationWorker>>();
        _mockNotificationService = new Mock<INotificationService>();
        _mockSnsClient = new Mock<IAmazonSimpleNotificationService>();
        _notificationWorker = new NotificationWorker(_mockLogger.Object,
            _mockNotificationService.Object,
            _mockSnsClient.Object);
    }

    [Fact]
    public async Task SendNotificationAsync_TestWithValidInput()
    {
        //Arrange
        var userId = Guid.NewGuid();
        var notificationDto = CreateTestNotificationDto();

        //Act
        var result = _notificationWorker.SendNotificationAsync(userId, notificationDto);
        await result;

        //Assert
        Assert.True(result.IsCompletedSuccessfully);
    }
```

```
    [Fact]
    public async Task SendNotificationAsync_TestWithMinNotificationDto_ValidUserId()
    {
        //Arrange
        var userId = Guid.NewGuid();
        var notificationDto = new NotificationDTO
        {
            Id = default,
            Type = NotificationType.AuctionClosed,
            CreationDate = default,
            Message = "",
            MainImageUrl = "",
            AuctionId = default,
            AuctionTitle = ""
        };

        //Act
        var result = _notificationWorker.SendNotificationAsync(userId, notificationDto);
        await result;

        //Assert
        Assert.True(result.IsCompletedSuccessfully);
    }

    [Fact]
    public async Task SendNotificationAsync_TestWithMaxNotificationDto_ValidUserId()
    {
        //Arrange
        var userId = Guid.NewGuid();
        var notificationDto = new NotificationDTO
        {
            Id = default,
            Type = NotificationType.AuctionBid,
            CreationDate = default,
            Message = new string('a', 300),
            MainImageUrl = new string('a', 300),
            AuctionId = default,
            AuctionTitle = new string('a', 300)
        };

        //Act
        var result = _notificationWorker.SendNotificationAsync(userId, notificationDto);
        await result;

        //Assert
        Assert.True(result.IsCompletedSuccessfully);
    }

    [Fact]
    public async Task SendNotificationAsync_TestWithEmptyUserId_ValidNotificationDto()
    {
        //Arrange
        var userId = Guid.Empty;
        var notificationDto = CreateTestNotificationDto();

        //Act & Assert
        await Assert.ThrowsAsync<Exception>(() =>
            _notificationWorker.SendNotificationAsync(userId, notificationDto));

    }

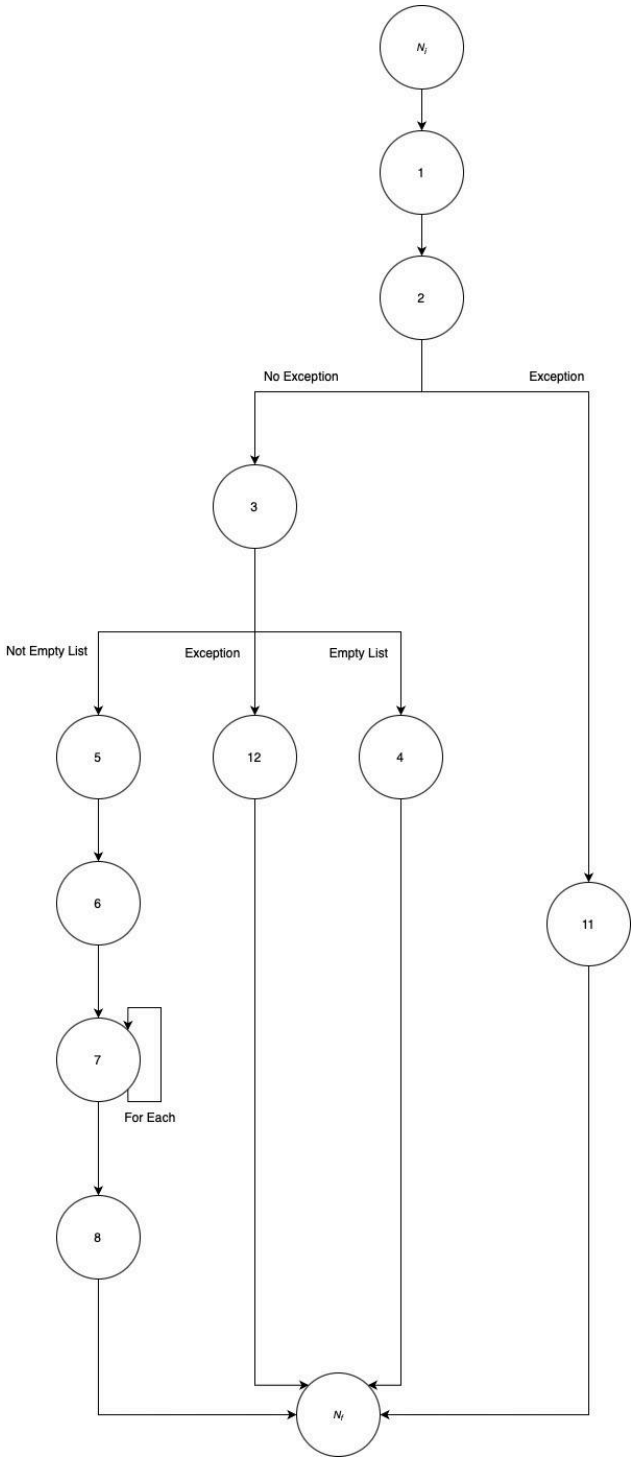
    [Fact]
    public async Task SendNotificationAsync_TestWithNullNotificationDto_ValidUserId()
    {
        //Arrange
        var userId = Guid.NewGuid();
        NotificationDTO notificationDto = null;

        //Act & Assert
        await Assert.ThrowsAsync<Exception>(() =>
            _notificationWorker.SendNotificationAsync(userId, notificationDto));
    }
}
```

Strategia White Box

La strategia di Black Box precedentemente descritta non riesce a coprire l'interezza del metodo precedentemente esaminato, dunque risulta necessaria una strategia White Box, per la quale si fornisce il seguente Grafo dei flussi di controllo e i test sui possibili percorsi di esecuzione.

Grafo



Codice xUnit White Box Test

```
public class WhiteBoxTests
{
    private readonly NotificationWorker _notificationWorker;
    private readonly Mock<ILogger<NotificationWorker>> _mockLogger;
    private readonly Mock<INotificationService> _mockNotificationService;
    private readonly Mock<IAmazonSimpleNotificationService> _mockSnsClient;

    public WhiteBoxTests()
    {
        _mockLogger = new Mock<ILogger<NotificationWorker>>();
        _mockNotificationService = new Mock<INotificationService>();
        _mockSnsClient = new Mock<IAmazonSimpleNotificationService>();
        _notificationWorker = new NotificationWorker(_mockLogger.Object,
            _mockNotificationService.Object,
            _mockSnsClient.Object);
    }

    [Fact]
    public async Task TestPath_1_2_3_5_6_7_8()
    {
        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();
        var endpointArns = new List<string>
        {
            "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone",
            "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad"
        };

        _mockNotificationService.Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ReturnsAsync(new Notification());
        _mockNotificationService.Setup(s => s.GetEndPointArnFromUserIdAsync(userId))
            .ReturnsAsync(endpointArns);
        _mockSnsClient.Setup(s => s.PublishAsync(It.IsAny<PublishRequest>(), default))
            .ReturnsAsync(new PublishResponse());

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        // Verifica che il logger sia stato chiamato un numero di volte compreso in un intervallo
        // 1 ad inizio AddNotificationAsync, 1 alla fine, 1 quanod inizia ad inviare la notifica, 1
    //dopo la fine dell'invio di notifica.
        // 2 per ogni endpoint.
        int expectedInformationLogs = 4 + (endpointArns.Count * 2);

        _mockLogger.Verify(
            x => x.Log(
                LogLevel.Information,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.Between(5, expectedInformationLogs, Range.Inclusive));
        _mockLogger.Verify(
            x => x.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.Never);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Once);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default),
            Times.Exactly(endpointArns.Count));
    }
```

```
    [Fact]
    public async Task TestPath_1_2_10_8()
    {
        //Arrange
        Guid userId = Guid.Empty;
        NotificationDTO notificationDto = null;
        var exception = new Exception("One or more parameter is empty.");

        // Act
        await Assert.ThrowsAsync<Exception>(() =>
            _notificationWorker.SendNotificationAsync(userId, notificationDto));

        // Assert
        _mockLogger.Verify(l => l.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.Once);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Never);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Never);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default), Times.Never);
    }

    [Fact]
    public async Task TestPath_1_2_11_8()
    {
        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();
        var exception = new Exception("Add notification failed");

        _mockNotificationService
            .Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ThrowsAsync(exception);

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        _mockLogger.Verify(l => l.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.AtLeastOnce);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Never);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default), Times.Never);
    }
```

```
    [Fact]
    public async Task TestPath_1_2_3_12_8()
    {

        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();
        var exception = new Exception("Get endpoints failed");

        _mockNotificationService.Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ReturnsAsync(new Notification());
        _mockNotificationService.Setup(s => s.GetEndPointArnFromUserIdAsync(userId))
            .ThrowsAsync(exception);

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        _mockLogger.Verify(l => l.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.AtLeastOnce);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Once);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default), Times.Never);
    }

    [Fact]
    public async Task TestPath_1_2_3_4_8()
    {

        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();

        _mockNotificationService.Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ReturnsAsync(new Notification());

        _mockNotificationService.Setup(s => s.GetEndPointArnFromUserIdAsync(userId))
            .ReturnsAsync(new List<string>());

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        _mockLogger.Verify(l => l.Log(
                LogLevel.Warning,
                It.IsAny<EventId>(),
                It.Is<It.IsAnyType>((state, type) => state.ToString().Contains("No endpoints found")),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.Once);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Once);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default), Times.Never);
    }
```

```
    [Fact]
    public async Task TestPath_1_2_3_4_8()
    {

        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();

        _mockNotificationService.Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ReturnsAsync(new Notification());

        _mockNotificationService.Setup(s => s.GetEndPointArnFromUserIdAsync(userId))
            .ReturnsAsync(new List<string>());

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        _mockLogger.Verify(l => l.Log(
                LogLevel.Warning,
                It.IsAny<EventId>(),
                It.Is<It.IsAnyType>((state, type) => state.ToString().Contains("No endpoints found")),
                It.IsAny<Exception>(),
                (Func<It.IsAnyType, Exception, string>)It.IsAny<object>()),
            Times.Once);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Once);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default), Times.Never);
    }

    [Fact]
    public async Task TestPathWithExceptionOnPublish_1_2_3_5_6_7_8()
    {
        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();
        var endpointArns = new List<string> { "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone"
};
        var exception = new Exception("Publish failed");

        _mockNotificationService.Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ReturnsAsync(new Notification());
        _mockNotificationService.Setup(s => s.GetEndPointArnFromUserIdAsync(userId))
            .ReturnsAsync(endpointArns);
        _mockSnsClient.Setup(s => s.PublishAsync(It.IsAny<PublishRequest>(), default))
            .ThrowsAsync(exception);

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        _mockLogger.Verify(
            x => x.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                exception,
                It.IsAny<Func<It.IsAnyType, Exception, string>>()),
            Times.Once);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Once);
        _mockSnsClient.Verify(s => s.PublishAsync(It.IsAny<PublishRequest>(), default),
            Times.Once);
    }
```

```
    [Fact]
    public async Task TestPathPublishFailsForOneEndpoint_1_2_3_5_6_7_8()
    {
        // Arrange
        var userId = GetValidUserId();
        var notificationDto = CreateTestNotificationDto();
        var endpointArns = new List<string>
        {
            "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone",
            "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad",
            "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone2"
        };
        var exception =
            new Exception("Publish failed for arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad");

        _mockNotificationService
            .Setup(s => s.AddNotificationAsync(notificationDto, userId))
            .ReturnsAsync(new Notification());
        _mockNotificationService
            .Setup(s => s.GetEndPointArnFromUserIdAsync(userId))
            .ReturnsAsync(endpointArns);
        // Successo per tutti gli endpoint eccetto arn:endpoint2
        _mockSnsClient
            .Setup(s => s.PublishAsync(
                It.Is<PublishRequest>(p =>
                    p.TargetArn != "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad"), default))
            .ReturnsAsync(new PublishResponse());
        // Setup per "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad"
        _mockSnsClient
            .Setup(x => x.PublishAsync(
                It.Is<PublishRequest>(p =>
                    p.TargetArn == "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad"), default))
            .ThrowsAsync(exception);

        // Act
        await _notificationWorker.SendNotificationAsync(userId, notificationDto);

        // Assert
        _mockLogger.Verify(
            x => x.Log(
                LogLevel.Error,
                It.IsAny<EventId>(),
                It.IsAny<It.IsAnyType>(),
                It.Is<Exception>(ex =>
                    ex.Message.Contains(
                        "Publish failed for arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad")),
                It.IsAny<Func<It.IsAnyType, Exception, string>>()),
            Times.Once);
        _mockNotificationService.Verify(s => s.AddNotificationAsync(notificationDto, userId),
Times.Once);
        _mockNotificationService.Verify(s => s.GetEndPointArnFromUserIdAsync(userId), Times.Once);
        _mockSnsClient.Verify(x => x.PublishAsync(It.Is<PublishRequest>(p =>
                p.TargetArn == "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone"
                || p.TargetArn == "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/iphone2"), default),

            Times.Exactly(2));
        _mockSnsClient.Verify(
            x => x.PublishAsync(
                It.Is<PublishRequest>(p =>
                    p.TargetArn == "arn:aws:sns:eu-west-1:123150819972:endpoint/APNS/ipad"), default),
            Times.Once);
    }
}
```